

MUSICMATCH



Sergio Sánchez Niño

PROYECTO INTERMODULAR DE DESARROLLO DE APLICACIONES MULTIPLATAFORMA (0492)

Desarrollo de Aplicaciones
Multiplataforma 2025-2026

Tutor: Jesús Arcis Díaz

I.E.S. Castillo de Luna

RESUMEN

El presente proyecto consiste en el desarrollo de MusicMatch, una aplicación móvil orientada a conectar músicos mediante criterios de afinidad musical. El problema que aborda es la dificultad que tienen muchos músicos, especialmente en el ámbito local y amateur, para encontrar personas compatibles con las que colaborar, formar una banda o participar en nuevos proyectos musicales. Aunque existen redes sociales y plataformas musicales, muchas de ellas no están diseñadas específicamente para filtrar usuarios por instrumento, estilo musical, nivel de experiencia o ubicación.

Como solución, se propone un prototipo funcional basado en una arquitectura cliente-servidor. La aplicación móvil permite al usuario registrarse, verificar su cuenta, configurar un perfil musical, explorar otros perfiles, recibir coincidencias y comunicarse mediante mensajería privada. El backend centraliza la lógica de negocio, la autenticación, la gestión de perfiles, el sistema de matching, los bloqueos y el almacenamiento de mensajes.

Las principales tecnologías utilizadas han sido Kotlin, Jetpack Compose y Android Studio para el desarrollo de la aplicación móvil, y Java, Spring Boot, Spring Security, JPA/Hibernate y MySQL para el backend y la persistencia de datos. También se han empleado herramientas como Docker, Git/GitHub, Postman, Trello y Railway para la organización, pruebas y despliegue del proyecto.

Como resultado, se ha obtenido un MVP funcional que permite validar la viabilidad técnica de MusicMatch y demostrar que la propuesta puede servir como base para una plataforma más completa de conexión entre músicos.

PALABRAS CLAVE

MusicMatch, aplicación móvil, músicos, matching, Android, Spring Boot, MySQL.

ABSTRACT

This project presents the development of MusicMatch, a mobile application aimed at connecting musicians through musical affinity criteria. The problem addressed is the difficulty many musicians, especially in local and amateur contexts, face when trying to find compatible people to collaborate with, form a band or take part in new musical projects. Although social networks and music platforms already exist, many of them are not specifically designed to filter users by instrument, music style, experience level or location.

As a solution, a functional prototype based on a client-server architecture is proposed. The mobile application allows users to register, verify their account, configure a musical profile, explore other profiles, receive matches and communicate through private messaging. The backend centralizes the business logic, authentication, profile management, matching system, user blocking and message storage.

The main technologies used were Kotlin, Jetpack Compose and Android Studio for the mobile application and Java, Spring Boot, Spring Security, JPA/Hibernate and MySQL for the backend and data persistence. Tools such as Docker, Git/GitHub, Postman, Trello and Railway were also used for organization, testing and deployment.

As a result, a functional MVP has been obtained, making it possible to validate the technical feasibility of MusicMatch and demonstrate that the proposal can serve as a solid foundation for a more complete platform for connecting musicians.

KEYWORDS

MusicMatch, mobile application, musicians, matching, Android, Spring Boot, MySQL.

ÍNDICE

1. INTRODUCCIÓN DEL PROYECTO.....	9
1.1 Contexto del proyecto.....	10
1.2 Identificación del problema.....	11
1.3 Alcance del proyecto.....	12
1.4 Objetivos del proyecto.....	13
1.5 Entregables y estructura de la memoria.....	14
2. MARCO TEÓRICO Y ESTUDIO DEL SECTOR.....	15
2.1 Marco conceptual y tecnológico.....	16
2.2 Ecosistema TIC relacionado con el proyecto.....	17
2.2.1. Redes sociales generalistas.....	17
2.2.2. Plataformas digitales orientadas a la música.....	18
2.2.3. Aplicaciones específicas de conexión entre músicos.....	19
2.2.4. Sistemas de matching y recomendación.....	20
2.2.5. Posicionamiento del proyecto dentro del ecosistema TIC.....	21
2.3. Necesidades reales del mercado.....	22
2.4. Oportunidades y líneas de negocio.....	23
2.5. Tipo de proyecto requerido.....	24
2.6. Caracterización general de la solución.....	25
2.6.1. Usuarios objetivo.....	25
2.6.2. Funcionalidades clave.....	25
2.6.3. Requisitos esenciales.....	26
2.6.4. Restricciones y prioridades.....	26
2.7. Empresas representativas y modelos organizativos.....	27
2.7.1. Empresas representativas.....	27
2.7.2. Modelos organizativos.....	28
3. ANÁLISIS DE USUARIO Y PROPUESTA DE VALOR.....	29
3.1 Perfiles de usuario.....	30
3.1.1. Buyer Persona 1 - Músico aficionado.....	30
3.1.2. Buyer Persona 2 - Músico con experiencia.....	31
3.1.3. Buyer Persona 3 - Administrador.....	32
3.2 Tareas clave de los usuarios.....	33
3.3 Propuesta de valor.....	34
3.4 Validación inicial de la idea.....	35
4. ESPECIFICACIÓN DE REQUISITOS.....	36
4.1 Historias de usuario.....	37
4.2 Requisitos funcionales.....	38
4.3 Requisitos no funcionales.....	39
4.4 Diagramas de casos de uso.....	40

5. DISEÑO DE LA SOLUCIÓN.....	41
5.1 Diseño UX (experiencia de usuario).....	41
5.1.1. Principios de diseño.....	41
5.1.2. Flujo principal de navegación.....	42
5.1.3. Estructura y diseño.....	44
5.1.4. Experiencia emocional del usuario.....	44
5.1.5. Wireframing y prototipado.....	45
5.2 Diseño UI (interfaz).....	46
5.2.1. Identidad visual de la solución.....	47
5.2.2. Tipografía y fuentes.....	48
5.2.3. Ventana de registro.....	49
5.2.4. Ventana de configuración de perfil musical.....	50
5.2.5. Ventana de inicio.....	51
5.2.6. Ventana de matching.....	52
5.2.7. Ventana de chat.....	53
5.2.8. Ventana de edición de perfil.....	54
5.3 Arquitectura del sistema.....	56
5.3.1. Visión general de la arquitectura.....	56
5.3.2. Arquitectura cliente-servidor.....	57
5.3.3. Arquitectura interna del backend.....	58
5.3.4. Arquitectura interna del cliente móvil.....	59
5.3.5. Flujo general de funcionamiento del sistema.....	60
5.4 Stack tecnológico.....	61
5.4.1. Cliente móvil.....	61
5.4.2. Servidor (Backend).....	62
5.4.3. Base de datos.....	62
5.5 Modelo de datos.....	64
6. PLANIFICACIÓN Y GESTIÓN DEL PROYECTO.....	75
6.1 Metodología de trabajo.....	76
6.2 Product Backlog.....	77
6.3 Fases / Sprints del proyecto.....	78
6.4 Recursos por fase.....	80
6.5 Criterios de aceptación.....	81
6.6 Cronograma general.....	83
7. GESTIÓN DE RIESGOS, SEGURIDAD Y CUMPLIMIENTO.....	84
7.1 Identificación de riesgos.....	85
7.2 Evaluación y mitigación.....	86
7.3 Seguridad y protección de datos.....	87
7.4 Licencias y sostenibilidad.....	89
7.5 Consideraciones éticas y accesibilidad.....	90
8. DESARROLLO DEL PROTOTIPO.....	91
8.1 Preparación del entorno de desarrollo.....	91
8.2 Configuración del repositorio.....	92
8.2.1. Estructura del proyecto.....	92

8.2.2. Control de versiones.....	93
8.3 Desarrollo del backend.....	94
8.3.1. Creación del proyecto Spring Boot.....	94
8.3.2. Configuración de MySQL.....	95
8.3.3. Implementación de la capa de configuración.....	96
8.3.4. Implementación de entidades.....	98
8.3.5. Implementación de repositorios.....	99
8.3.6. Implementación de servicios.....	101
8.3.7. Implementación de DTOs.....	106
8.3.8. Implementación de la API REST.....	107
8.3.9 Gestión de excepciones.....	108
8.4 Desarrollo de la aplicación móvil.....	109
8.4.1. Creación del proyecto Android Studio.....	109
8.4.2. Desarrollo de las ventanas principales.....	109
8.5 Integración cliente-servidor.....	113
8.6 Resultado del prototipo.....	116
9. DESPLIEGUE E INSTALACIÓN.....	117
9.1 Requisitos previos.....	117
9.2 Proceso de despliegue.....	118
9.2.1 Despliegue en Railway.....	118
9.2.2 Generación de release con CI/CD.....	119
9.3 Instalación y puesta en marcha.....	120
9.4 Manual de uso básico.....	122
10. PRUEBAS, VALIDACIÓN Y CONTROL.....	129
10.1 Plan de pruebas.....	130
10.2 Resultados de las pruebas.....	132
10.2.1 Pruebas automatizadas del backend.....	132
10.2.2 Pruebas de la API REST con Postman.....	133
10.2.3 Pruebas funcionales del cliente móvil.....	136
10.3 Gestión de incidencias y cambios.....	140
10.4 Validación con usuarios y verificación de requisitos.....	141
10.5 Limitaciones detectadas durante las pruebas.....	143
11. CONCLUSIONES Y TRABAJO FUTURO.....	144
11.1 Conclusiones generales.....	144
11.2 Valoración personal.....	145
11.3 Líneas futuras de desarrollo.....	146
12. GLOSARIO.....	148
13. REFERENCIAS.....	154

ÍNDICE DE FIGURAS

Figura 2.1 - Reseña sobre sugerencias de Instagram.....	17
Figura 2.2 - Reseña sobre sistema de recomendación de SoundCloud.....	18
Figura 2.3 - Reseña sobre limitaciones de versión gratuita de BandMix.....	19
Figura 2.4 - Reseña sobre filtros y edición de perfil en BandMix.....	19
Figura 2.5 - Reseña sobre poca relevancia de algunos resultados en LinkedIn.....	20
Figura 3.1 - Sketch inicial de la aplicación.....	35
Figura 4.1 - Diagrama de casos de uso.....	40
Figura 5.1 - Wireframing del diseño de las ventanas principales.....	45
Figura 5.2 - Mockup de ventana de registro en Figma.....	49
Figura 5.3 - Mockup de ventana de creación de perfil en Figma.....	50
Figura 5.4 - Mockup de ventana de inicio en Figma.....	51
Figura 5.5 - Mockup de ventana de matches en Figma.....	52
Figura 5.6 - Mockup de ventana de chat en Figma.....	53
Figura 5.7 - Mockup de ventana de edición de perfil.....	55
Figura 5.8 - Diagrama entidad-relación de la solución.....	73
Figura 6.1 - Tablero del Sprint 1 en Trello.....	76
Figura 6.2 - Versión inicial del repositorio de GitHub.....	81
Figura 6.3 - Diagrama de Gantt del proyecto.....	83
Figura 8.1 - Contenedor con servicio MySQL en Docker Desktop.....	92
Figura 8.2 - Estructura del proyecto en GitHub.....	93
Figura 8.3 - Estructura de ramas del repositorio.....	93
Figura 8.4 - Inicialización del backend con Spring Boot Initializr.....	94
Figura 8.5 - Configuración del archivo application.yaml.....	95
Figura 8.6 - Seguridad de endpoints y cifrado de contraseña.....	96
Figura 8.7 - Definición de la clase Profile.....	98
Figura 8.8 - Definición del repositorio para consultas de matches.....	100
Figura 8.9 - Método para la verificación de correo electrónico.....	102
Figura 8.10 - Servicio de envío de enlace por SMTP.....	103
Figura 8.11 - Método de generación de puntuación de afinidad.....	105
Figura 8.12 - Clase genérica para respuestas del servidor.....	106
Figura 8.13 - Controlador de endpoints de autenticación.....	107
Figura 8.14 - Clase global de gestión de excepciones del backend.....	108
Figura 8.15 - Estructura de carpetas del cliente Android.....	109
Figura 8.16 - Pantalla de carga de la aplicación.....	110
Figura 8.17 - Definición de la ventana de registro con Jetpack Compose.....	111
Figura 8.18 - Demostración de coincidencia generada en ventana de matching.....	112
Figura 8.19 - Demostración de ventana de chat.....	113
Figura 8.20 - Definición de la clase ApiClient con retrofit.....	114

Figura 8.21 - Definición de clase ApiService con peticiones REST.....	115
Figura 9.1 - Servicio backend y MySQL funcionando desde Railway.....	118
Figura 9.2 - Ejecución del flujo CI/CD en GitHub Actions.....	119
Figura 9.3 - Carga de imagen del servicio del backend en docker.....	120
Figura 9.4 - Creación de servicios con docker compose.....	120
Figura 9.5 - Ventana de configuración de dirección ip en cliente móvil.....	121
Figura 9.6 - Ejecución de endpoint ping en powershell.....	121
Figura 9.7 - Demostración de verificación de correo con SMTP.....	122
Figura 9.8 - Ventanas de configuración de perfil musical en cliente Android.....	123
Figura 9.9 - Demostración de aceptación de swipe y generación de chat.....	124
Figura 9.10 - Ventanas de restablecimiento de contraseña.....	125
Figura 9.11 - Estructura de carpetas del backend.....	126
Figura 9.12 - Prueba de endpoint protegido desde postman.....	127
Figura 9.13 - Estructura de tablas vista desde el servicio MySQL en Railway.....	127
Figura 9.14 - Diagrama de arquitectura y ejecución de la aplicación.....	128
Figura 10.1 - Prueba de ejecución del perfil de pruebas del backend.....	131
Figura 10.2 - Demostración de éxito de pruebas de JUnit.....	133
Figura 10.3 - Prueba de endpoint de login con postman.....	135
Figura 10.4 - Prueba de uso de la cabecera Authorization en postman.....	135
Figura 10.5 - Obtención de candidatos del usuario desde Postman.....	136
Figura 10.6 - Prueba de modificación de perfil desde la aplicación.....	138
Figura 10.7 - Prueba de uso de chat en tiempo real con otro usuario.....	139
Figura 10.8 - Historial de commits y resolución de incidencias clave.....	141

ÍNDICE DE TABLAS

Tabla 4.1 - Requisitos no funcionales de la aplicación.....	39
Tabla 5.1 - Paleta de colores de la solución.....	47
Tabla 5.2 - Tamaño fuente de los estilos de texto.....	48
Tabla 7.1 - Principales riesgos de la solución.....	85
Tabla 7.2 - Evaluación y mitigación de riesgos.....	86
Tabla 7.3 - Licencias del stack tecnológico utilizado.....	89
Tabla 8.1 - Criterios de afinidad del sistema de matching.....	104
Tabla 10.1 - Plan de pruebas y herramientas utilizadas.....	130
Tabla 10.2 - Resultados de las pruebas del backend.....	132
Tabla 10.3 - Estructura de pruebas de la Api REST.....	134
Tabla 10.4 - Resultado de pruebas funcionales en el cliente Android.....	137
Tabla 10.5 - Incidencias y soluciones aplicadas.....	140
Tabla 10.6 - Verificación de cumplimiento de requisitos funcionales.....	142

1. INTRODUCCIÓN DEL PROYECTO

El presente proyecto consiste en el desarrollo de MusicMatch, una aplicación móvil orientada a facilitar la conexión entre músicos a partir de criterios de afinidad musical. La idea principal nace de una necesidad frecuente dentro del ámbito musical, especialmente entre músicos aficionados, estudiantes o personas que buscan formar parte de un proyecto musical, encontrar otros músicos compatibles con los que colaborar, ensayar, formar una banda o iniciar nuevas propuestas creativas.

La solución planteada se desarrolla dentro del contexto del Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma, por lo que no solo se persigue crear una aplicación funcional, sino también aplicar de forma integrada los conocimientos adquiridos en distintos módulos del ciclo. A través del proyecto se trabajan aspectos relacionados con el análisis de requisitos, diseño de interfaces, programación móvil, desarrollo backend, bases de datos, seguridad, despliegue, pruebas y documentación técnica.

Desde el punto de vista funcional, MusicMatch se plantea como una aplicación móvil con arquitectura cliente-servidor. El cliente móvil permite al usuario registrarse, configurar su perfil musical, explorar otros perfiles, recibir coincidencias y comunicarse con otros usuarios. El servidor, por su parte, centraliza la lógica de negocio, la gestión de datos, la autenticación y el sistema de coincidencias entre músicos. De esta forma, el proyecto combina una necesidad real del sector musical con una solución tecnológica alineada con el perfil profesional de un desarrollador multiplataforma.

1.1 Contexto del proyecto

La música forma parte de uno de los sectores creativos con mayor capacidad de generar relaciones, proyectos colaborativos y comunidades. Muchos músicos no desarrollan su actividad de forma aislada, sino que necesitan contactar con otras personas para formar bandas, participar en ensayos, crear canciones, organizar actuaciones o simplemente compartir intereses musicales.

En los últimos años, la digitalización ha transformado profundamente la forma en la que las personas se comunican, consumen contenidos y establecen relaciones. Las plataformas digitales han permitido que artistas y músicos puedan compartir su trabajo, darse a conocer y contactar con otras personas de una manera más rápida que mediante métodos tradicionales. Sin embargo, muchas de estas herramientas no están pensadas específicamente para resolver la búsqueda de músicos compatibles.

Las redes sociales generalistas permiten publicar contenido y contactar con otros usuarios, pero no ofrecen un sistema especializado para filtrar por instrumento, estilo musical, nivel de experiencia o ubicación. Del mismo modo, algunas plataformas musicales están más centradas en la difusión de canciones o perfiles artísticos que en facilitar la creación de colaboraciones reales entre músicos. Por ello, sigue existiendo una necesidad concreta de disponer de una herramienta sencilla, directa y especializada en conectar músicos con intereses compatibles.

En este contexto surge MusicMatch, una aplicación que combina elementos propios de las redes sociales, los sistemas de recomendación y las aplicaciones móviles. Su finalidad es facilitar el contacto entre músicos mediante perfiles estructurados y criterios de afinidad musical, permitiendo que el usuario pueda descubrir personas con gustos, instrumentos y objetivos similares.

Además, el proyecto se enmarca en un contexto académico. Como proyecto final del ciclo de Desarrollo de Aplicaciones Multiplataforma, MusicMatch permite aplicar conocimientos técnicos y metodológicos de forma global. No se trata únicamente de programar una aplicación, sino de llevar a cabo un proceso completo de desarrollo software desde el análisis de una necesidad y la definición de requisitos hasta su implementación y documentación.

1.2 Identificación del problema

El problema principal que aborda este proyecto es la dificultad que tienen muchos músicos para encontrar a otras personas compatibles con las que colaborar. Aunque actualmente existen múltiples canales digitales de comunicación, la búsqueda de músicos sigue dependiendo en gran medida de redes personales, publicaciones en redes sociales, anuncios en locales de ensayo o recomendaciones informales.

Estos métodos presentan varias limitaciones. En primer lugar, no garantizan que las personas encontradas compartan los mismos intereses musicales, nivel de experiencia, disponibilidad o ubicación. En segundo lugar, requieren una búsqueda manual que puede resultar lenta y poco eficaz. Por último, no siempre permiten estructurar la información más importante para un músico, como los instrumentos que toca, los estilos que le interesan o el tipo de proyecto que busca.

Las redes sociales generalistas tampoco resuelven completamente esta necesidad. Aunque permiten contactar con otras personas, no están diseñadas para realizar un matching musical real. Sus sistemas de búsqueda suelen estar orientados a nombres, etiquetas, popularidad o contenido publicado, pero no a criterios específicos de compatibilidad musical.

Por este motivo, se identifica la necesidad de una aplicación especializada que permita crear perfiles musicales y generar coincidencias entre usuarios teniendo en cuenta información relevante, como instrumentos, estilos, nivel de experiencia y localización. De esta forma, el proceso de búsqueda se vuelve más organizado, eficiente y adaptado a las necesidades reales de los músicos.

1.3 Alcance del proyecto

El alcance del proyecto se centra en el desarrollo de un MVP, es decir, un producto mínimo viable que permita demostrar la viabilidad técnica y funcional de la idea. El objetivo no es crear una versión comercial completamente finalizada, sino una base funcional que recoja las características esenciales de la aplicación.

Dentro del alcance del proyecto se incluye el desarrollo de una aplicación móvil para Android, conectada a un backend mediante una API REST. El sistema permite gestionar usuarios, registrar cuentas, iniciar sesión, configurar perfiles musicales, editar información personal, explorar otros usuarios, generar coincidencias entre músicos y permitir una comunicación básica mediante mensajería privada.

También forma parte del alcance la creación de una base de datos relacional que permita almacenar la información de los usuarios, sus perfiles musicales, instrumentos, estilos, coincidencias y mensajes. Además, se contempla la implementación de medidas básicas de seguridad, como autenticación mediante tokens, cifrado de contraseñas y control de acceso a los recursos protegidos.

Por otro lado, quedan fuera del alcance inicial algunas funcionalidades avanzadas que podrían incorporarse en versiones futuras. Entre ellas se encuentran la mensajería en tiempo real mediante WebSockets, la gestión completa de grupos musicales, los algoritmos de matching basados en inteligencia artificial avanzada, las integraciones con servicios externos, el sistema completo de notificaciones push y un panel de administración totalmente desarrollado.

Esta delimitación permite ajustar el proyecto al tiempo y recursos disponibles dentro del contexto académico, priorizando las funcionalidades esenciales y garantizando que el resultado final sea coherente, funcional y evaluable.

1.4 Objetivos del proyecto

El objetivo general del proyecto es desarrollar una aplicación móvil que facilite la conexión entre músicos mediante un sistema de perfiles y coincidencias basado en criterios de afinidad musical.

A partir de este objetivo general, se establecen los siguientes objetivos específicos:

- Analizar las necesidades de los músicos que buscan colaborar, formar bandas o participar en proyectos musicales.
- Diseñar una solución móvil sencilla, intuitiva y adaptada al perfil de usuario objetivo.
- Definir los requisitos funcionales y no funcionales necesarios para el correcto funcionamiento del sistema.
- Diseñar la arquitectura de la aplicación, separando la parte cliente, el backend y la base de datos.
- Implementar un sistema de registro, inicio de sesión y gestión de usuarios de forma segura.
- Desarrollar perfiles musicales que incluyan información relevante como instrumentos, estilos, nivel de experiencia, ciudad y biografía.
- Implementar un sistema de matching que permita generar coincidencias entre usuarios según criterios musicales.
- Permitir la comunicación entre usuarios compatibles mediante un sistema básico de mensajería.
- Desplegar el backend y preparar la aplicación para su ejecución y validación.
- Realizar pruebas funcionales y técnicas que permitan comprobar el correcto funcionamiento del prototipo.
- Elaborar una documentación completa que recoja el análisis, diseño, desarrollo, pruebas, despliegue y conclusiones del proyecto.

Desde el punto de vista formativo, el proyecto también tiene como objetivo integrar contenidos propios del ciclo de Desarrollo de Aplicaciones Multiplataforma. En el desarrollo de MusicMatch se aplican conocimientos relacionados con programación, bases de datos, acceso a datos, desarrollo de interfaces, programación móvil, servicios en red, seguridad, despliegue y control de versiones. Por tanto, el proyecto permite demostrar una visión global del proceso de creación de una aplicación multiplataforma.

1.5 Entregables y estructura de la memoria

Los principales entregables del proyecto son la aplicación móvil, el backend, la base de datos, el repositorio del código fuente, la documentación técnica, las pruebas realizadas y la presente memoria. Todos estos elementos permiten justificar el proceso seguido y demostrar el funcionamiento del prototipo desarrollado.

La memoria se estructura de forma progresiva, siguiendo las fases habituales de un proyecto software. En primer lugar, se presenta la introducción del proyecto, donde se describe el contexto, el problema detectado, el alcance y los objetivos. A continuación, se desarrolla el marco teórico y el estudio del sector, con el fin de situar la propuesta dentro del ámbito tecnológico y musical.

Posteriormente, se realiza el análisis de usuario y la propuesta de valor, donde se identifican los perfiles principales de usuarios y las necesidades que debe cubrir la aplicación. Después, se definen los requisitos funcionales y no funcionales, que sirven como base para el diseño y desarrollo del sistema.

En los siguientes apartados se presenta el diseño de la solución, incluyendo la experiencia de usuario, la interfaz, la arquitectura, el stack tecnológico y el modelo de datos. A continuación, se recoge la planificación del proyecto, la gestión de riesgos, las medidas de seguridad y el proceso de desarrollo del prototipo.

Finalmente, la memoria incluye el despliegue e instalación, el plan de pruebas, la validación del sistema y las conclusiones finales. Esta estructura permite mostrar de forma ordenada todo el proceso seguido, desde la identificación de la necesidad inicial hasta la obtención de un prototipo funcional y sus posibles líneas de evolución futura.

2. MARCO TEÓRICO Y ESTUDIO DEL SECTOR

Este capítulo está centrado en la contextualización del proyecto dentro del sector tecnológico y musical a través del análisis de los conceptos clave, el ecosistema TIC relacionado y las necesidades reales del mercado a las que está orientada esta propuesta.

Este marco teórico permite demostrar que este proyecto responde a un problema real dentro del ámbito musical amateur, es decir, a la dificultad de los músicos para encontrar músicos compatibles con los que poder colaborar o formar una banda. Para ello se ha realizado un análisis de las plataformas digitales existentes que están relacionadas con el ámbito musical y la interacción social, identificando sus principales limitaciones y justificando las ventajas que ofrece el desarrollo de una solución que se especialice en la conexión entre músicos mediante criterios de afinidad musical.

2.1 Marco conceptual y tecnológico

El presente proyecto se sitúa en el marco de las aplicaciones sociales digitales, más concretamente en aquellas orientadas a la conexión de personas a partir de intereses comunes, también conocidas habitualmente como aplicaciones de matchmaking. Este tipo de aplicaciones se caracterizan principalmente por permitir la creación de perfiles de usuario, la interacción entre los miembros y la generación de conexiones entre personas afines.

En el contexto de este proyecto, se sitúa el foco en el sector musical, donde el objetivo principal consiste en facilitar el contacto entre músicos que desean entablar colaboraciones, formar bandas o participar en proyectos musicales conjuntos. Para ello, la aplicación propuesta utiliza el concepto de matchmaking, que consiste en crear coincidencias entre usuarios a partir de determinados criterios de compatibilidad.

Partiendo de que la aplicación está orientada a la conexión de músicos afines, estos criterios de matching incluyen aspectos como los instrumentos que interpreta cada usuario, los estilos musicales o el nivel de experiencia. La utilización de estos datos permite generar coincidencias entre perfiles musicales que presentan afinidad, facilitando el proceso de encontrar colaboradores adecuados.

Desde el punto de vista técnico, este tipo de aplicaciones suelen construirse sobre plataformas digitales que gestionan una comunidad de usuarios, centralizando la información en un sistema común.

En este sentido, el sistema se basa en una arquitectura cliente-servidor, en la que se distinguen conceptualmente un componente de interacción con el usuario (cliente) y un componente encargado de gestionar la información y la lógica del sistema (servidor).

Para este proyecto, este modelo conceptual sirve de base para la creación de perfiles musicales y para la implementación de un sistema de coincidencia que conecte músicos compatibles.

2.2 Ecosistema TIC relacionado con el proyecto

El proyecto se enmarca dentro de un ecosistema TIC en el que conviven varios tipos de plataformas digitales orientadas a la interacción social, la música y los sistemas de emparejamiento entre usuarios, es por ello, que para contextualizar la propuesta dentro del sector, resulta necesario analizar los principales tipos de aplicaciones existentes y su relación con el propósito del proyecto.

Analizar este tipo de soluciones y sus elementos permite identificar qué funcionalidades son relevantes para el desarrollo de una aplicación orientada a la conexión e interacción entre músicos, además de definir los problemas y limitaciones que presentan las soluciones ya existentes en el mercado.

2.2.1. Redes sociales generalistas

Redes sociales como Instagram o Facebook son ampliamente utilizadas por músicos para darse a conocer y establecer contactos con otros artistas. Estas plataformas permiten crear perfiles, interactuar mediante publicaciones y mensajes, y formar parte de comunidades musicales.

No obstante, presentan limitaciones importantes para la búsqueda de músicos compatibles ya que la visibilidad se centra en la popularidad o interacciones genéricas, sin ningún tipo de filtro relacionado con estilos o instrumentos musicales. Como resultado, muchos músicos, principalmente los más nuevos o aficionados pierden mucho tiempo en búsquedas poco efectivas o contactos que no cumplen con sus expectativas.

En este sentido, estas limitaciones justifican la necesidad de un sistema que estructure los perfiles musicales y permita el filtrado y emparejamiento de usuarios de manera eficiente, garantizando recomendaciones relevantes basadas en compatibilidad musical real.

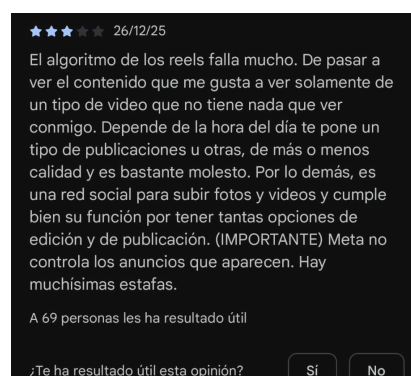


Figura 2.1 - Reseña sobre sugerencias de Instagram

En la figura se puede observar un ejemplo de reseña de instagram en el que usuarios muestran frustración por sugerencias de contenidos poco relevantes y que no se adapta a sus necesidades e intereses reales. Esto da sentido a la creación de un sistema específico y especializado de conexión de músicos, ya que las redes sociales generalistas suelen mostrar contenido genérico y con bajo nivel de afinidad.

2.2.2. Plataformas digitales orientadas a la música

Existen plataformas centradas en la publicación y difusión de contenido musical, como son SoundCloud o BandCamp, que permiten a los artistas compartir sus obras y llegar a un público más amplio. Estas aplicaciones ofrecen funcionalidades de escucha, valoración de contenido y en algunos casos incluso interacción entre usuarios.

Aunque estas plataformas favorecen más a la visibilidad de los músicos, no tienen como objetivo principal facilitar la creación de grupos musicales o la búsqueda de colaboraciones. La información recogida en los perfiles suele centrarse en el contenido que publican y no en otros aspectos clave, como la disponibilidad, los objetivos musicales o la compatibilidad.

Por este motivo, aunque estas plataformas pueden llegar a ser útiles para compartir música o promocionar contenido, no resuelven de forma directa el problema de encontrar músicos con los que colaborar o formar grupos.

"Plataforma para compartir tu música y podcast"

★★★★☆ 3,0

16 de febrero de 2023

Utilizo SoundCloud para subir alguna composición creada por mí y para escuchar podcasts. Como versión gratuita no está mal pero no pagaría por la versión premium con las opciones que tiene.

⊕ Puntos a favor:

Me ha encantado que pueda usar mis canciones en Instagram aún usando la versión gratuita También su servicio de streaming para los podcast.

⊖ Puntos en contra:

Me parece que el sistema en el que se recomienda las canciones no es el ideoneo siendo muy complicado dar a conocer tus canciones

Figura 2.2 - Reseña sobre sistema de recomendación de SoundCloud

Según algunas reseñas encontradas en distintos sitios web, los usuarios tienen dificultades para darse a conocer debido a sistemas de recomendación poco eficientes o poco adaptados a sus necesidades, lo que hace difícil entablar relaciones musicales reales a través de ellas.

2.2.3. Aplicaciones específicas de conexión entre músicos

También existen aplicaciones orientadas directamente a la conexión entre músicos, como es el caso de BandMix o JamKazam, cuyo principal objetivo es facilitar la formación de bandas o colaboraciones. Estas plataformas suelen incluir perfiles musicales más detallados y opciones de búsqueda basadas en instrumentos o estilos.

No obstante, muchas de estas soluciones presentan barreras de uso, como interfaces poco intuitivas, funcionalidades avanzadas restringidas a versiones por suscripción o una base de usuarios que se limita a ciertas regiones geográficas. Esto reduce considerablemente su accesibilidad, especialmente para músicos nuevos o estudiantes que buscan una herramienta sencilla y directa.

En muchos casos, estas limitaciones se reflejan también en opiniones de usuarios a través de foros o tiendas, en las que se mencionan problemas relacionados con el coste de funcionalidades o con la escasa presencia en regiones concretas.

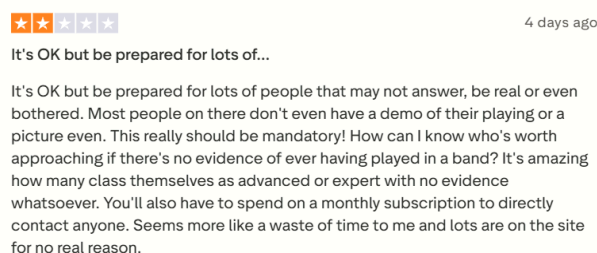


Figura 2.3 - Reseña sobre limitaciones de versión gratuita de BandMix

En la figura se observa una reseña de la aplicación BandMix, la cual hace referencia a la limitación de funcionalidades gratuitas, siendo en este caso el sistema de mensajería el principal afectado, ya que no se permite contactar con otros usuarios.

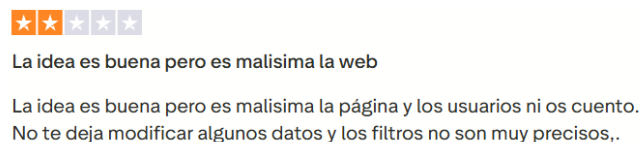


Figura 2.4 - Reseña sobre filtros y edición de perfil en BandMix

En la figura anterior, se puede ver otra reseña que hace referencia a la ineficiencia de los filtros y la imposibilidad de modificar algunos de los datos del perfil.

2.2.4. Sistemas de matching y recomendación

Los sistemas de matching se utilizan de forma habitual en aplicaciones de citas o plataformas profesionales, como Tinder o LinkedIn, donde se conectan usuarios a partir de criterios de afinidad definidos en sus perfiles. Estos sistemas demuestran ser muy eficientes a la hora de aplicar reglas de compatibilidad para sugerir conexiones relevantes.

Este enfoque resulta especialmente relevante para el ámbito musical, ya que permite automatizar una gran parte del proceso de búsqueda de colaboradores. En la solución que se plantea en este proyecto, el sistema de matching se basa en la comparación de información presente en los perfiles de usuario, como los instrumentos que interpretan, los estilos musicales o el nivel de experiencia.

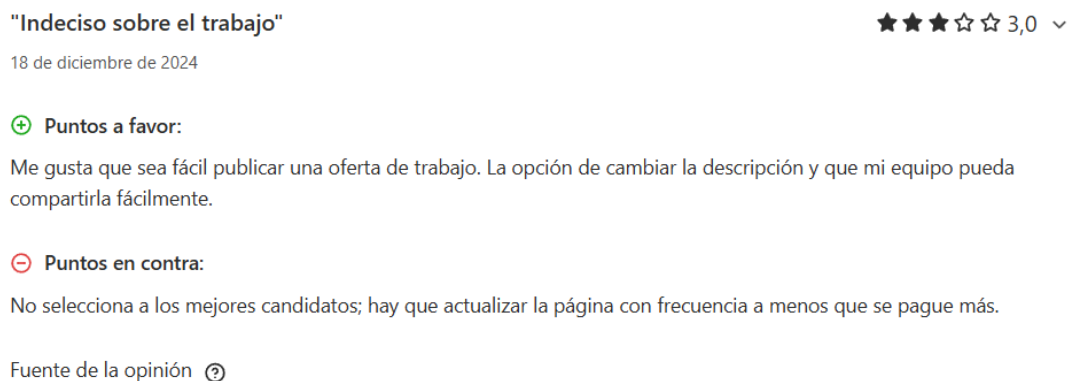


Figura 2.5 - Reseña sobre poca relevancia de algunos resultados en LinkedIn

Este tipo de plataformas suele tener buenas valoraciones en el público general, sin embargo, como se puede observar en la figura, muchos usuarios encuentran problemas en la falta de priorización de los resultados más relevantes, por lo que es conveniente implementar un sistema que de mayor visibilidad a las coincidencias más precisas y con mayor afinidad.

2.2.5. Posicionamiento del proyecto dentro del ecosistema TIC

A partir del análisis de las plataformas existentes, se observa que las soluciones actuales cubren parcialmente las necesidades de los músicos, pero no ofrecen una herramienta que combine la simplicidad, especialización y afinidad musical. El proyecto propuesto se posiciona dentro del ecosistema TIC como una aplicación especializada, que integra elementos de redes sociales, plataformas musicales y sistemas de matching, adaptándose a un contexto específico centrado en la conexión entre músicos locales.

La aplicación propuesta combina elementos de redes sociales, plataformas musicales y sistemas de matching para facilitar la identificación de perfiles afines y para simplificar la interacción y proceso de encontrar colaboraciones musicales.

2.3. Necesidades reales del mercado

El análisis del mercado musical local y amateur demuestra la existencia de una serie de necesidades que actualmente no están completamente cubiertas por las plataformas digitales disponibles. A pesar de la gran fuerza de las redes sociales y aplicaciones orientadas a la música, muchos músicos siguen encontrando dificultades a la hora de establecer colaboraciones estables y encontrar personas compatibles con las que poder formar una banda.

Esta situación es especialmente frecuente en el ámbito amateur o local, donde un gran número de músicos suele recurrir a métodos informales como anuncios en locales de ensayo, publicaciones en redes sociales o recomendaciones personales para poder encontrar miembros para un proyecto.

Una de las principales necesidades detectadas es la dificultad para contactar con músicos afines, especialmente a nivel local. La mayoría de los músicos dependen de disponer de contactos personales, anuncios en locales de ensayo o publicaciones en las redes sociales, lo que limita considerablemente el alcance y eficacia de la búsqueda. Este tipo de métodos suelen ser muy impredecibles y no garantizan la compatibilidad musical entre las personas que entran en contacto.

Otra necesidad relevante es la falta de soluciones especializadas en afinidad musical. Las redes sociales generalistas no están diseñadas para filtrar usuarios por criterios musicales específicos como el estilo, instrumento o nivel de experiencia. Como resultado, los músicos se ven obligados a realizar búsquedas manuales, invertir tiempo en conversaciones poco productivas o aceptar colaboraciones que no se ajustan a sus intereses reales.

Asimismo, las aplicaciones específicas existentes para conectar músicos presentan diversas limitaciones de accesibilidad y usabilidad. En muchos casos, estas plataformas cuentan con interfaces poco intuitivas, funcionalidades restringidas a suscripciones o una base de usuarios limitadas a ciertas zonas geográficas. Esto supone una barrera de entrada para músicos principiantes, estudiantes o usuarios que buscan una herramienta de uso sencillo y directo.

Por último, se detecta una clara falta de soluciones con enfoque local, que faciliten la conexión entre músicos de una misma ciudad o entorno cercano. La proximidad geográfica es un factor clave en el ámbito musical, ya que facilita los ensayos presenciales y la participación en proyectos conjuntos. Sin embargo, un gran número de plataformas no dan prioridad a este criterio o lo tratan de forma secundaria, lo que dificulta la creación de colaboraciones reales.

En conjunto, estas necesidades evidencian la existencia de un nicho de mercado para una aplicación especializada que permita conectar músicos de forma eficiente, sencilla y basada en criterios reales de compatibilidad musical, especialmente en el ámbito local, que es donde se sitúa el enfoque principal de este proyecto.

2.4. Oportunidades y líneas de negocio

El análisis de las necesidades del mercado permite identificar diversas oportunidades de desarrollo y posibles líneas de negocio asociadas al proyecto. Aunque la aplicación se plantea inicialmente como una solución funcional y formativa, su enfoque especializado abre la puerta a diferentes vías de evolución y sostenibilidad en un contexto real.

Una de las principales oportunidades detectadas es la existencia de nichos de mercado poco explotados, especialmente en el ámbito de los músicos locales, amateurs y estudiantes. Este colectivo suele quedar fuera del foco de las grandes plataformas musicales, que priorizan a artistas profesionales o con una alta visibilidad. Una aplicación centrada en la conexión por afinidad musical y proximidad geográfica permite cubrir este segmento de usuarios de forma directa y específica.

Otra oportunidad relevante es la posibilidad de crecimiento progresivo de la comunidad de usuarios. A diferencia de otras aplicaciones generalistas, una plataforma especializada en músicos puede fomentar relaciones más estables y recurrentes, ya que los usuarios tienden a utilizarla de forma continuada para encontrar nuevas colaboraciones, proyectos o bandas. Este crecimiento orgánico de la comunidad puede convertirse en un punto diferencial frente a soluciones más genéricas, ya que una plataforma centrada en músicos favorece la creación de relaciones estables entre usuarios con intereses similares.

Desde el punto de vista de las líneas de negocio, el proyecto podría evolucionar hacia un modelo en el que las funcionalidades básicas sean gratuitas y accesibles para todos los usuarios, mientras que determinadas características más avanzadas se ofrezcan por suscripción. Entre estas funcionalidades podrían incluirse filtros más avanzados de búsqueda o herramientas avanzadas para la gestión de grupos y proyectos, evitando funciones básicas que puedan limitar la experiencia de usuario.

Asimismo, la aplicación podría explorar líneas de colaboración con entidades del sector musical, como escuelas de música, locales de ensayo, salas de conciertos o asociaciones culturales. Estas colaboraciones permitirían ofrecer nuevas oportunidades a los usuarios, como difusión de eventos locales o el acceso a recursos específicos, al mismo tiempo que se genera una posible vía de monetización directa.

Por último, el proyecto presenta oportunidades de evolución funcional y tecnológica, como la incorporación de nuevas plataformas, mejoras en los algoritmos de matching o la integración con servicios externos relacionados con la música. Estas posibilidades de crecimiento refuerzan la viabilidad del proyecto como una solución escalable y adaptable a distintas necesidades del mercado, aunque en el contexto académico del ciclo formativo se limita a una versión funcional y controlada.

2.5. Tipo de proyecto requerido

Partiendo del análisis del contexto, las necesidades detectadas y las oportunidades identificadas en el sector, se ha tomado la decisión de que la solución más adecuada consiste en desarrollar una aplicación móvil orientada a la conexión entre músicos. Este formato permite una mayor facilidad en la interacción entre usuarios, gestionar perfiles y establecer contactos entre músicos de forma sencilla y accesible.

El uso de una aplicación móvil resulta especialmente adecuado en este contexto, ya que los dispositivos móviles son en la actualidad el principal medio de comunicación y acceso a las redes sociales por parte de los usuarios. En el caso de los músicos, una aplicación móvil les permite consultar perfiles, recibir notificaciones de coincidencias o gestionar conversaciones de forma rápida, favoreciendo la creación de colaboraciones y la organización de proyectos de forma ágil.

Frente a una página web tradicional, una aplicación móvil ofrece una experiencia de usuario más optimizada, con interfaces adaptadas al uso táctil y acceso directo desde el propio dispositivo. Además, permite la integración de funcionalidades propias de entornos móviles, como son las notificaciones, que resultan muy útiles para aplicaciones basadas en las interacciones frecuentes.

Por otro lado, aunque las aplicaciones web progresivas (PWA) presentan ventajas en términos de accesibilidad y despliegue, presentan mayores limitaciones en la integración con determinadas funcionalidades del sistema operativo que en el caso de aplicaciones móviles. En el contexto de este proyecto, el desarrollo de una aplicación móvil permite ofrecer una experiencia más fluida y trabajar con tecnologías directamente relacionadas con los módulos del curso formativo, especialmente el de desarrollo de aplicaciones multiplataforma.

Por este motivo, se opta por desarrollar una aplicación móvil basada en una arquitectura cliente-servidor, donde el cliente gestiona la interacción con el usuario y el servidor centraliza la lógica de negocio. Este enfoque facilita la organización del sistema y permite incorporar nuevos clientes en el futuro de una forma más sencilla.

2.6. Caracterización general de la solución

La solución propuesta consiste en el desarrollo de una aplicación móvil multiplataforma orientada a facilitar la conexión entre músicos locales mediante un sistema de matchmaking basado en criterios de afinidad musical. La aplicación está diseñada para ofrecer una experiencia sencilla, accesible y centrada en las necesidades reales de los usuarios.

2.6.1. Usuarios objetivo

La aplicación está dirigida principalmente a músicos locales, tanto estudiantes como aficionados o músicos con experiencia intermedia, que buscan formar bandas, participar en proyectos musicales o establecer colaboraciones habituales. Se trata de usuarios que, en general, disponen de conocimientos técnicos básicos y utilizan los dispositivos móviles como medio habitual de comunicación y organización.

Además, se contempla la existencia de un perfil administrativo, encargado de la gestión básica de la plataforma, como la supervisión de contenidos, el control de usuarios y el mantenimiento general del sistema. Este perfil no está orientado al uso musical, sino a garantizar su correcto funcionamiento.

2.6.2. Funcionalidades clave

La aplicación ofrecerá un conjunto de funcionalidades esenciales orientadas a cubrir las necesidades detectadas en el análisis previo. Entre las principales funcionalidades se pueden destacar las siguientes:

- Registro e inicio de sesión de usuarios.
- Creación y edición de un perfil musical que incluya información específica como instrumento, estilos musicales, nivel de experiencia u objetivos.
- Búsqueda y visualización de otros perfiles musicales mediante filtros básicos.
- Sistema de emparejamiento o recomendaciones basado en criterios de afinidad musical.
- Posibilidad de establecer contacto entre usuarios compatibles.

Estas funcionalidades constituyen el núcleo del prototipo desarrollado en este proyecto y se describirán con más detalle en próximos capítulos.

2.6.3. Requisitos esenciales

Para garantizar el funcionamiento de la solución, se establecen una serie de requisitos esenciales que deben cumplirse desde el punto de vista funcional y técnico. Entre ellos se pueden destacar:

- Disponibilidad de un sistema de autenticación seguro.
- Persistencia de datos de usuarios mediante una base de datos centralizada.
- Separación clara entre cliente y servidor.
- Interfaz intuitiva y fácil de usar.
- Rendimiento adecuado para un uso fluido de la aplicación.

Estos requisitos constituyen la base mínima necesaria para ofrecer a los usuarios una experiencia coherente y estable.

2.6.4. Restricciones y prioridades

El desarrollo del proyecto se encuentra condicionado por una serie de restricciones propias de su carácter académico. Entre las principales limitaciones se incluyen el tiempo disponible, los recursos técnicos y el alcance definido para el propio proyecto. Por este motivo, se prioriza el desarrollo de un prototipo funcional, centrado en demostrar la viabilidad de la solución y la correcta aplicación de los conocimientos adquiridos durante el ciclo formativo.

En consecuencia, se dejan fuera del alcance inicial funcionalidades avanzadas como sistemas de gestión de grupos musicales y mensajería grupal, algoritmos de recomendación basados en inteligencia artificial o integraciones con servicios externos. Estas características se contemplan como posibles líneas de evolución futura, pero no forman parte del núcleo del proyecto desarrollado.

2.7. Empresas representativas y modelos organizativos

El tipo de solución planteada en este proyecto se corresponde principalmente con el trabajo habitual de empresas que se dedican al desarrollo de software y aplicaciones móviles. En Andalucía existen numerosas empresas tecnológicas que desarrollan productos similares, tanto a nivel de aplicaciones móviles como de plataformas digitales basadas en arquitecturas cliente-servidor.

2.7.1. Empresas representativas

El desarrollo de aplicaciones móviles y plataformas digitales forma parte de la actividad de un gran número de empresas tecnológicas de la comunidad autónoma de Andalucía. A continuación se presentan algunos ejemplos significativos:

- **Isotrol (Sevilla).** Empresa tecnológica especializada en soluciones software y plataformas digitales.
- **OpenSistemas (Sevilla).** Dedicada al desarrollo de software, consultoría tecnológica y soluciones a medida.
- **i3net (Cádiz).** Consultora tecnológica especializada en servicios informáticos, soluciones digitales y desarrollo de software personalizado para empresas y organizaciones.
- **Satinel System (Cádiz).** Empresa de servicios y soporte informático con enfoque en soluciones tecnológicas, ciberseguridad y desarrollo de aplicaciones a medida.
- **Iditic Software (Cádiz).** Empresa de desarrollo de software especializada en sistemas ERP-CRM pero que también desarrolla soluciones multiplataforma y aplicaciones a medida según las necesidades de cada cliente.

Estas empresas representan el tipo de entorno profesional en el que se desarrollan algunos proyectos de carácter similar al propuesto, combinando tecnologías móviles, servidores backend y bases de datos.

2.7.2. Modelos organizativos

En las empresas del sector TIC dedicadas al desarrollo de aplicaciones móviles, la organización interna suele estructurarse por roles y departamentos especializados.

Dirección general

- **Dirección general.** Se encarga de la toma de decisiones y supervisa la actividad global de la empresa.
- **Project Manager.** Planifica, coordina y controla el avance del proyecto.
- **Product Owner.** Define y prioriza los requisitos del producto.

Departamento de análisis y diseño

- **Analista funcional.** Analiza y recoge los requerimientos y define las funcionalidades del sistema.
- **Diseñador UX/UI.** Diseña la experiencia de usuario y la interfaz gráfica de la aplicación.

Departamento de desarrollo

- **Desarrollador frontend.** Implementa la interfaz gráfica y la lógica del lado del cliente.
- **Desarrollador backend.** Desarrolla la lógica de negocio y la gestión de la base de datos.
- **Desarrollador full stack.** Combina las tareas tanto del lado del usuario como la lógica backend.

Departamento de calidad y pruebas

- **Tester/QA.** Realiza pruebas funcionales sobre la aplicación, detectando errores y verificando el cumplimiento de los requisitos funcionales de la aplicación.

Gracias a esta estructura organizativa se consigue distribuir las responsabilidades del proyecto entre distintos perfiles y departamentos especializados, facilitando el desarrollo de aplicaciones complejas mediante un trabajo coordinado y organizado.

3. ANÁLISIS DE USUARIO Y PROPUESTA DE VALOR

El éxito de una aplicación orientada a la conexión entre músicos depende directamente de su capacidad para adaptarse a las necesidades de los usuarios, especialmente en lo relacionado con la búsqueda de perfiles compatibles y la facilidad de contacto. Este capítulo se centra en el análisis de los perfiles de usuarios identificados en el contexto del proyecto, sus necesidades específicas y las tareas clave que deberán poder realizar dentro del sistema.

A partir de este análisis se consigue definir la propuesta de valor del proyecto, identificando los elementos diferenciadores que aporta la aplicación frente a otras soluciones existentes en el mercado. Además, se describe el proceso de validación inicial de la idea mediante la creación de prototipos y la recogida de feedback preliminar.

Con este análisis se consigue garantizar que el diseño de la solución y las funcionalidades implementadas se encuentren alineadas con las necesidades reales de los usuarios.

3.1 Perfiles de usuario

Para comprender las necesidades de los usuarios y diseñar una solución que se adapte a ellas, se han definido distintos perfiles de usuario mediante la metodología Buyer Persona.

A partir del análisis realizado se ha logrado identificar tres perfiles principales dentro de la aplicación.

Estos perfiles de usuario han sido definidos teniendo en cuenta las necesidades detectadas tras el análisis del sector, y sirven como base para determinar qué funcionalidades forman parte del alcance del MVP, especialmente en lo relacionado a perfiles musicales y el matching entre músicos.

3.1.1. Buyer Persona 1 - Músico aficionado

Datos generales: Alejandro Pérez, 21 años

Situación: Estudiante universitario

Alejandro toca la guitarra desde hace 4 años como hobby. Le gusta tocar música durante su tiempo libre y le gustaría formar parte de una banda con otros músicos de su ciudad para poder ensayar y tocar ocasionalmente en pequeños eventos locales.

Objetivos:

- Conocer otros músicos con intereses similares.
- Encontrar personas con las que poder formar una banda.
- Descubrir músicos cercanos geográficamente.

Necesidades:

- Una plataforma sencilla donde encontrar músicos.
- Poder ver qué instrumentos tocan otros usuarios.
- Saber qué estilos musicales les gustan.

Frustraciones:

- En redes sociales resulta complejo encontrar músicos por su zona.
- Los anuncios en locales de ensayo no siempre funcionan.
- Muchas aplicaciones musicales están orientadas a músicos profesionales.

Este perfil representa al usuario principal del sistema, por lo que las funcionalidades del MVP se centran esencialmente en cubrir sus necesidades.

3.1.2. Buyer Persona 2 - Músico con experiencia

Datos: Marta Herrera, 29 años

Situación: Profesora de música y baterista

Marta tiene experiencia tocando en varias bandas y actualmente quiere encontrar nuevos proyectos musicales. Le interesa encontrar músicos comprometidos con los que poder crear un grupo estable y preparar un repertorio para tocar en vivo.

Objetivos:

- Encontrar músicos con un nivel de experiencia similar.
- Participar en proyectos musicales serios.
- Crear grupos musicales estables.

Necesidades:

- Poder filtrar músicos por nivel de experiencia.
- Ver información detallada de los perfiles.
- Comunicarse fácilmente con otros músicos.

Frustraciones:

- Muchas plataformas tienen pocos usuarios activos.
- No es fácil encontrar músicos plenamente compatibles.

Este perfil influye en la incorporación de filtros más específicos para el sistema de búsqueda y matching, ya que demuestra la necesidad de un sistema de búsqueda y filtrado fuerte.

3.1.3. Buyer Persona 3 - Administrador

Datos: Carlos Sánchez, 34 años

Situación: Técnico informático y administrador de la plataforma

Carlos es el responsable de supervisar el correcto funcionamiento de la aplicación. Su labor consiste en garantizar que el sistema funcione de manera estable y que los usuarios respeten las normas de convivencia de la plataforma.

No participa de forma directa en la actividad de la aplicación, pero tiene acceso a herramientas de gestión que le permiten controlar el estado del sistema y resolver posibles incidencias.

Objetivos:

- Garantizar el funcionamiento estable de la plataforma.
- Supervisar la actividad de los usuarios registrados.
- Detectar y gestionar posibles incidencias o comportamientos inadecuados.

Necesidades:

- Acceso a herramientas de administración del sistema.
- Posibilidad de gestionar usuarios registrados.
- Acceso a información básica del funcionamiento.

Frustraciones:

- Falta de visibilidad sobre posibles problemas dentro de la plataforma.
- Necesidad de herramientas que faciliten la supervisión.

Este perfil se tiene en cuenta en un nivel más técnico, ya que, a pesar de no formar parte del uso principal de la aplicación, es necesario para garantizar la seguridad y funcionamiento del sistema.

3.2 Tareas clave de los usuarios

En base al análisis de los perfiles de usuario se han podido identificar las principales tareas que los usuarios deben realizar dentro de la aplicación.

Estas tareas constituyen la base para la definición de las historias de usuario y los requisitos funcionales del sistema.

Las principales tareas que se han podido identificar son las siguientes:

- Gestionar el perfil:
 1. Registrarse en la aplicación.
 2. Iniciar sesión.
 3. Crear y editar su perfil musical.
- Descubrir músicos:
 1. Buscar otros músicos según filtros.
 2. Visualizar perfiles musicales.
 3. Encontrar coincidencias musicales.
- Interactuar con otros usuarios:
 1. Aceptar o rechazar coincidencias.
 2. Enviar mensajes privados.

Todas estas tareas se centran en las funcionalidades principales del MVP del proyecto, dando prioridad a la creación de perfiles musicales y al descubrimiento de músicos compatibles. Las funciones de gestión de grupos y chats avanzados se reservan como posibles ramas para futuras ampliaciones.

3.3 Propuesta de valor

La propuesta de valor del proyecto se basa esencialmente en ofrecer una herramienta sencilla y especializada que permita conectar músicos a partir de criterios reales de compatibilidad musical.

A diferencia de las redes sociales generalistas o de las plataformas musicales centradas en la difusión de contenido, esta aplicación se centra directamente en conectar músicos, facilitando la formación de bandas o creación de proyectos.

Los principales elementos que definen la propuesta de valor son los siguientes:

- **Afinidad musical.** El sistema de matching facilita la conexión entre usuarios, teniendo en cuenta criterios relevantes como los instrumentos, los estilos musicales y el nivel de experiencia. Todo esto aumenta la probabilidad de encontrar colaboradores compatibles.
- **Enfoque específico.** La aplicación está diseñada específicamente para músicos, lo que permite ofrecer funcionalidades adaptadas a sus necesidades reales.
- **Simplicidad de uso.** El diseño de la aplicación prioriza una experiencia de usuario sencilla e intuitiva, reduciendo las barreras de entrada para músicos sin experiencia técnica.

Esta propuesta de valor se construye directamente a partir de las necesidades detectadas en los perfiles de usuario, especialmente en lo relacionado con la dificultad para encontrar músicos compatibles de forma sencilla. Por esta razón, el proyecto prioriza un sistema de matching que se base en criterios musicales estructurados frente a enfoques más genéricos.

3.4 Validación inicial de la idea

Antes de iniciar el desarrollo del prototipo se realizó una validación preliminar de la idea mediante la elaboración de prototipo inicial de la interfaz utilizando la herramienta Figma.

Este prototipo permitió representar visualmente el flujo principal de navegación de la aplicación, incluyendo las pantallas de registro, exploración de perfiles, sistema de coincidencias y mensajería.

A partir de este prototipo se han recogido opiniones de varios usuarios potenciales, en especial estudiantes y músicos aficionados, quienes destacaron la utilidad del sistema de perfiles y la facilidad para entender el flujo de búsqueda de músicos.

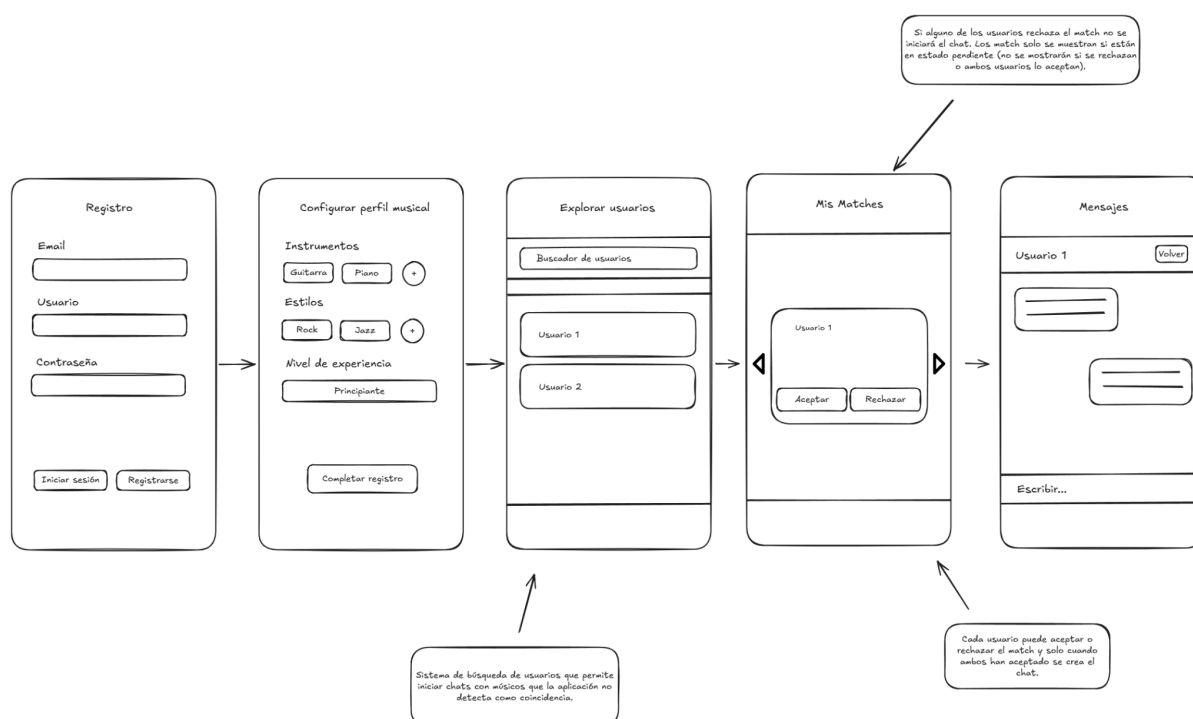


Figura 3.1 - Sketch inicial de la aplicación

A partir del sketch inicial de la figura anterior se ha podido realizar una validación de la estructura básica y flujo de la aplicación. Como se puede ver, es un esquema simple sin muchos detalles de diseño, cuyo único objetivo es validar la idea inicial a través de los usuarios objetivos de la aplicación.

Este proceso permitió validar especialmente las funcionalidades clave del MVP, como la creación de perfiles musicales y la exploración de otros usuarios, confirmando que el enfoque del proyecto resulta comprensible y útil para el público objetivo.

4. ESPECIFICACIÓN DE REQUISITOS

Este apartado tiene como objetivo definir los requisitos que debe cumplir la aplicación propuesta, tomando como base las necesidades detectadas en el análisis de usuarios del capítulo 3. A partir de estas necesidades, se concretan las funcionalidades que formarán parte del MVP, así como las características que debe cumplir el sistema para garantizar su funcionamiento.

Con ello, se busca garantizar que la solución propuesta cubra todas las necesidades de los usuarios y se ajuste a los objetivos planteados.

4.1 Historias de usuario

Las historias de usuario tienen el fin de describir las funcionalidades de la solución desde la perspectiva del usuario final, enfocándose en el valor que aporta cada una de las acciones. Se relacionan de forma directa con las tareas clave que se identificaron en el apartado anterior:

1. Registro e inicio de sesión:

- “Como usuario quiero registrarme con email, contraseña y nombre de usuario, para que el inicio de sesión sea sencillo y cómodo”.
- “Como usuario quiero iniciar sesión para recuperar mis datos guardados desde cualquier lugar”.

2. Configuración del perfil:

- “Como músico quiero añadir los instrumentos musicales que toco, para que otros usuarios conozcan mis habilidades”.
- “Como músico quiero añadir mis estilos musicales favoritos, para que el sistema pueda sugerirme músicos compatibles”.
- “Como músico quiero indicar mi nivel de experiencia en cada instrumento, para que la aplicación me muestre resultados afines”.

3. Exploración de usuarios y matches:

- “Como músico quiero explorar perfiles de otros músicos cercanos, para poder encontrar colaboradores en mi zona”.
- “Como músico quiero aceptar o rechazar coincidencias sugeridas por la aplicación, para así gestionar mis posibles conexiones”.
- “Como músico quiero ver el estado de mis coincidencias (pendiente, aceptado, rechazado), para tener control sobre mis interacciones”.

4. Mensajería:

- “Como usuario quiero enviar mensajes privados, para poder organizar colaboraciones musicales”.

5. Administración:

- “Como administrador quiero gestionar usuarios y grupos de forma sencilla, para así garantizar el correcto funcionamiento de la plataforma”.

Estas historias de usuario se centran en las funcionalidades esenciales del MVP, priorizando la creación de perfiles musicales, el sistema de matching y la interacción básica entre usuarios.

4.2 Requisitos funcionales

Los requisitos funcionales definen qué funcionalidades debe tener la aplicación para que se cumplan las necesidades definidas en las historias de usuario.

Los siguientes requisitos funcionales se han definido a partir de las historias de usuario y se centran en las funcionalidades principales del MVP del proyecto:

1. Gestión de usuarios

- RF1. El sistema debe permitir el registro de nuevos usuarios mediante email, nombre de usuario y contraseña.
- RF2. El sistema debe permitir el inicio y cierre de sesión de forma segura.
- RF3. El sistema debe permitir la recuperación de la contraseña.

2. Perfil musical

- RF4. El sistema debe permitir la selección de instrumentos musicales y estilos.
- RF5. El sistema debe permitir indicar el nivel de experiencia por cada instrumento seleccionado.
- RF6. El sistema debe permitir editar la biografía del músico y la foto de perfil.

3. Sistema de matchmaking

- RF7. El sistema debe generar coincidencias automáticamente según la afinidad y localización/ciudad de la persona.
- RF8. El sistema debe controlar y mostrar los estados del match (pendiente, rechazado o aceptado).
- RF9. El sistema debe notificar al usuario de sus nuevas coincidencias.

4. Exploración de usuarios

- RF10. El sistema debe mostrar un listado de músicos cercanos.
- RF11. El sistema debe permitir filtrar búsquedas por instrumentos y estilos.
- RF12. El sistema debe permitir buscar músicos por nombre.

5. Mensajería

- RF13. El sistema debe permitir el envío y recepción de mensajes privados a otros músicos.
- RF14. El sistema debe permitir visualizar los mensajes por orden cronológico.

6. Administración

- RF15. El sistema debe permitir la gestión de roles de usuarios.
- RF16. El sistema debe permitir a los administradores supervisar el contenido.
- RF17. El sistema debe permitir a los administradores eliminar y bloquear cuentas si fuese necesario.

Estos últimos requisitos asociados a la administración se tienen en consideración para los requisitos funcionales del sistema, sin embargo, no forman parte del MVP y se quedan abiertos como aplicación futura.

4.3 Requisitos no funcionales

Los requisitos no funcionales definen las condiciones de calidad que debe cumplir la aplicación, asegurando un funcionamiento adecuado en aspectos como rendimiento, seguridad o usabilidad. Estos requisitos se han definido teniendo en cuenta que la aplicación propuesta es una app móvil y las necesidades de los músicos identificadas previamente.

ID	Requisito	Descripción
RNF1	Rendimiento	La aplicación debe cargar el listado de usuario en menos de tres segundos en condiciones normales de uso
RNF2	Seguridad	Las contraseñas de los usuarios deben almacenarse de forma cifrada y el sistema debe implementar un mecanismo de autenticación segura basado en credenciales
RNF3	Escalabilidad	El sistema debe soportar el crecimiento del número de usuarios sin afectar significativamente al rendimiento del sistema de matching
RNF4	Disponibilidad	El sistema debe estar operativo al menos el 90% del tiempo, excluyendo periodos de mantenimiento
RNF5	Usabilidad	La interfaz de usuario debe ser intuitiva y adaptada a dispositivos móviles, facilitando la navegación y el acceso a las funcionalidades principales de forma rápida y sencilla
RNF6	Compatibilidad	El sistema debe ser compatible con dispositivos móviles Android y adaptarse a diferentes tamaños de pantalla
RNF7	Mantenibilidad	El sistema debe estar estructurado de forma modular, facilitando su mantenimiento y la evolución futura de la aplicación

Tabla 4.1 - Requisitos no funcionales de la aplicación

Estos requisitos no funcionales se tendrán en cuenta durante el diseño de la arquitectura del sistema, influyendo en decisiones como la estructura del backend o la gestión de la comunicación cliente-servidor.

4.4 Diagramas de casos de uso

Los diagramas de casos de uso permiten representar de forma visual las interacciones entre los distintos tipos de usuario y el sistema, centrándose en las funcionalidades principales del MVP, como la gestión de perfiles, el sistema de matching y la interacción básica entre los usuarios.

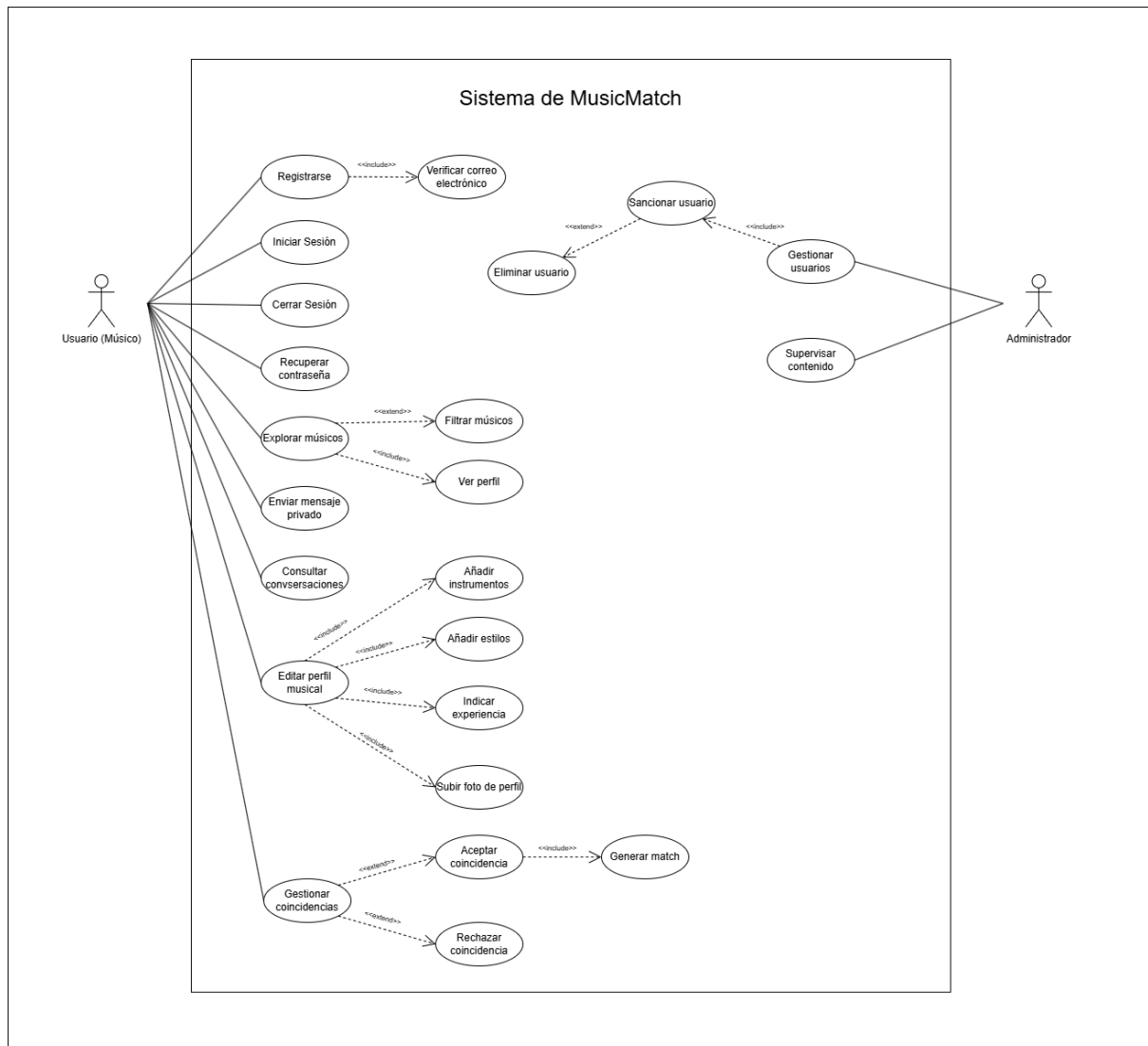


Figura 4.1 - Diagrama de casos de uso

En el diagrama se representan los flujos principales del sistema, centrados en las acciones clave de los usuarios como el registro, la configuración del perfil musical y la exploración de coincidencias. No se incluyen funcionalidades avanzadas fuera del alcance del MVP.

5. DISEÑO DE LA SOLUCIÓN

5.1 Diseño UX (experiencia de usuario)

El diseño de la experiencia de usuario (UX) tiene el objetivo de garantizar una interacción intuitiva y eficiente, centrada en facilitar la búsqueda de músicos compatibles según instrumento, estilo musical y localización, así como la comunicación entre usuarios.

Para alcanzar estos objetivos se ha priorizado un enfoque centrado en el usuario, teniendo en cuenta sus principales necesidades:

- Crear y gestionar su perfil musical.
- Explorar otros músicos según intereses y localización.
- Gestionar coincidencias (aceptar o rechazar).
- Comunicarse por mensajes privados.
- Editar información del perfil con posterioridad.
- Agregar foto y una breve biografía.

La aplicación está orientada a músicos aficionados o semiprofesionales, por lo que se da prioridad a la simplicidad en los flujos principales (registro, matchmaking, mensajes) para reducir la curva de aprendizaje y evitar el abandono temprano de la aplicación.

5.1.1. Principios de diseño

Para el diseño de la interfaz se han aplicado algunos principios orientados a facilitar el uso de la aplicación en los flujos principales:

- **Simplicidad.** Cada pantalla muestra exclusivamente la información esencial. Por ejemplo, en el sistema de matchmaking se presentan únicamente los datos relevantes del músico, incluyendo instrumentos, estilos, nivel de experiencia y localización.
- **Consistencia.** Los botones, colores y estructuras de navegación mantienen la coherencia en toda la aplicación, permitiendo al usuario reutilizar patrones ya aprendidos y reduciendo el esfuerzo cognitivo.

- **Jerarquía visual.** La información se organiza en distintos niveles de relevancia mediante tamaños fuente, colores o disposición de elementos, permitiendo identificar rápidamente datos clave como el nombre de usuario o sus características musicales.
- **Feedback inmediato.** Las acciones principales del usuario, como puede ser aceptar o rechazar coincidencias, generan una respuesta visual inmediata, reforzando la sensación de control y evitando incertidumbre.
- **Accesibilidad básica.** Se utilizan contrastes de color y tamaños de texto adecuados y legibles para garantizar una experiencia lo más cómoda posible en distintos dispositivos.

La aplicación de estos principios se ha alineado con el alcance del MVP, priorizando la optimización de los flujos críticos del sistema. Esto permite ofrecer una experiencia de usuario coherente y eficiente en funcionalidades clave como el registro, el sistema de matchmaking y la mensajería privada, asegurando una base sólida sobre la que poder evolucionar e incorporar mejoras.

5.1.2. Flujo principal de navegación

El flujo principal de navegación se ha diseñado para permitir que el usuario pueda comenzar a interactuar con otros músicos en el menor número de pasos posibles, priorizando las funcionalidades de registro, generación de coincidencias y conversación.

1. Registro y creación de perfil

El proceso de incorporación de usuarios se estructura en varias fases consecutivas:

- I. Registro de usuario (Correo, contraseña y nombre de usuario)
- II. Configuración de perfil musical (Instrumentos, estilos, nivel)
- III. Finalización de registro.

Este flujo se plantea como un proceso guiado y progresivo, evitando formularios extensos en una única pantalla. El objetivo es reducir la fricción y minimizar el abandono durante el proceso inicial.

2. Búsqueda y generación de matches

Una vez se ha completado el registro, el usuario accede al sistema de exploración de perfiles:

- I. Visualización de músicos compatibles según los criterios definidos.
- II. Consulta de información relevante de cada perfil musical.
- III. Posibilidad de aceptar o rechazar coincidencias.

Cuando dos usuarios aceptan el match, se habilita automáticamente la comunicación entre ellos.

Este flujo se ha diseñado con el fin de favorecer una interacción rápida y directa, permitiendo evaluar perfiles de forma ágil y centrando la atención en la toma de decisiones.

3. Mensajes

Una vez que los dos miembros de un match aceptan la coincidencia, se habilita un canal de comunicación entre ambos usuarios que permite:

- I. Envío de mensajes de texto a través de un chat con otro músico.
- II. Recepción y notificación de mensajes por parte de otros músicos.

En el flujo del MVP se contempla únicamente el sistema de matching básico y la interacción rápida entre usuarios mediante mensajería privada. Funcionalidades como la gestión de grupos se reservan para futuras ampliaciones del sistema.

5.1.3. Estructura y diseño

La aplicación se estructura en varias secciones. Las principales son:

- Inicio.
- Matches.
- Mensajes.
- Perfil.

Dado que la solución se implementa como una aplicación móvil, se ha optado por incluir una barra de navegación inferior que permite el acceso a diferentes secciones. Esta decisión responde a patrones de diseño habituales en aplicaciones móviles, facilitando la navegación con una sola mano y reduciendo el número de acciones necesarias para cambiar de pantalla.

5.1.4. Experiencia emocional del usuario

La experiencia emocional del usuario se ha planteado con el propósito de transmitir una sensación de cercanía, creatividad y profesionalidad dentro del entorno musical. Para ello, se opta por un diseño visual limpio, resaltando la información relevante del perfil y permitiendo al usuario evaluar los perfiles musicales de forma rápida.

Asimismo, el uso de una paleta de colores moderna y coherente con el ámbito musical, contribuye a reforzar la identidad visual de la aplicación, generando una experiencia atractiva y alineada con las expectativas del usuario objetivo.

5.1.5. Wireframing y prototipado

El proceso de diseño ha incluido la elaboración de wireframes que representan la estructura y el flujo principal de la aplicación. Estos esquemas permiten visualizar de forma preliminar la organización de las pantallas y las interacciones entre ellas.

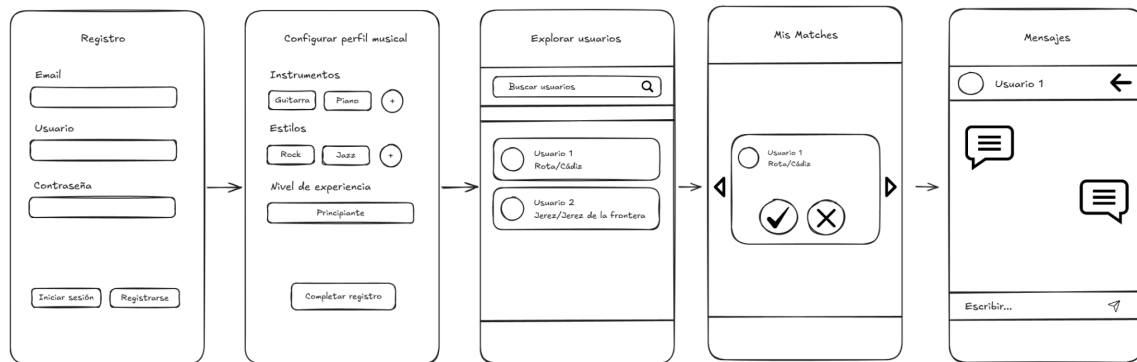


Figura 5.1 - Wireframing del diseño de las ventanas principales

En la figura anterior se muestra un sketch que recoge las principales ventanas del sistema, incluyendo registro, exploración de perfiles, coincidencias y mensajería. Este prototipo inicial se centra en la disposición de los elementos y el flujo de navegación, sin entrar en detalles visuales como colores, tipografía o identidad gráfica.

Estos aspectos específicos de diseño se desarrollan en el apartado de diseño UI, donde se presentan mockups más detallados de la interfaz final. Debe tenerse en cuenta que algunas ventanas secundarias como el apartado de perfil, no se han representado en este prototipo, ya que su objetivo es mostrar el flujo principal de la aplicación.

Por su parte, el conjunto del diseño UX se ha centrado en reducir la complejidad de uso y facilitar la interacción rápida entre músicos, alineándose con el objetivo principal de generar coincidencias entre usuarios de manera eficiente.

5.2 Diseño UI (interfaz)

El diseño de la interfaz de usuario tiene el objetivo de definir la apariencia visual de la aplicación y la manera en la que los elementos gráficos se presentan e interactúan con el usuario.

En el caso de esta aplicación, el diseño se ha realizado siguiendo principios de simplicidad, claridad visual y usabilidad, asegurando una representación clara de las funcionalidades definidas en el diseño UX.

El diseño de la interfaz aplica principios de Material Design, lo que permitirá garantizar la consistencia en componentes como botones, formularios y navegación, adaptándose a estándares de diseño en aplicaciones móviles.

5.2.1. Identidad visual de la solución

La identidad visual de la aplicación se ha diseñado para que transmita una estética moderna y relacionada con el entorno musical y plataformas sociales. El objetivo es crear una interfaz que sea atractiva y que aporte una mayor facilidad en la interacción.

Para ello, se ha seleccionado una paleta de colores que combina un color principal, utilizado para destacar acciones importantes, con una gama de colores más neutros que permitan definir textos más legibles.

Paleta de colores utilizada:

Elemento	Color	Uso
Color principal	#2C3E50 (Azul oscuro)	Botones principales y cabeceras
Color secundario	#8E44AD (Morado)	Elementos interactivos destacados
Color de aceptación	#27AE60 (Verde)	Aceptar coincidencias
Color de rechazo	#E74C3C (Rojo)	Rechazar coincidencias
Color de fondo	#F5F5F5 (Gris claro)	Fondo general para la aplicación
Color del texto	#333333 (Gris oscuro)	Texto principal de la aplicación

Tabla 5.1 - Paleta de colores de la solución

Gracias al uso de esta paleta, se consigue que las acciones queden claramente diferenciadas dentro de la interfaz, facilitando la comprensión por parte del usuario.

5.2.2. Tipografía y fuentes

En cuanto a la tipografía seleccionada, esta busca priorizar la legibilidad en pantallas de dispositivos móviles, manteniendo una estética moderna al mercado actual.

Se ha optado por utilizar la tipografía Roboto con la fuente sans-serif ampliamente utilizadas en aplicaciones móviles, ya que estas ofrecen una mayor legibilidad en pantallas de diversos tamaños.

Para los tamaños fuentes se define la siguiente tabla:

Elemento	Tamaño	Uso
Títulos	22-24 px	Nombre de cada sección
Subtítulos	18 px	Secciones dentro de una misma pantalla
Texto de contenido	14-16 px	Información del perfil y mensajes
Textos de interacción	14-16 px	Acciones de usuario (botones, etiquetas, etc)

Tabla 5.2 - Tamaño fuente de los estilos de texto

Con esta jerarquía se permite que el usuario pueda identificar rápidamente la información relevante dentro de cada pantalla.

5.2.3. Ventana de registro

La ventana de registro representa el primer punto de interacción entre el usuario y el sistema. En ella, los usuarios pueden crear una nueva cuenta introduciendo la información básica de autenticación.

La interfaz presenta los principales campos de entrada organizados verticalmente, permitiendo una mayor adaptación a dispositivos móviles y facilitando la introducción de datos por parte del usuario.

Principales componentes:

- Campo de email.
- Campo de nombre de usuario.
- Campo de contraseña.
- Botón para registrarse.
- Botón para iniciar sesión.

El botón de registro se destaca utilizando el color principal de la aplicación, esto con el fin de indicar de forma clara la acción principal que debe realizar el usuario.

A continuación se refleja una representación visual en Figma bastante cercana al resultado esperado en la solución final:

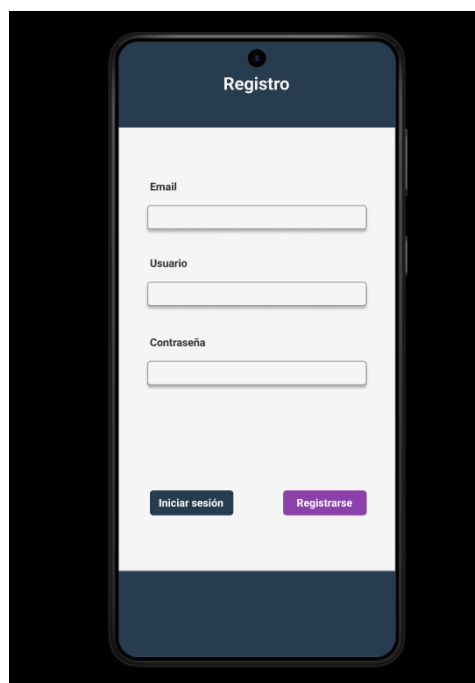


Figura 5.2 - Mockup de ventana de registro en Figma

5.2.4. Ventana de configuración de perfil musical

Una vez el usuario haya completado el proceso de registro, se le deberá mostrar una ventana en la que pueda configurar el perfil musical. Esta pantalla permite introducir la información que la aplicación necesita para el matchmaking.

El diseño de esta interfaz se centra en facilitar la selección de instrumentos, estilos y nivel de experiencia a través de elementos visuales sencillos que eviten la introducción de texto manual por parte del usuario.

Elementos principales:

- Selector de instrumentos.
- Selector de estilos.
- Selector de nivel de experiencia.
- Selector de ciudad o región (agregado respecto al sketch inicial)
- Botón para finalizar la creación del perfil.

Los instrumentos y estilos se representarán a través de elementos seleccionables, permitiendo al usuario elegir múltiples opciones de forma rápida y sencilla. El color de los instrumentos se mostrará rojo si es nivel principiante, amarillo para nivel intermedio y verde si es experto.

A continuación se puede observar una representación visual realizada en Figma que refleja de forma muy cercana el diseño final de la configuración de perfil musical:

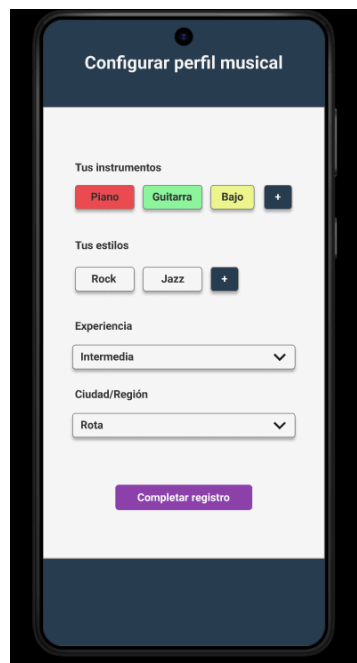


Figura 5.3 - Mockup de ventana de creación de perfil en Figma

5.2.5. Ventana de inicio

La ventana de inicio permite a los usuarios visualizar otros músicos registrados en la aplicación mediante filtros y un buscador por nombre. Esta ventana facilita conocer otros músicos independientemente del matchmaking.

Los perfiles se representan mediante tarjetas que representan la información básica de cada usuario.

Elementos principales:

- Buscador de músicos.
- Opciones de filtrado.
- Listado de músicos con información básica.

Esta estructura permite navegar a través de los distintos perfiles musicales de forma sencilla y cómoda, permitiendo conocer otros músicos de forma rápida y cómoda.

A continuación se muestra un diseño en Figma que muestra un resultado aproximado de la visualización de la ventana en la aplicación:

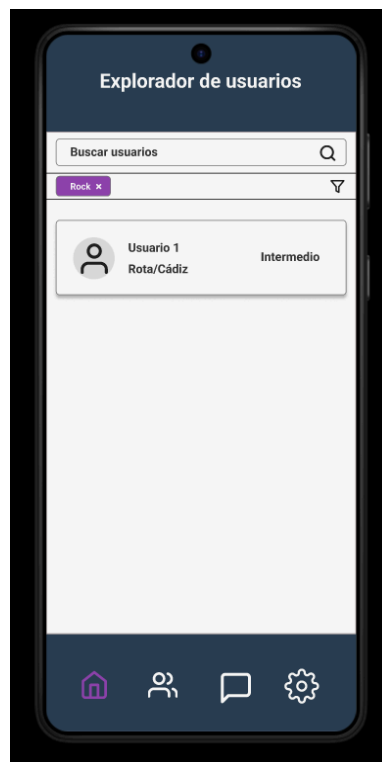


Figura 5.4 - Mockup de ventana de inicio en Figma

5.2.6. Ventana de matching

La ventana de coincidencias es aquella en la que se muestran los matches que genera el sistema basándose en criterios de afinidad musical. En ella, los usuarios pueden revisar las coincidencias sugeridas y aceptar o rechazar el match.

Se prioriza el uso de tarjetas individuales para centrar la atención del usuario en una única decisión por pantalla.

Sus elementos principales son:

- Tarjeta de perfil de usuario coincidente.
- Botón para aceptar la coincidencia.
- Botón para rechazar la coincidencia.

Estos controles permitirán al usuario controlar de forma rápida y sencilla las coincidencias que la aplicación genera automáticamente.

Acto seguido se presenta un prototipo visual de la ventana de coincidencias diseñado en Figma:



Figura 5.5 - Mockup de ventana de matches en Figma

5.2.7. Ventana de chat

La ventana de chat permite la comunicación entre usuarios que han aceptado una coincidencia. Esta funcionalidad permite a los músicos coordinar y organizar colaboraciones o ensayos.

La interfaz de mensajería se ha diseñado siguiendo patrones conocidos para reducir la curva de aprendizaje.

Principales elementos:

- Área de visualización de mensajes.
- Campo de texto para escribir mensajes.
- Botón de envío.

Los mensajes se mostrarán organizados por orden cronológico para facilitar a los usuarios el seguimiento de la conversación.

A continuación se muestra el diseño del mockup desarrollado en Figma:



Figura 5.6 - Mockup de ventana de chat en Figma

5.2.8. Ventana de edición de perfil

La ventana de edición de perfil permite al usuario modificar la información previamente configurada durante el registro, adaptando su perfil a los posibles cambios de localización, intereses o situación musical.

Esta funcionalidad resulta especialmente relevante para el contexto de la aplicación, ya que la información del perfil influye directamente en el sistema de matching y en la visibilidad frente a otros usuarios.

La interfaz utiliza componentes y una estructura visual muy similar a la pantalla de configuración inicial, garantizando consistencia y reduciendo la curva de aprendizaje.

Elementos principales:

- Cambio de foto de perfil.
- Edición de nombre de usuario.
- Edición de localización/ciudad.
- Edición de instrumentos musicales.
- Edición de estilos musicales.
- Modificación del nivel de experiencia.
- Edición de biografía del usuario.

Los distintos campos se presentan mediante componentes visuales ya utilizados en el registro, permitiendo al usuario realizar modificaciones de forma rápida y sin necesidad de reaprender la interfaz.

A continuación se muestra el diseño del mockup desarrollado en Figma, que representa un diseño muy cercano a la versión final de la ventana de edición del perfil:

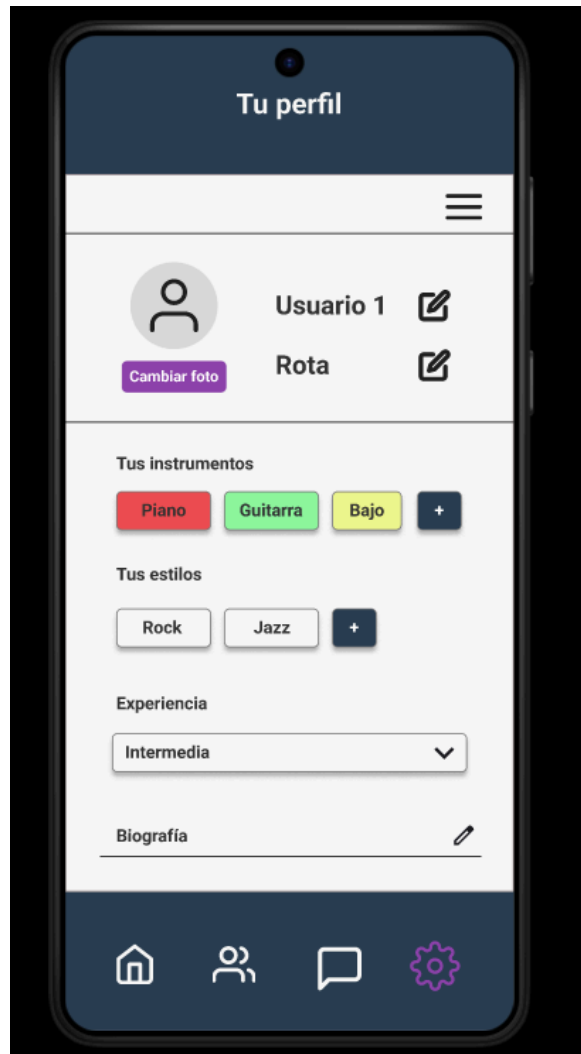


Figura 5.7 - Mockup de ventana de edición de perfil

En conjunto, el diseño UI se ha orientado a garantizar una interfaz clara, consistente y adaptada al entorno móvil, facilitando la interacción en los flujos del sistema (registro, matching y mensajería), que constituyen el núcleo funcional del MVP.

5.3 Arquitectura del sistema

5.3.1. Visión general de la arquitectura

La arquitectura del sistema se ha definido teniendo en cuenta las necesidades concretas de la aplicación, es decir, la gestión de perfiles musicales, generar coincidencias entre usuarios y permitir la comunicación mediante mensajes.

Para dar respuesta a estos requerimientos, se ha adoptado un modelo cliente-servidor, en el que la aplicación móvil se encarga de la interacción con el usuario, mientras que el backend centraliza la lógica de negocio, enfocada principalmente a el cálculo de coincidencias y la validación de datos. La base de datos, por su parte, se encarga de persistir la información necesaria para garantizar el funcionamiento del sistema.

La adopción de este modelo no responde exclusivamente a un enfoque teórico, sino a necesidades concretas del proyecto:

- El sistema de matchmaking requiere lógica centralizada para garantizar resultados coherentes entre usuarios.
- La mensajería necesita un control fiable desde el servidor para asegurar la integridad de los envíos.
- La gestión de perfiles debe mantenerse consistente independientemente del cliente desde el que se acceda al sistema.

Por ello, se opta por una arquitectura que separa claramente las responsabilidades y evita que el cliente móvil se encargue de la lógica crítica.

Desde un punto de vista estructural, el sistema se organiza en tres bloques principales:

- Cliente móvil para sistemas Android.
- Servidor backend centralizado.
- Sistema de persistencia de datos.

La comunicación entre cliente y servidor se realiza mediante una API REST, permitiendo un intercambio de datos sencillo, desacoplado y adaptado al contexto de una aplicación móvil.

5.3.2. Arquitectura cliente-servidor

El modelo cliente-servidor establece una división clara entre la capa de presentación y la lógica de negocio, algo clave en este proyecto debido a la criticidad del sistema de matching entre músicos.

Cliente móvil

El cliente móvil se encarga exclusivamente de controlar la interacción con el usuario y de representar la información recibida desde el servidor.

Sus responsabilidades en el contexto del proyecto son:

- Mostrar la información de perfiles musicales.
- Gestionar la interacción con el sistema de matches.
- Permitir la comunicación entre los usuarios.
- Enviar y recibir información del backend.

Se ha decidido no incluir el cálculo de coincidencias ni lógica de negocio compleja en el cliente, ya que esto podría generar inconsistencias o manipulaciones. Toda la lógica crítica se concentra en el servidor.

Base de datos

La persistencia de datos se realiza a través de un modelo relacional, seleccionado por su adecuación al modelo de datos del proyecto, el cual se basa esencialmente en las relaciones entre usuarios, instrumentos, estilos y matches.

Esta elección está directamente relacionada con las necesidades del sistema:

- Relaciones N:M entre usuarios, instrumentos y estilos.
- Gestión de matches únicos entre pares de usuarios.
- Integridad de los datos en operaciones concurrentes.

El acceso a los datos se abstrae mediante un ORM, lo que permite trabajar los datos como objetos, simplificando el desarrollo al evitar el uso de consultas SQL avanzadas.

5.3.3. Arquitectura interna del backend

El backend sigue una arquitectura en capas que divide de forma clara las responsabilidades del sistema en distintos niveles funcionales. Esta estructura permite aplicar el principio de separación de responsabilidades y facilita la escalabilidad del proyecto.

Controladores

Actúan como punto de entrada a la aplicación, exponiendo los endpoints necesarios para las acciones principales:

- Registro de usuarios.
- Consulta de perfiles.
- Gestión de coincidencias.
- Mensajería.

Su función es delegar en la capa de servicios sin llegar a incorporar ninguna lógica de negocio.

Capa de servicios

La capa de servicios contiene la lógica central del sistema. Aquí se encuentran implementadas las reglas que determinan el funcionamiento real de la aplicación.

En el contexto de este proyecto se encarga de:

- Generación y gestión de matches.
- Validación de datos.
- Control del flujo de mensajes.

Esta capa es clave para garantizar que todas las reglas del sistema se aplican de forma uniforme.

Capa de repositorios

Gestiona el acceso a los datos, permitiendo operar sobre las entidades sin exponer detalles críticos de la base de datos.

Entidades

Representan las entidades definidas en el modelo relacional, permitiendo reflejar de forma directa las relaciones necesarias para que la aplicación funcione. Las entidades se desarrollarán en el apartado orientado al modelo de datos.

5.3.4. Arquitectura interna del cliente móvil

El cliente móvil sigue un patrón MVVM, lo que permite separar claramente la interfaz de la lógica de presentación de datos.

View

Se encarga exclusivamente de la representación de la información y de capturar eventos e interacciones del usuario.

ViewModel

Gestiona la comunicación con el backend y el estado de la interfaz, permitiendo:

- Realizar llamadas a la API REST.
- Gestionar los estados de carga y error.
- Transformar los datos recibidos en estructuras y formatos adecuados para la interfaz.
- Mantener el estado ante cambios de configuración del dispositivo.

Esta capa permite mantener una arquitectura limpia y evitar la sobrecarga de la capa de presentación.

Model

Define las estructuras de datos utilizadas en la aplicación, tanto a nivel de dominio como en la comunicación con el servidor

Con este patrón se facilita el mantenimiento del código y la escalabilidad del cliente.

5.3.5. Flujo general de funcionamiento del sistema

El funcionamiento del sistema desde que recibe una acción del usuario, hasta que se recibe una respuesta, sigue un flujo estructurado que garantiza la coherencia y seguridad en la gestión de la información:

1. El usuario realiza una acción en la aplicación móvil.
2. La vista comunica el evento capturado al ViewModel.
3. El ViewModel envía una solicitud HTTP al servidor.
4. El servidor procesa la petición (Validación de datos, aplicar reglas de negocio, consultar o modificar la base de datos, etc.)
5. El servidor genera la respuesta en formato JSON.
6. El cliente recibe la respuesta y actualiza la interfaz de usuario.

Con este flujo se garantiza que la lógica crítica permanezca en el servidor, reduciendo riesgos de manipulación y mejorando la consistencia de la información.

5.4 Stack tecnológico

El stack tecnológico seleccionado para el desarrollo de este proyecto ha sido definido considerando criterios de coherencia con la arquitectura, adecuación al alcance académico y alineación con los contenidos del ciclo formativo de Desarrollo de Aplicaciones Multiplataforma.

A diferencia del apartado anterior, donde se define la organización del sistema, en este apartado se detallan las tecnologías concretas utilizadas para implementar dicha arquitectura.

5.4.1. Cliente móvil

Para el desarrollo del cliente móvil se ha hecho uso de las siguientes tecnologías:

- **Android Studio** como entorno de desarrollo y framework para el desarrollo de la aplicación móvil.
- **Kotlin** como lenguaje de programación con el que se desarrollará la lógica de la aplicación.
- **Jetpack Compose** como tecnología de desarrollo para la interfaz de usuario.

La elección de Kotlin se debe a su gran integración con el ecosistema Android y a características que facilitan el desarrollo, como la gestión segura de nulos o el uso de una sintaxis más concisa.

Por su parte, Jetpack Compose permite implementar el patrón MVVM definido anteriormente, facilitando:

- La separación clara entre la lógica y la presentación.
- La creación de interfaces reactivas.
- La reutilización de componentes.

El uso de estas dos tecnologías permiten desarrollar un cliente alineado con las prácticas actuales del desarrollo Android, además de reforzar los conocimientos adquiridos en los módulos de Programación Multimedia y aplicaciones móviles y Desarrollo de Interfaces

5.4.2. Servidor (Backend)

El backend del proyecto ha sido desarrollado con las siguientes tecnologías:

- **Java** como lenguaje de programación principal para el desarrollo de la lógica.
- **Spring Boot** como framework.
- **JPA** con hibernate para la persistencia.

Spring Boot se ha seleccionado por su capacidad para estructurar aplicaciones basadas en servicios de forma rápida y organizada, lo que encaja con la arquitectura en capas definida previamente.

Su uso permite:

- Crear APIs REST de forma sencilla.
- Integrar la capa de persistencia sin configuraciones complejas.
- Organizar el proyecto de forma clara.

La elección de java se encuentra alineada con los contenidos del ciclo formativo y permite mantener coherencia con las tecnologías comúnmente utilizadas en entornos profesionales.

5.4.3. Base de datos

Para la gestión de datos se ha optado por las siguientes tecnologías:

- MySQL como sistema gestor de base de datos relacional.
- JPA con la especificación de hibernate como capa de acceso a datos.

El uso de JPA permite trabajar a través de técnicas ORM, otorgando un nivel de abstracción más alto, lo que facilita el mantenimiento del código, la gestión de relaciones y entidades y la integración con el backend.

Esta elección refuerza directamente los conocimientos del módulo de Bases de Datos y Acceso a Datos.

5.4.4. Comunicación cliente-servidor

La comunicación entre el cliente y el backend se realiza mediante el uso de API REST con protocolo HTTP y formato JSON. Para ello, se utiliza la librería retrofit en el cliente, que permite consumir la API desde el cliente Android.

Con este enfoque se consigue un desacoplamiento completo entre cliente y servidor, lo que facilita la evolución futura del sistema, como por ejemplo, con la integración de nuevos clientes.

5.5 Modelo de datos

Este apartado tiene como objetivo definir la estructura del modelo de datos utilizado en la aplicación, describiendo las principales entidades, sus atributos más relevantes y las relaciones existentes entre ellas. El diseño del modelo se ha realizado siguiendo un enfoque relacional, con el fin de garantizar la integridad de los datos, evitar duplicidades innecesarias y facilitar el desarrollo posterior de la capa de persistencia mediante Spring Data JPA.

El modelo de datos se ha diseñado teniendo en cuenta las necesidades principales de la aplicación, es decir, gestión de usuarios, configuración de perfiles musicales, selección de instrumentos y estilos, sistema de exploración y coincidencias, bloqueo de perfiles, mensajería privada y procesos de seguridad relacionados con la verificación de correo electrónico y recuperación de contraseña.

A diferencia de versiones iniciales del diseño, en la estructura final se separa la información de autenticación del usuario de la información musical del perfil. De esta forma, la entidad User almacena los datos propios de la cuenta, mientras que la entidad Profile recoge los datos visibles y musicales utilizados por el sistema de matching. Esta separación permite organizar mejor la información y facilita futuras ampliaciones del sistema.

1. User

La entidad User representa la cuenta registrada dentro de la aplicación. Su función principal es almacenar la información necesaria para la autenticación, identificación y seguridad del usuario.

Sus principales atributos son:

- **id**: identificador único del usuario dentro del sistema.
- **username**: nombre de usuario utilizado para identificarlo dentro de la aplicación.
- **email**: correo electrónico asociado a la cuenta. Este campo debe ser único para evitar registros duplicados.
- **password**: contraseña del usuario, almacenada de forma cifrada para garantizar la seguridad de las credenciales.
- **registrationDate**: fecha y hora en la que se registra el usuario.
- **emailVerified**: campo lógico que indica si el usuario ha verificado su correo electrónico.

- **tokenVersion**: campo utilizado para controlar la validez de los tokens asociados al usuario.

Esta entidad mantiene una relación uno a uno con Profile, de forma que cada usuario tiene asociado un único perfil musical. Esta separación permite mantener diferenciados los datos internos de la cuenta y los datos públicos o funcionales utilizados dentro de la aplicación.

2. Profile

La entidad Profile representa el perfil musical del usuario. Contiene la información que será visible para otros usuarios y que se utilizará para el sistema de exploración, filtrado y matching.

Sus principales atributos son:

- **id**: identificador del perfil. Este identificador se corresponde con el identificador del usuario asociado, ya que se utiliza una relación uno a uno con clave compartida.
- **experienceLevel**: nivel de experiencia general del músico, definido mediante un conjunto limitado de valores.
- **biography**: breve descripción personal o musical del usuario.
- **city**: ciudad asociada al perfil.
- **profilePicture**: URL o referencia a la imagen de perfil del usuario.
- **user**: usuario al que pertenece el perfil.

La relación entre User y Profile se implementa mediante una relación @OneToOne con @MapsId, lo que significa que el perfil utiliza el mismo identificador que el usuario. Esta decisión permite mantener una correspondencia directa entre cuenta y perfil, evitando que puedan existir perfiles sin usuario asociado.

Además, Profile se relaciona con otras entidades importantes del sistema, como City, Style, ProfileInstrument, ProfileSwipe, Match, Chat, Message y ProfileBlock.

3. City

La entidad City representa las ciudades disponibles dentro de la aplicación. Su finalidad es normalizar la localización de los usuarios, evitando que cada perfil introduzca la ciudad como texto libre.

Sus principales atributos son:

- **id:** identificador único de la ciudad.
- **name:** nombre de la ciudad. Este campo es único y obligatorio.

En el MVP, la localización del usuario se gestiona mediante la selección de una ciudad dentro de un conjunto de opciones disponibles. Esto permite simplificar el desarrollo inicial y utilizar la ciudad como criterio básico para la búsqueda y el matching. La geolocalización real mediante GPS no forma parte del modelo actual, aunque se contempla como una posible línea futura de desarrollo.

La relación entre Profile y City es de muchos a uno, ya que muchos perfiles pueden pertenecer a una misma ciudad, pero cada perfil tiene asociada una ciudad concreta.

4. Instrument

La entidad Instrument representa los instrumentos musicales disponibles en la aplicación.

Sus principales atributos son:

- **id:** identificador único del instrumento.
- **name:** nombre del instrumento, por ejemplo guitarra, piano, batería o bajo. Este campo es único y obligatorio.

Esta entidad no se relaciona directamente con Profile mediante una relación muchos a muchos simple, ya que es necesario almacenar información adicional sobre el nivel que tiene cada usuario con cada instrumento. Por este motivo, se utiliza una entidad intermedia llamada ProfileInstrument.

5. ProfileInstrument

La entidad ProfileInstrument representa la relación entre un perfil musical y un instrumento concreto. Esta entidad permite almacenar no solo qué instrumento toca un usuario, sino también el nivel de experiencia que tiene con ese instrumento.

Sus principales atributos son:

- **id:** identificador único de la relación.
- **profile:** perfil musical asociado.
- **instrument:** instrumento seleccionado por el perfil.

- **level:** nivel de experiencia del usuario en ese instrumento.

Esta estructura permite que un usuario pueda seleccionar varios instrumentos y asignar un nivel diferente a cada uno de ellos. Por ejemplo, un músico podría indicar que toca la guitarra a nivel avanzado y el piano a nivel principiante.

Desde el punto de vista del modelo relacional, ProfileInstrument actúa como una tabla intermedia enriquecida entre Profile e Instrument, ya que además de las claves foráneas incluye el atributo level. Esta decisión mejora la precisión del perfil musical y permite que el sistema de matching pueda tener en cuenta no solo los instrumentos compartidos, sino también el nivel de experiencia asociado a cada uno.

6. Style

La entidad Style representa los estilos o géneros musicales disponibles en la aplicación.

Sus principales atributos son:

- **id:** identificador único del estilo musical.
- **name:** nombre del estilo, por ejemplo rock, pop, jazz, metal o flamenco. Este campo es único y obligatorio.

La relación entre Profile y Style es de muchos a muchos, ya que un perfil puede estar asociado a varios estilos musicales y un mismo estilo puede aparecer en muchos perfiles. Esta relación se implementa mediante la tabla intermedia profile_style, que contiene las referencias al perfil y al estilo correspondiente.

Esta estructura permite que los estilos musicales sean reutilizables y estén normalizados dentro de la base de datos, evitando duplicidades y facilitando su uso en los filtros y en el sistema de matching.

7. ProfileSwipe

La entidad ProfileSwipe almacena las acciones de aceptación o rechazo que realiza un perfil sobre otro durante el proceso de exploración o matching.

Sus principales atributos son:

- **id:** identificador único de la acción.
- **fromProfile:** perfil que realiza la acción.
- **toProfile:** perfil sobre el que se realiza la acción.

- **liked**: campo lógico que indica si el perfil ha sido aceptado o rechazado.
- **createdAt**: fecha y hora en la que se registra la acción.

Esta entidad resulta esencial para el funcionamiento del sistema de matching, ya que permite registrar el historial de decisiones de los usuarios. Cuando un perfil indica que le interesa otro perfil, se almacena un registro con liked en valor positivo. Si ambos perfiles se aceptan mutuamente, el sistema puede generar una coincidencia real entre ellos mediante la entidad Match.

De esta forma, ProfileSwipe permite diferenciar entre una acción individual de interés o rechazo y un match confirmado entre dos perfiles.

8. Match

La entidad Match representa una coincidencia entre dos perfiles musicales. Esta coincidencia se genera cuando existe compatibilidad o aceptación mutua entre ambos usuarios.

Sus principales atributos son:

- **id**: identificador único del match.
- **user1**: primer perfil que forma parte de la coincidencia.
- **user2**: segundo perfil que forma parte de la coincidencia.
- **active**: campo lógico que indica si el match se encuentra activo.

La relación entre Match y Profile se establece mediante dos relaciones muchos a uno, una para cada perfil participante. Esta estructura permite representar de forma sencilla una coincidencia entre dos usuarios concretos.

El campo active permite controlar si la relación sigue vigente o si ha sido desactivada posteriormente. Esto resulta útil para mantener un historial de coincidencias sin necesidad de eliminar registros de la base de datos.

9. Chat

La entidad Chat representa una conversación entre dos perfiles. En el MVP, los chats se limitan a conversaciones privadas entre dos usuarios, aunque la estructura podría ampliarse en el futuro para permitir conversaciones grupales.

Sus principales atributos son:

- **id**: identificador único del chat.
- **user1**: primer perfil participante en la conversación.
- **user2**: segundo perfil participante en la conversación.
- **match**: match asociado al chat. Puede ser nulo si el chat se origina a partir de una solicitud u otro flujo diferente.
- **status**: estado del chat, definido mediante un enumerado.
- **blocked**: campo lógico que indica si el chat se encuentra bloqueado.
- **createdAt**: fecha y hora de creación del chat.
- **messages**: conjunto de mensajes asociados al chat.

La entidad Chat se relaciona con Profile mediante dos relaciones muchos a uno, correspondientes a los dos usuarios que participan en la conversación. Además, puede estar vinculada a un Match mediante una relación uno a uno. Esto permite que, cuando dos usuarios hacen match, se pueda crear una conversación asociada a esa coincidencia.

La relación entre Chat y Message es de uno a muchos, ya que un chat puede contener muchos mensajes, pero cada mensaje pertenece a un único chat. Además, se utiliza borrado en cascada y eliminación de huérfanos, por lo que los mensajes asociados se gestionan conjuntamente con la conversación.

10. Message

La entidad Message representa cada uno de los mensajes enviados dentro de un chat. Sus principales atributos son:

- **id**: identificador único del mensaje.
- **chat**: chat al que pertenece el mensaje.
- **sender**: perfil que envía el mensaje.
- **content**: contenido textual del mensaje.
- **createdAt**: fecha y hora de envío del mensaje.

Cada mensaje pertenece a un único chat y es enviado por un único perfil. El contenido del mensaje se almacena como texto, permitiendo conversaciones de longitud variable.

En el contexto del MVP, la mensajería funciona mediante conversaciones privadas entre dos usuarios. La comunicación en tiempo real mediante WebSockets no forma parte del modelo actual, aunque se contempla como mejora futura para ofrecer una experiencia más fluida.

11. ProfileBlock

La entidad ProfileBlock representa el bloqueo de un perfil por parte de otro. Esta funcionalidad permite mejorar la seguridad y el control de la experiencia de usuario dentro de la aplicación.

Sus principales atributos son:

- **id**: identificador único del bloqueo.
- **blocker**: perfil que realiza el bloqueo.
- **blocked**: perfil que ha sido bloqueado.

Esta relación permite registrar qué usuarios han sido bloqueados por otros, evitando interacciones no deseadas. A diferencia del campo blocked de la entidad Chat, que afecta a una conversación concreta, ProfileBlock permite representar un bloqueo entre perfiles a nivel más general.

Esta entidad resulta relevante para futuras ampliaciones del sistema, ya que permitiría impedir la aparición de perfiles bloqueados en búsquedas, matches o nuevas conversaciones.

12. PendingRegistration

La entidad PendingRegistration representa un registro pendiente de confirmación mediante correo electrónico. Su objetivo es almacenar temporalmente los datos de un usuario antes de crear definitivamente la cuenta en el sistema.

Sus principales atributos son:

- **id**: identificador único del registro pendiente.
- **username**: nombre de usuario solicitado.
- **email**: correo electrónico asociado al registro.
- **passwordHash**: contraseña cifrada introducida durante el registro.

- **tokenHash**: hash del token de verificación.
- **expiresAt**: fecha y hora de expiración del registro pendiente.
- **usedAt**: fecha y hora en la que el registro fue confirmado, en caso de haberse utilizado.
- **createdAt**: fecha y hora de creación del registro pendiente.

Esta entidad incluye restricciones de unicidad sobre el correo electrónico, el nombre de usuario y el hash del token. Gracias a ello, se evita que existan registros pendientes duplicados y se refuerza la seguridad del proceso de verificación.

El uso de PendingRegistration permite que las cuentas no se activen completamente hasta que el usuario confirme su correo electrónico. Esto mejora la fiabilidad del sistema y evita la creación de usuarios con correos no verificados.

13. PasswordResetToken

La entidad PasswordResetToken almacena los tokens utilizados para el proceso de recuperación de contraseña.

Sus principales atributos son:

- **id**: identificador único del token.
- **token**: valor único del token de recuperación.
- **user**: usuario al que pertenece el token.
- **expiresAt**: fecha y hora de expiración del token.
- **usedAt**: fecha y hora en la que el token fue utilizado.

Cada token pertenece a un usuario concreto y tiene una fecha de expiración, lo que permite limitar temporalmente su validez. Además, el campo usedAt permite controlar si el token ya ha sido utilizado, evitando reutilizaciones indebidas.

Esta entidad contribuye a reforzar la seguridad del sistema, ya que permite gestionar la recuperación de contraseña de forma controlada y trazable.

Relaciones principales del modelo

El modelo de datos se organiza en torno a la entidad Profile, ya que es la parte del usuario que participa en la mayoría de funcionalidades sociales y musicales de la aplicación.

Las relaciones principales son las siguientes:

- **User - Profile:** relación uno a uno. Cada usuario tiene un único perfil musical asociado, y cada perfil pertenece a un único usuario.
- **Profile - City:** relación muchos a uno. Muchos perfiles pueden pertenecer a una misma ciudad.
- **Profile - Instrument:** relación muchos a muchos enriquecida mediante ProfileInstrument, ya que se almacena también el nivel del usuario con cada instrumento.
- **Profile - Style:** relación muchos a muchos mediante la tabla intermedia profile_style.
- **Profile - ProfileSwipe:** relación entre perfiles que permite registrar acciones de aceptación o rechazo.
- **Profile - Match:** relación entre dos perfiles que han generado una coincidencia.
- **Match - Chat:** relación uno a uno opcional, que permite asociar una conversación a un match.
- **Chat - Message:** relación uno a muchos. Un chat puede contener varios mensajes.
- **Profile - Message:** relación muchos a uno. Cada mensaje tiene un perfil emisor.
- **Profile - ProfileBlock:** relación entre perfiles que permite gestionar bloqueos.
- **User - PasswordResetToken:** relación muchos a uno. Un usuario puede tener varios tokens de recuperación generados a lo largo del tiempo.

Gracias a estas relaciones se consigue un modelo coherente con las funcionalidades principales de MusicMatch. La separación entre cuenta de usuario y perfil musical permite organizar mejor los datos, las entidades Instrument, Style y City normalizan la información usada en el matching, y las entidades ProfileSwipe, Match, Chat y Message permiten representar el flujo social principal de la aplicación.

En conjunto, el modelo de datos diseñado permite cubrir las necesidades del MVP y, al mismo tiempo, deja una base preparada para futuras ampliaciones, como la mensajería en tiempo real, la gestión de grupos musicales, el uso de geolocalización real o la mejora del sistema de recomendaciones mediante algoritmos más avanzados.

A continuación se muestra el diagrama relacional del modelo de datos, incluyendo las principales entidades, relaciones y atributos presentes en la aplicación. La tabla `pendingRegistration` no se relaciona con ninguna entidad debido a que su único propósito es almacenar los datos de usuarios no verificados antes de su creación definitiva.

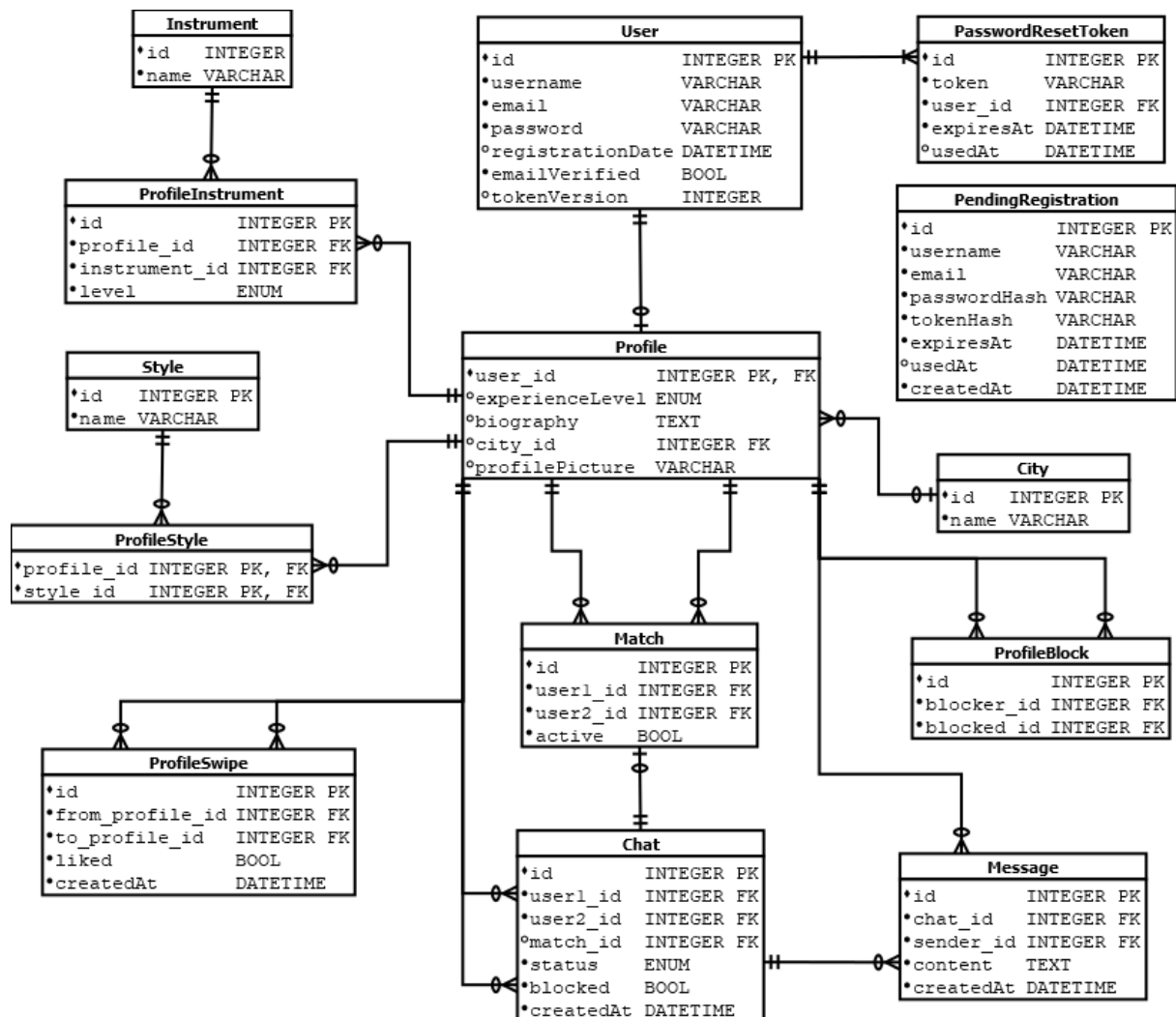


Figura 5.8 - Diagrama entidad-relación de la solución

Gracias al diseño de este modelo relacional se logra simplificar en gran medida el desarrollo de la lógica de matching entre músicos, y los campos existentes para la configuración del perfil musical.

Consideración sobre la gestión de roles

En el diseño inicial del modelo de datos se contemplaba la existencia de una entidad Rol, destinada a diferenciar distintos tipos de usuarios dentro del sistema, como músico o administrador. Esta entidad habría permitido asignar permisos específicos y controlar el acceso a determinadas funcionalidades según el tipo de usuario.

Sin embargo, en la versión actual del MVP esta funcionalidad no se ha implementado como entidad independiente dentro del modelo de datos. La aplicación se centra principalmente en el flujo del usuario músico, por lo que no ha sido necesario desarrollar todavía un sistema completo de roles y permisos. De este modo, todos los usuarios registrados siguen un flujo similar basado en la creación de una cuenta, configuración de perfil musical, exploración de otros perfiles, generación de matches y mensajería.

No obstante, la gestión de roles se mantiene como una posible mejora futura del sistema. En versiones posteriores, podría incorporarse una entidad Role o un campo enumerado dentro de User para distinguir entre usuarios músicos, administradores u otros perfiles específicos. Esta ampliación permitiría desarrollar un panel de administración completo, restringir el acceso a determinadas rutas del backend, gestionar permisos y mejorar el control interno de la plataforma.

Por tanto, aunque la gestión de roles no forma parte del modelo implementado en el MVP, se considera una decisión de diseño relevante para la evolución futura de MusicMatch, especialmente si la aplicación creciera y necesitara funciones de administración, moderación o supervisión de usuarios.

6. PLANIFICACIÓN Y GESTIÓN DEL PROYECTO

El desarrollo de la aplicación propuesta en este proyecto se ha organizado mediante una planificación estructurada que permite transformar la idea de una plataforma móvil orientada a la conexión entre músicos con intereses similares en un prototipo funcional alineado con el alcance definido en el MVP.

Dado que el objetivo principal del proyecto no es desarrollar una plataforma completa en producción, sino una versión funcional que demuestre la viabilidad del sistema de matchmaking entre músicos, la planificación se ha centrado en priorizar la creación de perfiles musicales, el sistema de coincidencias entre usuarios y la comunicación mediante mensajería.

Para organizar el desarrollo de estas funcionalidades se ha adoptado un enfoque de trabajo iterativo inspirado en la metodología ágil de Scrum. Este enfoque permite dividir el desarrollo en pequeñas fases de trabajo que facilitan el seguimiento del progreso y la adopción del proyecto conforme se avanza en su implementación.

Dentro de este proceso se han integrado dos herramientas principales que han permitido estructurar y documentar el trabajo realizado:

- **Git y GitHub**, utilizados como herramientas para el control de versiones del código del proyecto.
- **Trello**, empleado como herramienta de organización de tareas basada en la metodología de Scrum.

La combinación de estas herramientas permite mantener una relación clara entre la planificación del proyecto y el desarrollo del código, facilitando tanto la organización del trabajo como la trazabilidad de las decisiones durante la evolución del sistema.

6.1 Metodología de trabajo

Una vez se ha definido el enfoque general de planificación del proyecto, el desarrollo de la aplicación se ha organizado mediante un flujo de trabajo que permite estructurar las tareas necesarias para construir el MVP de forma progresiva.

Para ello, las funcionalidades del sistema se han dividido en tareas más pequeñas relacionadas con el desarrollo de cada parte de la aplicación. Estas tareas abarcan diferentes aspectos del proyecto, desde la preparación inicial del entorno de desarrollo hasta la implementación de las funcionalidades de matching entre músicos y mensajería.

En el contexto de la aplicación propuesta, estas tareas incluyen actividades como el diseño del modelo de datos para perfiles musicales, la implementación del sistema de registro de usuarios, el desarrollo del sistema de coincidencias por afinidad musical o la creación del sistema de mensajería entre usuarios.

La organización de estas tareas se ha realizado mediante un tablero de Trello que permite visualizar el estado de cada actividad dentro del proceso de desarrollo. Las tareas se distribuyen en diferentes columnas que representan su estado dentro del flujo de trabajo del proyecto, lo que facilita identificar qué funcionalidades están pendientes, cuales están actualmente en desarrollo y cuáles ya se han implementado.

Este sistema de organización permite mantener una visión clara del progreso del proyecto y facilita la planificación de las siguientes funcionalidades que deben implementarse para completar el MVP del proyecto.

A continuación se muestra el tablero de Trello que se ha utilizado para organizar las tareas del proyecto.

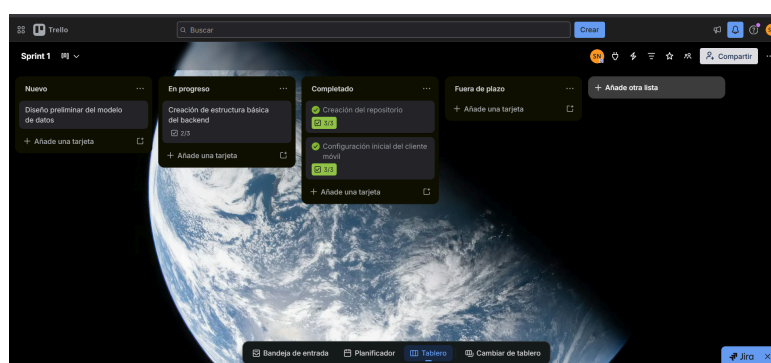


Figura 6.1 - Tablero del Sprint 1 en Trello

Con esta organización se logra visualizar de forma clara el estado en el que se encuentra cada tarea y el avance en tiempo real del prototipo del proyecto.

6.2 Product Backlog

El Product Backlog representa el conjunto de funcionalidades y tareas necesarias para desarrollar el prototipo de la aplicación. Este backlog se ha construido tomando como base las historias de usuario definidas durante el capítulo de requisitos y las tareas clave identificadas durante el análisis de usuarios.

En lugar de definir el backlog únicamente desde un punto de vista técnico, se ha organizado a partir de las acciones principales que los usuarios deben poder realizar dentro de la aplicación, permitiendo mantener el desarrollo centrado en el valor que aporta el sistema a los músicos que utilizarán la plataforma.

Entre las funcionalidades incluidas en el backlog se encuentran las siguientes:

- **Gestión de usuarios.** Permite a los músicos crear una cuenta dentro de la aplicación e iniciar sesión para acceder a sus datos personales. Esta funcionalidad constituye el punto de entrada al sistema.
- **Configuración del perfil musical.** Una vez registrado, el usuario puede configurar su perfil musical indicando la información relevante para el sistema de coincidencias, como pueden ser los instrumentos, sus estilos, experiencia o localización.
- **Exploración de músicos.** Permite visualizar otros perfiles musicales registrados en la plataforma a través de filtros o buscador por nombre, permitiendo encontrar músicos con los que no se ha establecido coincidencias.
- **Sistema de coincidencias.** El sistema genera recomendaciones de perfiles musicales que presentan una afinidad con el usuario según la información configurada en el perfil.
- **Mensajería entre usuarios.** Cuando dos usuarios aceptan una coincidencia, se habilita un canal de comunicación mediante mensajes privados que permite iniciar una colaboración musical.

Cada una de estas funcionalidades se ha dividido en tareas más pequeñas dentro del tablero de Trello, lo que permite gestionar el trabajo de forma clara y visualizar el progreso del proyecto a lo largo del tiempo.

6.3 Fases / Sprints del proyecto

El desarrollo del proyecto se ha dividido en varios sprints que reflejan las distintas etapas del desarrollo de la aplicación. Cada uno de estos sprints ha tenido como objetivo implementar una parte concreta del sistema, siguiendo una progresión lógica desde la base técnica hasta las funcionalidades visibles para el usuario.

Sprint 1 - Preparación del entorno

Durante este sprint inicial se han realizado las tareas necesarias para preparar el entorno de desarrollo del proyecto.

Entre las principales actividades realizadas destacan:

- Creación del repositorio del proyecto en GitHub.
- Configuración inicial del cliente móvil en Android Studio.
- Creación de la estructura básica del backend en Spring Boot.
- Diseño preliminar del modelo de datos para la gestión de perfiles musicales.

Este sprint constituye la base técnica sobre la que se ha construido el resto de la aplicación.

Sprint 2 - Gestión de usuarios y perfiles musicales

El segundo sprint se ha centrado en implementar el sistema de registro de usuarios y la configuración de los perfiles musicales.

Durante esta fase se ha desarrollado funcionalidades como:

- Registro de nuevos usuarios.
- Inicio de sesión.
- Creación del perfil musical.
- Selección de instrumentos, estilos, nivel de experiencia y localización.

Estas funcionalidades son fundamentales para el funcionamiento del sistema de matchmaking, ya que proporciona la información necesaria para identificar la compatibilidad de los perfiles.

Sprint 3 - Exploración de usuarios y sistema de matching

En este sprint se ha implementado el sistema que permite explorar músicos dentro de la aplicación.

Las tareas realizadas incluyen:

- Visualización de perfiles musicales.
- Implementación del sistema de matching basado en afinidad musical.
- Gestión de aceptación o rechazo de coincidencias.

Este sprint representa uno de los elementos centrales del proyecto, ya que permite materializar la propuesta de valor de la aplicación.

Sprint 4 - Sistema de mensajería

El último sprint se ha centrado en implementar el sistema de comunicación entre usuarios que han aceptado una coincidencia.

Durante esta fase se han desarrollado funcionalidades como:

- Creación de conversaciones entre usuarios.
- Envío y recepción de mensajes.
- Visualización cronológica de las conversaciones.

Este sistema permite a los músicos comenzar a organizar colaboraciones, ensayos o proyectos musicales una vez establecida la coincidencia dentro de la plataforma.

6.4 Recursos por fase

El desarrollo del proyecto se ha apoyado en recursos tecnológicos y humanos muy alineados con las necesidades concretas del MVP, priorizando la creación de perfiles musicales, el sistema de coincidencias entre usuarios y la mensajería.

Herramientas de desarrollo

- **Android Studio.** Se ha utilizado como entorno de desarrollo principal para la implementación de la aplicación móvil, garantizando la integración con Kotlin y Jetpack Compose. Este recurso es esencial para cumplir con los criterios de usabilidad y compatibilidad, permitiendo que la interfaz sea intuitiva y accesible en dispositivos móviles, tal y como se definen en los requisitos no funcionales RNF5 y RNF6.
- Para el backend, se ha empleado un entorno de desarrollo basado en Java y Spring Boot, que proporciona una estructura modular que permite mantener la separación entre la lógica de negocio y la persistencia de datos. Esto asegura que los requisitos funcionales relacionados con la gestión de usuarios, perfiles y matches (RF1 - RF9) puedan implementarse de manera consistente y escalable.

Control de versiones

El código del proyecto se ha gestionado a través de Git, con el repositorio remoto alojado en GitHub. Este enfoque, además de garantizar un historial completo de cambios con la posibilidad de revertirlos, sino que también permite:

- Mantener la integridad del código frente a cambios concurrentes durante la implementación de funcionalidades críticas como el sistema de matching.
- Documentar la evolución del proyecto, facilitando la trazabilidad de decisiones técnicas relacionadas con los requisitos funcionales.
- Gestionar ramas de desarrollo para separar la implementación de nuevas funcionalidades de la versión estable del prototipo, asegurando que el MVP siempre cumpla con los criterios de aceptación definidos.

Con estas herramientas se consigue utilizar un entorno adecuado y adaptado a las necesidades funcionales del proyecto desarrollado, además de mantener un control de versiones mediante Git.

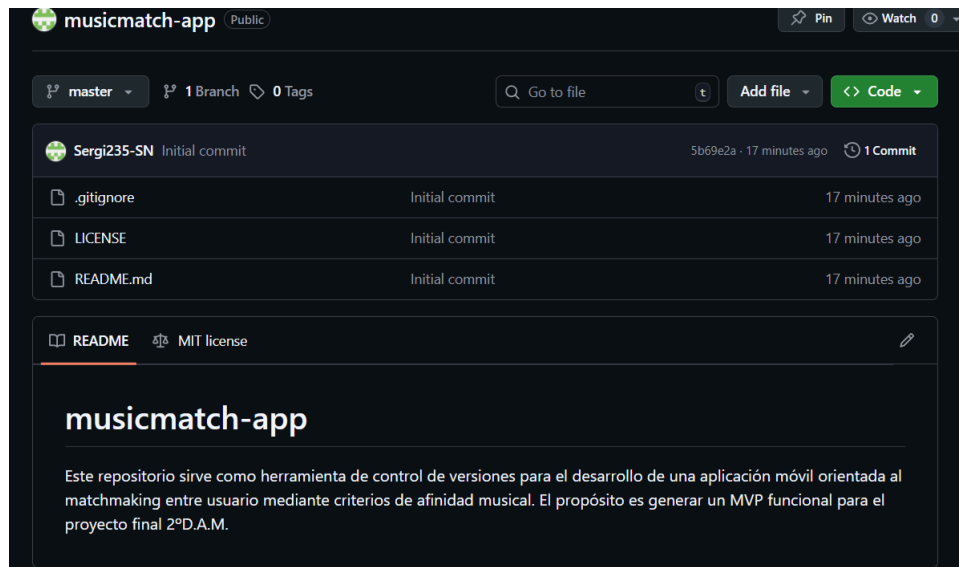


Figura 6.2 - Versión inicial del repositorio de GitHub

6.5 Criterios de aceptación

Los criterios de aceptación se han definido para garantizar que cada funcionalidad implementada cumple con los objetivos del MVP y responde a los requisitos funcionales y no funcionales establecidos previamente. Su propósito es servir de referencia objetiva para validar el correcto funcionamiento del prototipo.

Gestión de usuarios

- Un usuario puede registrarse con correo, nombre y contraseña, cumpliendo con el requisito funcional RF1 y el requisito RNF2 referente a la autenticación segura.
- El inicio de sesión recupera correctamente los datos almacenados, garantizando el requisito RF2 y el requisito de disponibilidad mínima RNF4.
- La edición de perfiles refleja los cambios en la base de datos sin inconsistencias, cumpliendo con RF6 y RNF7 (mantenibilidad y consistencia de datos).

Perfil musical y exploración de usuarios

- El sistema permite seleccionar instrumentos, estilos y nivel de experiencia de manera clara y sin errores, cumpliendo con los requisitos RF4-RF6.

- Los usuarios pueden visualizar otros perfiles musicales correctamente filtrados por ciudad o afinidad, lo que cumple los requisitos RF10 - RF12 y asegurando la usabilidad (RNF5).

Sistema de coincidencias

- Las coincidencias se generan automáticamente según criterios de afinidad musical y proximidad geográfica, cumpliendo RF7 y RF8.
- El usuario puede aceptar o rechazar coincidencias y recibir notificaciones sobre nuevas coincidencias, garantizando RF9 y RNF5.

Mensajería

- Dos usuarios que han aceptado un match pueden enviarse mensajes privados de manera confiable, cumpliendo RF13 y RF14.
- Los mensajes se muestran en orden cronológico y sin pérdida de información, cumpliendo con los criterios de rendimiento y disponibilidad (RNF1 y RNF4).

Cada criterio de aceptación está directamente relacionado con un historia de usuario y a los requisitos del MVP, de modo que la validación del prototipo puede realizarse de manera objetiva y verificable, asegurando que el proyecto cumple su propósito de conectar músicos afines de forma segura, eficiente y sencilla.

6.6 Cronograma general

Para planificar el desarrollo del proyecto se ha elaborado un cronograma temporal que organiza las distintas fases y actividades del proyecto, desde el inicio del curso académico hasta la finalización del proyecto.

Este cronograma incluye tareas relacionadas con el análisis, diseño, desarrollo, pruebas y documentación del sistema. La planificación se ha representado mediante una hoja de cálculo que permite visualizar la duración estimada de cada actividad y su distribución temporal a lo largo del proyecto.

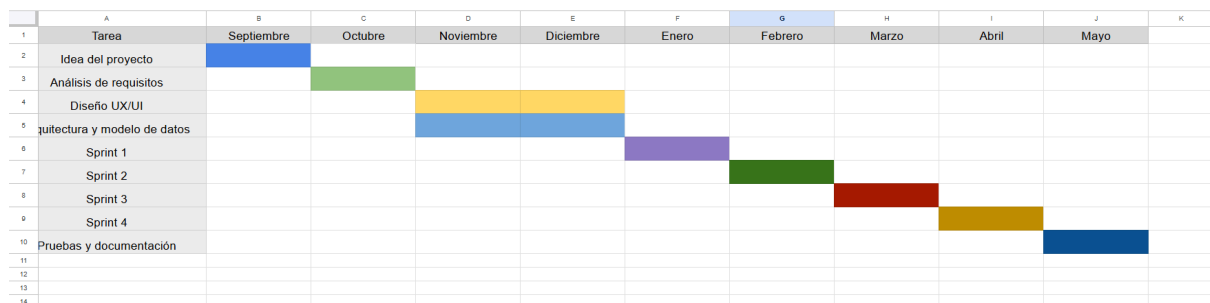


Figura 6.3 - Diagrama de Gantt del proyecto

Gracias a ese cronograma se puede obtener una visión general de las fases del proyecto y su duración total, permitiendo una mejor gestión del tiempo a lo largo del desarrollo de las mismas.

7. GESTIÓN DE RIESGOS, SEGURIDAD Y CUMPLIMIENTO

La solución desarrollada permite a los usuarios crear un perfil musical, descubrir otros músicos compatibles mediante un sistema de matching y comunicarse mediante mensajería privada. Estas funcionalidades implican el tratamiento de información personal básica y la gestión de interacciones directas entre usuarios.

En aplicaciones móviles que utilizan arquitectura cliente-servidor y servicios backend, los principales riesgos suelen estar relacionados con la seguridad en las comunicaciones, la protección de datos almacenados y el control de acceso a los servicios de la API. Estudios recientes muestran que una gran parte de los ataques contra aplicaciones móviles se concentran en los endpoints del backend y los sistemas de autenticación, ya que son los que concentran la lógica principal del sistema.

Además, en informes recientes se indica que el 62% de las organizaciones han sufrido al menos un incidente de seguridad relacionado con aplicaciones móviles, a pesar de que la mayoría considere que sus sistemas son seguros, lo que demuestra la necesidad de incorporar medidas de seguridad desde el diseño del sistema.

Por este motivo, el diseño de la solución propuesta incorpora mecanismos de seguridad alineados con las recomendaciones de organismos especializados como OWASP, especialmente en lo relacionado con la protección de datos, validación de entradas y seguridad de las comunicaciones.

7.1 Identificación de riesgos

La identificación de riesgos se ha realizado teniendo en cuenta las principales funcionalidades del sistema y su arquitectura:

- Registro y autenticación de usuarios.
- Gestión de perfiles musicales.
- Algoritmo de matching.
- Sistema de mensajería.
- API REST de comunicación cliente-servidor.

Según el estándar OWASP, algunos de los riesgos más comunes en aplicaciones móviles incluyen problemas de autenticación, almacenamiento inseguro de datos, validación incorrecta de entradas y comunicaciones no cifradas.

Teniendo en cuenta estas referencias y el funcionamiento de la aplicación desarrollada, se han identificado los siguientes riesgos:

ID	Riesgo	Descripción	Área afectada
R1	Manipulación del algoritmo de matching	Alteración del cliente para generar coincidencias falsas	Backend/ lógica de negocio
R2	Acceso no autorizado a perfiles	Usuarios acceden a información no autorizada de otros perfiles	API REST
R3	Filtración de datos personales	Exposición de correos o información sensible de los usuarios	Base de datos
R4	Interceptación de mensajes	Captura de datos transmitidos entre cliente y servidor.	Comunicación API
R5	Uso indebido del sistema de mensajería	Envío masivo de mensajes o spam	Sistema de chat
R6	Manipulación de entradas de usuario	Datos maliciosos que afecten al backend	Controladores API

Tabla 7.1 - Principales riesgos de la solución

Este análisis permite identificar los puntos críticos del sistema que requieren medidas de seguridad específicas.

7.2 Evaluación y mitigación

Una vez se han identificado estos riesgos, se ha realizado una evaluación teniendo en cuenta tanto la probabilidad de ocurrencia como el impacto que tiene sobre el funcionamiento del sistema:

Riesgo	Probabilidad	Impacto	Nivel de riesgo	Estrategia de mitigación
R1	Media	Alta	Alto	Centralizar la lógica en el backend
R2	Media	Alta	Alto	Gestionar la autenticación y el control de permisos
R3	Baja	Alta	Medio	Cifrado de credenciales
R4	Media	Media	Medio	Uso de protocolo HTTPS
R5	Media	Media	Medio	Permitir bloquear, rechazar o aceptar el chat con un usuario
R6	Media	Alta	Alto	Validar las entradas de datos

Tabla 7.2 - Evaluación y mitigación de riesgos

A continuación se detalla y justifican algunas de las estrategias de mitigación seleccionadas para cada uno de los riesgos:

Centralización de algoritmo de matching

El cálculo de coincidencias entre los músicos se realiza exclusivamente en el backend, quedando completamente aislado del cliente móvil. Esta decisión responde al principio de separar la lógica crítica del cliente para impedir que pueda ser modificado o manipulado por un atacante.

De este modo se logra evitar que un usuario pueda alterar la aplicación para generar matches artificiales o manipular el funcionamiento de la lógica interna.

Validación de datos de usuario

Los perfiles musicales contienen información introducida manualmente por el usuario, como instrumentos o estilos musicales. Si estos datos no son validados correctamente, podrían introducirse valores maliciosos que afecten a consultas o a la lógica del backend.

El riesgo de inyección de datos sigue siendo una de las causas más frecuentes de brechas de seguridad en aplicaciones con bases de datos, por lo que resulta fundamental que todas las entradas sean validadas antes de ser procesadas por la lógica interna del sistema.

Restricción del sistema de mensajería

El sistema de chat debe gestionarse de forma que los usuarios puedan decidir que usuarios pueden enviarles mensajes y que usuarios no. Para ello, se ha implementado un sistema en el que el usuario puede aceptar o rechazar la recepción de mensajes por parte de un usuario, además de permitir el bloqueo para evitar recibir solicitudes.

Con esta decisión se consigue reducir el envío de mensajes no deseados y el uso de la aplicación como un canal de spam. Este enfoque se inspira en el funcionamiento de aplicaciones de matching o de mensajería ya existentes, donde la comunicación solo se permite si ambos usuarios están de acuerdo.

7.3 Seguridad y protección de datos

La aplicación implementa diversas medidas orientadas a proteger la información de los usuarios.

Protección de credenciales

Las contraseñas se almacenan en la base de datos mediante un hash criptográfico, evitando el almacenamiento en texto plano.

Este tipo de prácticas es fundamental para reducir el impacto de posibles filtraciones de datos, ya que el almacenamiento inseguro de datos es una de las vulnerabilidades más comunes en aplicaciones móviles.

Comunicación segura

La comunicación entre el cliente Android y el backend se realiza mediante una API REST con protocolo HTTPS.

Las aplicaciones móviles tienden a operar en redes no confiables como redes públicas, lo que hace imprescindible el uso de cifrado en la capa de transporte para evitar ataques de tipo man-in-the-middle.

Protección de la API

El backend actúa como único punto de acceso a la base de datos. El cliente móvil no puede acceder a los datos de forma directa, sino que todas las operaciones deben pasar por la API REST.

Este modelo permite validar todas las solicitudes, aplicar reglas de negocio y controlar los permisos de acceso. Además, diversos estudios muestran que un gran número de ataques a aplicaciones móviles se dirigen a las APIs del backend, lo que refuerza la necesidad de centralizar el control en el servidor.

Minimización de datos personales

La aplicación solo debe almacenar la información estrictamente necesaria para el funcionamiento del sistema:

- Nombre de usuario.
- Correo electrónico.
- Ciudad aproximada.
- Instrumentos musicales.
- Estilos musicales.

No se almacenan datos especialmente sensibles como una ubicación GPS exacta o información bancaria, lo que reduce en gran medida el impacto en caso de filtraciones de datos.

7.4 Licencias y sostenibilidad

El sistema se ha desarrollado utilizando tecnologías ampliamente adoptadas en el desarrollo de aplicaciones móviles y servicios backend.

Tecnología	Licencia	Uso
Kotlin	Apache 2.0	Desarrollo del cliente móvil
Jetpack Compose	Apache 2.0	Interfaz de usuario
Spring Boot	Apache 2.0	Backend del sistema
MySQL	GPL	Base de datos relacional

Tabla 7.3 - Licencias del stack tecnológico utilizado

El uso de tecnologías de código abierto permite evitar costes de licencias, facilita la evolución futura del sistema y permite aprovechar comunidades activas de desarrollo. Además, estas tecnologías cuentan con un soporte continuo y una amplia adopción en la industria, lo que contribuye a la sostenibilidad del proyecto a largo plazo.

7.5 Consideraciones éticas y accesibilidad

El objetivo principal de la aplicación es facilitar la conexión entre músicos que buscan establecer colaboraciones o formar proyectos musicales. Este tipo de plataforma implica considerar aspectos éticos relacionados con la privacidad y la interacción entre los usuarios.

Privacidad de los perfiles

Los perfiles muestran únicamente la información relevante y relacionada con la actividad musical del usuario, evitando incluir información sensible como el correo electrónico, lo que permite mantener un equilibrio entre visibilidad dentro de la comunidad y protección de la privacidad.

Interacción responsable entre usuarios

El sistema de mensajería limita la comunicación entre usuarios a aquellos casos en los que ambos han aceptado la coincidencia o la fuente de comunicación, evitando mensajes no deseados, el contacto abusivo y el spam dentro de la plataforma.

Accesibilidad

Aunque el objetivo principal del proyecto es el desarrollo de un prototipo funcional, se han considerado algunos principios básicos de accesibilidad para garantizar que la aplicación pueda ser utilizada por el mayor número de usuarios posible. En el contexto europeo, las recomendaciones de accesibilidad digital se basan en las directrices WCAG del W3C, que establecen criterios para mejorar la percepción, comprensión y manejo de interfaces digitales.

En este proyecto, el uso de tecnologías del ecosistema Android como Jetpack Compose permite que la aplicación sea compatible con las funcionalidades de accesibilidad integradas en el sistema operativo, como lectores de pantalla, ampliación de texto o ajustes de contraste definidos por el propio usuario. De esta forma, la aplicación puede adaptarse automáticamente a las configuraciones de accesibilidad del dispositivo sin necesidad de implementar soluciones adicionales, lo que facilita su uso por personas con distintas necesidades.

8. DESARROLLO DEL PROTOTIPO

Una vez definida la planificación, la arquitectura del sistema y el diseño general de la solución, se procedió con el desarrollo del prototipo funcional de la aplicación. Esta fase consistió en transformar la propuesta teórica en una aplicación operativa compuesta por tres partes claramente diferenciadas.

El objetivo de este apartado es describir el proceso real de implementación del prototipo, explicando cómo se han desarrollado las principales funcionalidades del sistema, es decir, la autenticación de usuarios, creación y edición de perfiles musicales, la exploración de músicos, el matchmaking y la interacción entre usuarios mediante mensajería privada.

A diferencia de los capítulos anteriores, este apartado se centra en el trabajo técnico realizado durante el desarrollo. Por ello, se describen tanto la preparación del entorno como la estructura del proyecto y la implementación de las distintas capas.

8.1 Preparación del entorno de desarrollo

Antes de comenzar la implementación del prototipo fue necesario preparar un entorno de desarrollo que permitiese trabajar de forma organizada tanto en el backend como en el cliente móvil.

Las principales herramientas que se han instalado y configurado para el desarrollo son:

- **Android Studio** para el desarrollo del cliente móvil, para ello, se ha utilizado la versión **narwhal 3** en conjunto con la API de Android 36.0, que permite desarrollar la aplicación para sistemas Android modernos.
- **JDK 25** para compilar y ejecutar el código Java correspondiente al backend desarrollado en Spring Boot.
- **Git** como sistema de control de versiones para mantener el historial de cambios y facilitar la conexión con el repositorio remoto de GitHub.
- **Docker Desktop** para la creación del contenedor con la base de datos MySQL que se conectará con el backend.

El uso de Docker Desktop permitió ejecutar el servicio de persistencia en MySQL de forma aislada evitando así depender de una instalación local concreta. Esto facilita en gran medida la portabilidad del proyecto, ya que cualquier equipo puede levantar el servicio de base de datos partiendo de la misma configuración.

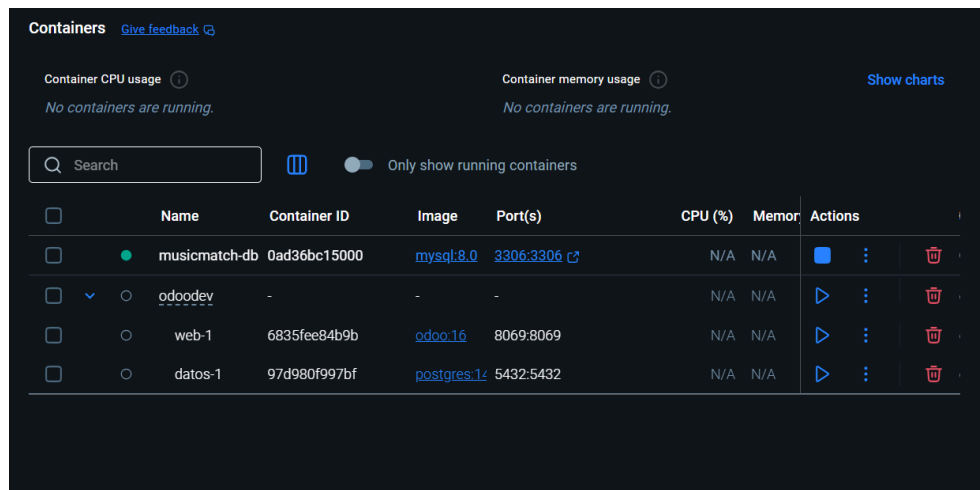


Figura 8.1 - Contenedor con servicio MySQL en Docker Desktop

8.2 Configuración del repositorio

8.2.1. Estructura del proyecto

Una vez preparado el entorno, con el fin de facilitar la organización del proyecto, se creó un repositorio de GitHub dividido en diferentes carpetas que separan las partes principales del sistema.

- La carpeta **backend** contiene el código que corresponde a la API REST desarrollada mediante Spring Boot.
- La carpeta **mobile** incluye el código de la aplicación Android que actúa como cliente del sistema.
- Por su parte, la carpeta **docs** se utiliza para almacenar la documentación y contenido adicional relacionado con el desarrollo.
- En cuanto a la carpeta **infraestructure**, esta se ha agregado con un archivo docker-compose.yaml que tiene el propósito de facilitar la creación de los servicios de backend y base de datos.

Esta estructura permite mantener una estructura clara y coherente con la arquitectura cliente-servidor definida anteriormente. Además, facilita el mantenimiento del proyecto, ya que cada parte puede evolucionar de forma independiente.

El repositorio remoto del proyecto se encuentra disponible en GitHub:

<https://github.com/Sergi235-SN/musicmatch-app>



Figura 8.2 - Estructura del proyecto en GitHub

8.2.2. Control de versiones

Para gestionar la evolución del proyecto se utilizó Git como sistema de control de versiones, permitiendo registrar los cambios realizados durante el desarrollo, conservar un historial de commits y mantener una copia remota del proyecto en GitHub.

Además, se siguió una organización basada en ramas por funcionalidad. Por ejemplo, se utilizaron ramas como `feature/auth-system` para el sistema de autenticación o `feature/matchmaking` para la lógica de coincidencias entre músicos. Este enfoque permitió trabajar cada bloque del sistema de manera más ordenada, evitando mezclar cambios en distintas funcionalidades.

Una vez comprobada cada funcionalidad, los cambios se integraban en una rama principal, manteniendo una versión estable del proyecto.



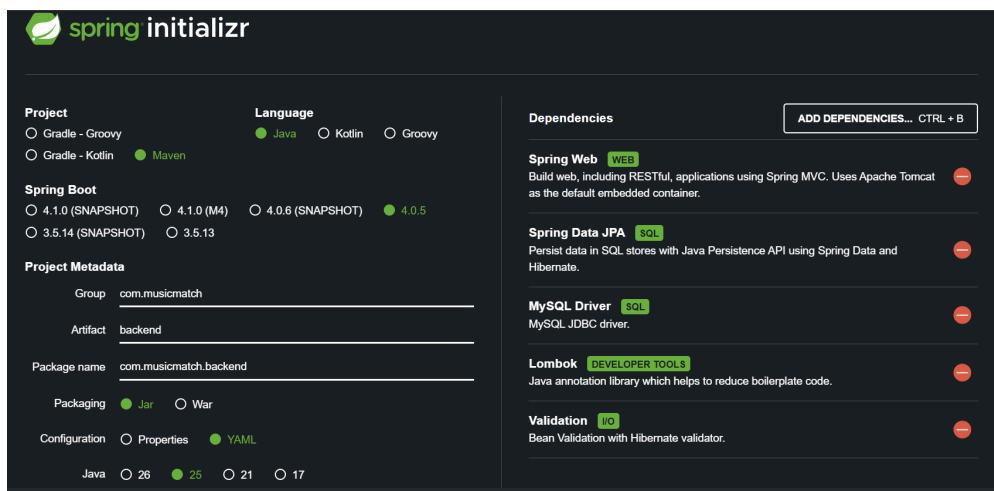
Figura 8.3 - Estructura de ramas del repositorio

8.3 Desarrollo del backend

El backend constituye el núcleo lógico del sistema. Su función principal es gestionar la autenticación, almacenar la información de los usuarios, aplicar la lógica de matching, controlar los bloqueos, gestionar chats, exponer todos los servicios a la aplicación móvil mediante una API REST

8.3.1. Creación del proyecto Spring Boot

El backend se generó utilizando Spring Boot, incorporando dependencias como Spring Web, Spring Data JPA, Spring Security, MySQL Driver y Lombok. Esta elección permitió construir una API REST organizada en capas y conectada a una base de datos relacional. Para la creación del proyecto se utilizó Spring Initializr, que permite generar un proyecto básico con las dependencias seleccionadas.



The screenshot displays the Spring Initializr web interface. On the left, under 'Project', 'Maven' is selected. Under 'Language', 'Java' is selected. Under 'Spring Boot', '4.0.5' is selected. Under 'Project Metadata', 'Group' is 'com.musicmatch', 'Artifact' is 'backend', 'Package name' is 'com.musicmatch.backend', 'Packaging' is 'Jar', 'Configuration' is 'YAML', and 'Java' version is '25'. On the right, under 'Dependencies', a list of selected dependencies is shown: 'Spring Web' (WEB), 'Spring Data JPA' (SQL), 'MySQL Driver' (SQL), 'Lombok' (DEVELOPER TOOLS), and 'Validation' (IO). Each dependency has a red minus icon to its right. A button 'ADD DEPENDENCIES... CTRL + B' is at the top right of the dependencies section.

Figura 8.4 - Inicialización del backend con Spring Boot Initializr

Spring Boot resultó adecuado para el proyecto porque reduce la configuración inicial y facilita la integración de funcionalidades importantes, como la persistencia con JPA, la seguridad con Spring Security y la exposición de endpoints mediante controladores REST.

8.3.2. Configuración de MySQL

La persistencia de datos se resolvió mediante una base de datos MySQL ejecutada en Docker. La conexión del backend con la base de datos se configuró mediante `application.yaml`, utilizando variables de entorno con valores por defecto o fallback local.

Esto permite ejecutar el proyecto tanto en un entorno local como en otros equipos sin modificar directamente el código fuente. Por ejemplo, parámetros como la URL de conexión, el usuario, la contraseña de la base de datos o la clave secreta de JWT pueden configurarse desde el entorno.

Este enfoque mejora la portabilidad y evita dejar datos sensibles escritos directamente en el código fuente.

```
spring:
  datasource:
    url: ${DB_URL:jdbc:mysql://localhost:3306/musicmatch}
    username: ${DB_USER:musicuser}
    password: ${DB_PASS:musicpass}
    driver-class-name: com.mysql.cj.jdbc.Driver
    hikari:
      initializationFailTimeout: 60000

  jpa:
    hibernate:
      ddl-auto: update
    show-sql: ${SHOW_SQL:false}
    properties:
      hibernate:
        dialect: org.hibernate.dialect.MySQLDialect

  mail:
    host: ${SPRING_MAIL_HOST:localhost}
    port: ${SPRING_MAIL_PORT:587}
    username: ${SPRING_MAIL_USERNAME:}
    password: ${SPRING_MAIL_PASSWORD:}
    properties:
```

Figura 8.5 - Configuración del archivo `application.yaml`

8.3.3. Implementación de la capa de configuración

La capa de configuración contiene clases necesarias para preparar el funcionamiento general del backend. Entre las más relevantes se encuentran SecurityConfig, JwtAuthenticationFilter, WebConfig SchedulingConfig y DataInitializer.

La clase SecurityConfig define la configuración principal de seguridad. En ella se desactiva el uso de sesiones mediante SessionCreationPolicy.STATELESS, ya que la autenticación se basa en tokens JWT. También se indican que endpoints son públicos, como el registro, el login, la verificación de correo o la recuperación de contraseña, y qué endpoints requieren de autenticación.

Además, en esta clase se declara el uso de BCryptPasswordEncoder, utilizado para almacenar las contraseñas de forma segura. Esto implica que la contraseña real del usuario nunca se guarda directamente en la base de datos, sino que se almacena su hash.

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .csrf(AbstractHttpConfigurer::disable)
        .cors(Customizer.withDefaults())
        .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .formLogin(AbstractHttpConfigurer::disable)
        .httpBasic(AbstractHttpConfigurer::disable)
        .exceptionHandling(ex -> ex
            .authenticationEntryPoint((request, response, authException) ->
                writeError(response, HttpStatus.UNAUTHORIZED, message: "No autenticado"))
            .accessDeniedHandler((request, response, accessDeniedException) ->
                writeError(response, HttpStatus.FORBIDDEN, message: "No autorizado"))
        )
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(
                "/api/auth/register",
                "/api/auth/login",
                "/api/auth/refresh",
                "/api/auth/verify-email",
                "/api/auth/resend-verification",
                "/api/auth/forgot-password",
                "/api/auth/reset-password",
                "/api/auth/reset-password-page",
                "/api/auth/verification-status",
                "/api/auth/ping"
            ).permitAll()
            .requestMatchers("/api/profile/avatar/**").permitAll()
            .anyRequest().authenticated()
        )
        .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
    return http.build();
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

private void writeError(HttpServletResponse response, HttpStatus status, String message) throws IOException {
    response.setStatus(status.value());
    response.setCharacterEncoding(StandardCharsets.UTF_8.name());
    response.setContentType(MediaType.APPLICATION_JSON_VALUE);

    ApiResponse<Void> body = new ApiResponse<>(success: false, message, data: null);
    response.getWriter().write(objectMapper.writeValueAsString(body));
}
```

Figura 8.6 - Seguridad de endpoints y cifrado de contraseña

Por otra parte, `JwtAuthenticationFilter` intercepta las peticiones protegidas, obtiene el token enviado en la cabecera `Authorization`, comprueba que sea válido y verifica que sea un token de tipo `access`. Si el token es correcto, se establece el usuario autenticado dentro del contexto de seguridad de Spring.

El uso de JWT permite que el servidor no tenga que almacenar sesiones. Cada petición protegida incluye un token que identifica al usuario, lo que facilita una arquitectura más ligera y adecuada para una API REST.

La clase `WebConfig` configura CORS, permitiendo definir los orígenes autorizados mediante variables de entorno. Esto facilita adaptar el backend a distintos entornos, como desarrollo local, pruebas o despliegue.

También se añadió `SchedulingConfig`, que habilita tareas programadas, y `DataInitializer`, que carga automáticamente datos iniciales de instrumentos, estilos musicales y ciudades cuando la base de datos está vacía. Esto permite probar la aplicación desde el primer arranque sin tener que insertar manualmente todos los datos.

8.3.4. Implementación de entidades

A partir del modelo de datos definido en el capítulo 5, se implementaron las entidades principales del sistema a partir de JPA. Estas entidades representan las tablas de la base de datos y las relaciones entre ellas.

La entidad Profile tiene un papel central dentro del sistema, ya que almacena la información utilizada para la búsqueda y el matching, incluyendo estilos, instrumentos, ciudad, nivel de experiencia, biografía y foto de perfil. Esta entidad está relacionada con User mediante @OneToOne y utiliza @MapsId, de forma que el perfil comparte el mismo identificador que el usuario.

También se utilizan relaciones @ManyToMany para vincular perfiles con estilos musicales, y @OneToMany para relacionar un perfil con sus instrumentos mediante la entidad intermedia ProfileInstrument.

```
@Entity
@Table(name = "profile")
@Getter
@Setter
public class Profile {

    @Id
    private Long id;

    @Enumerated(EnumType.STRING)
    @Column(name = "experience_level")
    private ExperienceLevel experienceLevel;

    @Column(columnDefinition = "TEXT")
    private String biography;

    @ManyToOne
    @JoinColumn(name = "city_id")
    private City city;

    @Column(name = "profile_picture_url")
    private String profilePicture;

    @OneToOne
    @MapsId
    @JoinColumn(name = "user_id")
    private User user;

    @ManyToMany
    @JoinTable(
        name = "profile_style",
        joinColumns = @JoinColumn(name = "profile_id"),
        inverseJoinColumns = @JoinColumn(name = "style_id")
    )
    private Set<Style> styles;

    @OneToMany(mappedBy = "profile", cascade = CascadeType.ALL)
    private List<ProfileInstrument> profileInstruments;
}
```

Figura 8.7 - Definición de la clase Profile

El sistema de matching se apoya en entidades como Match y ProfileSwipe. La primera representa una coincidencia confirmada entre dos perfiles, mientras que la segunda registra si un usuario ha aceptado o rechazado a otro. Esto permite evitar mostrar repetidamente perfiles ya valorados y controlar la reciprocidad necesaria para crear un match.

Además se implementó una entidad ProfileBlock, que permite registrar bloqueos entre usuarios. Cuando un usuario bloquea a otro, el sistema elimina swipes, chats y matches previos entre ambos, evitando futuras interacciones.

También se agregaron entidades específicas para el sistema de autenticación. Por un lado, PendingRegistration permite almacenar temporalmente una solicitud de registro antes de crear el usuario definitivo. En este caso, el usuario no se guarda directamente en la tabla User hasta que verifica su correo electrónico. Por su parte, PasswordResetToken almacena los tokens temporales utilizados para restablecer contraseñas.

Esta decisión mejora el control del proceso de registro, ya que evita crear cuentas definitivas que no han confirmado todavía su dirección de correo electrónico.

8.3.5. Implementación de repositorios

Una vez se definió la parte del dominio, se implementó la capa de repositorios mediante Spring Data JPA. Esta capa permite acceder a la base de datos sin escribir manualmente la mayoría de consultas SQL.

En los casos más sencillos se utilizaron métodos derivados del nombre, como búsquedas por email, nombre de usuario o identificador. En otros casos se añadieron consultas personalizadas mediante @Query, especialmente cuando era necesario comprobar relaciones entre dos perfiles independientemente del orden.

Un ejemplo importante aparece en MatchRepository, donde se comprueba si ya existe un match entre dos perfiles, independientemente de cual sea user1 y cual sea user2. Esto evita duplicados y garantiza que solo exista una coincidencia entre dos usuarios.

También destaca ChatRepository, que permite localizar conversaciones entre dos perfiles, obtener chats activos y recuperar solicitudes pendientes.

```

public interface MatchRepository extends JpaRepository<Match, Long> {

    @Query("""
        SELECT m FROM Match m
        WHERE (m.user1 = :p1 AND m.user2 = :p2)
           OR (m.user1 = :p2 AND m.user2 = :p1)
        """)
    Optional<Match> findBetweenProfiles(Profile p1, Profile p2);

    @Query("""
        SELECT CASE WHEN COUNT(m) > 0 THEN true ELSE false END
        FROM Match m
        WHERE (m.user1 = :p1 AND m.user2 = :p2)
           OR (m.user1 = :p2 AND m.user2 = :p1)
        """)
    boolean existsBetweenProfiles(Profile p1, Profile p2);

    @Modifying
    @Query("""
        DELETE FROM Match m
        WHERE (m.user1.id = :p1 AND m.user2.id = :p2)
           OR (m.user1.id = :p2 AND m.user2.id = :p1)
        """)
    void deleteBetweenProfiles(Long p1, Long p2);

    @Query("""
        SELECT m FROM Match m
        WHERE m.user1 = :profile OR m.user2 = :profile
        """)
    List<Match> findAllByProfile(Profile profile);
}

```

Figura 8.8 - Definición del repositorio para consultas de matches

Esta capa es importante porque separa el acceso a datos de la lógica de negocio. De esta forma, los servicios pueden trabajar con métodos ya preparados y centrarse en aplicar las reglas funcionales del sistema.

8.3.6. Implementación de servicios

La capa de servicios concentra la lógica principal del backend. En ella se desarrollan las operaciones que realmente definen el comportamiento de la aplicación.

Los servicios principales implementados son los siguientes:

Servicio de autenticación

UserService gestiona el ciclo completo de autenticación. En el registro se validan los datos introducidos, se normaliza el email, se comprueba que el usuario no existe y se almacena una solicitud temporal en PendingRegistration.

La contraseña se cifra mediante BCrypt antes de guardarse. Además, el token de verificación de correo electrónico no se almacena en texto plano, sino como un hash SHA-256. De este modo, aunque alguien accediera a la base de datos, no podría utilizar directamente el token de verificación.

Cuando el usuario pulsa el enlace de verificación, el sistema comprueba el token, valida que no esté caducado ni utilizado, crea el usuario definitivo en la tabla User y genera también su perfil asociado.

El login solo se permite si el usuario existe, la contraseña coincide con el hash almacenado y el correo ha sido verificado. Si todo es correcto, se generan dos tokens JWT:

- **Access Token.** Se encarga de autenticar las peticiones protegidas del backend.
- **Refresh Token.** Permite renovar el token de acceso de forma periódica.

```

private ApiResponse<Void> verifyPendingRegistrationToken(String token) {
    String tokenHash = hashToken(token);

    PendingRegistration pending = pendingRegistrationRepository.findByTokenHash(tokenHash).orElse(other: null);

    if (pending == null) {
        return new ApiResponse<>(success: false, message: "Token de verificación inválido", data: null);
    }

    if (pending.getUsedAt() != null) {
        return new ApiResponse<>(success: false, message: "Este enlace de verificación ya fue utilizado", data: null);
    }

    if (isExpired(pending)) {
        return new ApiResponse<>(success: false, message: "El enlace de verificación ha expirado", data: null);
    }

    if (userRepository.findByEmail(pending.getEmail()).isPresent()) {
        return new ApiResponse<>(success: false, message: "Ese correo ya fue verificado previamente", data: null);
    }

    if (userRepository.findByUsername(pending.getUsername()).isPresent()) {
        return new ApiResponse<>(success: false, message: "El nombre de usuario ya está en uso", data: null);
    }

    User user = new User();
    user.setUsername(pending.getUsername());
    user.setEmail(pending.getEmail());
    user.setPassword(pending.getPasswordHash());
    user.setEmailVerified(emailVerified: true);

    User saved = userRepository.save(user);

    Profile profile = new Profile();
    profile.setUser(saved);
    profileRepository.save(profile);

    pending.setUsedAt(LocalDateTime.now());
    pendingRegistrationRepository.save(pending);

    return new ApiResponse<>(success: true, message: "Correo verificado correctamente", data: null);
}

```

Figura 8.9 - Método para la verificación de correo electrónico

Servicio de email

Para el envío de correos se definió una interfaz EmailService, con dos métodos principales, uno para el envío de enlace de verificación de cuenta y otro para el envío de enlace para restablecer la contraseña. Esta decisión permite definir dos medios de verificación:

- **MockEmailService.** Muestra los enlaces a través de logs en la terminal. Es útil durante el desarrollo, ya que ahorra la configuración de un servicio SMTP para pruebas.
- **SmtplibEmailService.** Envía correos electrónicos reales a través de un servicio SMTP.

Gracias a `@ConditionalOnProperty`, el sistema puede cambiar entre un modo y otro mediante la variable de entorno `MAIL_ENABLED`. De este modo, en desarrollo puede conservarse la verificación basada en logs, mientras que para el despliegue se debería usar un servicio SMTP.

Esto aporta mayor flexibilidad y portabilidad, ya que el proyecto no depende obligatoriamente de tener configurado un correo real para poder probar la verificación.

```
@Service
@ConditionalOnProperty(name = "mail.enabled", havingValue = "true")
@RequiredArgsConstructor
public class SmtplibEmailService implements EmailService {

    private final JavaMailSender mailSender;

    @Value("${app.mail.from:no-reply@musicmatch.com}")
    private String from;

    @Value("${app.auth.verify-email-url-base}")
    private String verifyEmailUrlBase;

    @Value("${app.auth.reset-password-url-base}")
    private String resetPasswordUrlBase;

    @Override
    public void sendVerificationEmail(String to, String username, String token) {
        String verifyUrl = verifyEmailUrlBase + token;
        String safeUsername = escape(username);

        String subject = "Verifica tu cuenta de MusicMatch";

        String plainText =
            "Hola " + username + ",\n\n" +
            "Gracias por registrarte en MusicMatch.\n\n" +
            "Pulsa el botón \"Verificar cuenta\" para completar el proceso.\n\n" +
            "Si no has creado una cuenta, puedes ignorar este mensaje.";

        String html = buildEmailTemplate(
            title: "Verifica tu cuenta",
            ""
            Hola <strong>%s</strong>,<br><br>
            Gracias por registrarte en MusicMatch.<br><br>
            Pulsa el botón para verificar tu cuenta y continuar.
            ""
            .formatted(safeUsername),
            buttonText: "Verificar cuenta",
            verifyUrl
        );

        sendHtmlEmail(to, subject, plainText, html);
    }
}
```

Figura 8.10 - Servicio de envío de enlace por SMTP

Servicio de perfiles

ProfileService gestiona la edición parcial del perfil musical. Permite modificar la biografía, la ciudad, el nivel de experiencia, los estilos, los instrumentos y la imagen de perfil.

Para la imagen se valida que el archivo no esté vacío, que no supere el tamaño máximo permitido y que sea JPG o PNG. Además, se genera un nombre único mediante UUID, evitando conflictos entre archivos con mismo nombre.

También se incluye un endpoint público para servir imágenes desde la carpeta uploads, aplicando comprobaciones para evitar accesos a rutas no permitidas.

Servicio de matching

MatchService es uno de los servicios centrales del proyecto, ya que implementa el sistema de coincidencias entre músicos.

Antes de mostrar candidatos, el servicio comprueba que el perfil de usuario esté completo. Después filtra perfiles que no deben mostrarse, como perfiles incompletos, bloqueados, ya aceptados, ya emparejados o rechazados recientemente.

La puntuación de afinidad se calcula teniendo en cuenta varios factores:

Criterio	Puntuación
Misma ciudad	40 puntos
Mismo nivel de experiencia	20 puntos
Compatibilidad instrumental	30 puntos
Coincidencias de estilos	Hasta 20 puntos

Tabla 8.1 - Criterios de afinidad del sistema de matching

Solo se muestran perfiles que superan el umbral mínimo de afinidad. Además, los candidatos se ordenan de mayor a menor puntuación y se limita el número de resultados para evitar sobrecargar la respuesta.

Cuando un usuario hace un swipe positivo y el otro también lo ha aceptado previamente, se crea automáticamente un match y un chat activo entre ambos.

```

private double score(Profile a, Profile b) {

    double s = 0;

    if (a.getCity() != null && b.getCity() != null &&
        a.getCity().getId().equals(b.getCity().getId())) {
        s += 40;
    }

    if (a.getExperienceLevel() == b.getExperienceLevel()) {
        s += 20;
    }

    boolean compatibleInstrument = a.getProfileInstruments().stream()
        .anyMatch(i1 -> b.getProfileInstruments().stream()
            .anyMatch(i2 ->
                !i1.getInstrument().getId().equals(i2.getInstrument().getId())
                && i1.getLevel() == i2.getLevel()
            )
        );

    if (compatibleInstrument) s += 30;

    Set<Long> s1 = a.getStyles().stream().map(Style::getId).collect(Collectors.toSet());
    Set<Long> s2 = b.getStyles().stream().map(Style::getId).collect(Collectors.toSet());

    Set<Long> inter = new HashSet<>(s1);
    inter.retainAll(s2);

    Set<Long> union = new HashSet<>(s1);
    union.addAll(s2);

    s += union.isEmpty() ? 0 : ((double) inter.size() / union.size()) * 20;

    return s;
}

```

Figura 8.11 - Método de generación de puntuación de afinidad

El sistema también contempla bloqueos. Si un usuario bloquea a otro, se elimina cualquier swipe, chat o match existente entre ambos. De esta forma, el bloqueo tiene un efecto real sobre la interacción entre usuarios.

Servicio de chat

ChatService gestiona la comunicación entre usuarios. Permite crear o recuperar una solicitud de chat, aceptar o rechazar conversaciones pendientes, enviar mensajes y consultar conversaciones activas.

Antes de enviar un mensaje, el servicio comprueba que el chat exista, que esté activo, que el usuario pertenezca a la conversación y que el contenido no esté vacío. Esto evita accesos no autorizados y asegura que solo los participantes puedan escribir en un chat.

Los mensajes se recuperan por orden cronológico, lo que permite facilitar al usuario el seguimiento de la conversación de forma coherente.

8.3.7. Implementación de DTOs

Para la comunicación entre el backend y la aplicación móvil se utilizaron DTOs. Estos objetos permiten enviar y recibir únicamente los datos necesarios para cada operación, evitando exponer directamente las entidades internas de la base de datos.

Por ejemplo, `UserProfileResponse` devuelve los datos que necesita la app para mostrar el perfil del usuario autenticado, como nombre, biografía, ciudad, estilos, instrumentos, foto y email. En cambio, `UpdateProfileRequest` permite recibir únicamente los campos que el usuario desea modificar.

También se utilizó una clase genérica `ApiResponse<T>`, que unifica muchas respuestas del backend mediante tres campos:

- **success.** Indica si la operación ha sido exitosa.
- **message.** Mensaje que explica la causa del error.
- **data.** Datos que devuelve la operación.

Con esto se facilita la integración con la app móvil, ya que el cliente puede interpretar las respuestas de manera uniforme.

```
@Data
@AllArgsConstructor
public class ApiResponse<T> {
    private boolean success;
    private String message;
    private T data;
}
```

Figura 8.12 - Clase genérica para respuestas del servidor

8.3.8. Implementación de la API REST

La API REST se implementó separando los endpoints en controladores específicos, cada uno asociado a un bloque funcional o servicio.

AuthController incluye endpoints públicos como /register, /login, /verify-email, /forgot-passowrd y /reset-password. También incluye /me, que permite obtener los datos del usuario autenticado a partir de JWT.

ProfileController permite actualizar el perfil mediante PATH, consultar opciones musicales, obtener ciudades y subir imágenes de perfil. También permite acceder al perfil público de otros usuarios.

MatchController expone las operaciones del sistema de matching, es decir, la obtención de candidatos, hacer swipe, bloquear o desbloquear usuarios y consultar usuarios bloqueados.

ChatController permite iniciar solicitudes de chat, aceptar o rechazar solicitudes de otros usuarios, enviar mensajes y obtener chats activos o pendientes.

Por último, ProfileSearchController permite buscar perfiles aplicando filtros de texto, experiencia, estilos o instrumentos.

```
@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {

    private final UserService userService;
    private final JwtUtil jwtUtil;
    private final UserRepository userRepository;

    @PostMapping("/register")
    public ApiResponse<UserResponse> register(@RequestBody RegisterRequest request) {
        return userService.register(request);
    }

    @PostMapping("/login")
    public ApiResponse<LoginResponse> login(@RequestBody LoginRequest request) {
        return userService.login(request);
    }

    @PostMapping("/refresh")
    public ApiResponse<RefreshTokenResponse> refresh(@RequestBody RefreshTokenRequest request) {
        return userService.refresh(request);
    }

    @GetMapping("/verify-email")
    public ResponseEntity<String> verifyEmail(@RequestParam("token") String token) {
        ApiResponse<Void> result = userService.verifyEmail(token);

        if (result.isSuccess()) {
            return ResponseEntity.ok()
                .contentType(MediaType.TEXT_HTML)
                .body(buildSuccessHtml());
        }

        return ResponseEntity.badRequest()
            .contentType(MediaType.TEXT_HTML)
            .body(buildErrorHtml(result.getMessage()));
    }

    @PostMapping("/resend-verification")
    public ApiResponse<Void> resendVerification(@RequestBody EmailRequest request) {
        return userService.resendVerification(request);
    }
}
```

Figura 8.13 - Controlador de endpoints de autenticación

8.3.9 Gestión de excepciones

Para mejorar la robustez del backend se implementó un manejador global de excepciones mediante `GlobalExceptionHandler`. Esta clase centraliza la respuesta que tiene el backend frente a errores frecuentes, como recursos no encontrados, peticiones incorrectas, accesos no autorizados, archivos inválidos o conflictos de datos.

Además, se definieron excepciones personalizadas como `ResourceNotFoundException`, `BadRequestException`, `ForbiddenException` y `UserBlockedException`, lo que permite que los servicios expresen los errores de forma clara y que la API devuelva respuestas controladas al cliente.

El uso de este manejador de excepciones evita que los errores internos se muestran directamente al usuario y permite mantener una respuesta coherente en formato JSON.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiResponse<Void>> handleNotFound(ResourceNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ApiResponse<>(success: false, ex.getMessage(), data: null));
    }

    @ExceptionHandler(BadRequestException.class)
    public ResponseEntity<ApiResponse<Void>> handleBadRequest(BadRequestException ex) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body(new ApiResponse<>(success: false, ex.getMessage(), data: null));
    }

    @ExceptionHandler(ForbiddenException.class)
    public ResponseEntity<ApiResponse<Void>> handleForbidden(ForbiddenException ex) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN)
            .body(new ApiResponse<>(success: false, ex.getMessage(), data: null));
    }

    @ExceptionHandler(UserBlockedException.class)
    public ResponseEntity<ApiResponse<Void>> handleUserBlocked(UserBlockedException ex) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN)
            .body(new ApiResponse<>(success: false, ex.getMessage(), data: null));
    }

    @ExceptionHandler(MissingRequestHeaderException.class)
    public ResponseEntity<ApiResponse<Void>> handleMissingHeader(MissingRequestHeaderException ex) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
            .body(new ApiResponse<>(success: false, message: "Cabecera de autorización requerida", data: null));
    }
}
```

Figura 8.14 - Clase global de gestión de excepciones del backend

8.4 Desarrollo de la aplicación móvil

8.4.1. Creación del proyecto Android Studio

La aplicación móvil se desarrolló en Android Studio utilizando Kotlin y Jetpack Compose. Esta parte del prototipo actúa como cliente del sistema, por lo que su función principal es mostrar la interfaz al usuario, recoger sus acciones y comunicarse con el backend mediante API REST.

Para mantener una estructura organizada, el cliente móvil se dividió en varias capas siguiendo el patrón MVVM descrito en el apartado de arquitectura del capítulo 5. Esta separación permite que la interfaz no dependa directamente de las llamadas HTTP, sino que delegue la lógica en clases intermedias.

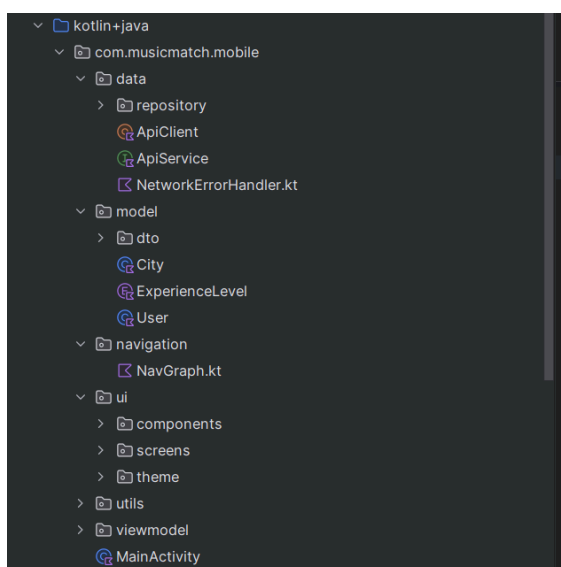


Figura 8.15 - Estructura de carpetas del cliente Android

8.4.2. Desarrollo de las ventanas principales

La aplicación incluye diferentes pantallas que cubren los flujos principales del prototipo, es decir, la autenticación, la configuración del servidor, matches, exploración de perfiles, matches, visualización y edición de perfil, y mensajería.

Una de las primeras pantallas desarrolladas fue SplashScreen, que se muestra al iniciar la aplicación. Esta pantalla no solo cumple una función visual, sino que también realiza comprobaciones iniciales importantes como la carga de la IP del servidor, verificación de la conexión con el endpoint /api/auth/ping y decide si el usuario debe ir a la configuración del servidor, al login o directamente a la pantalla principal si ya existe una sesión válida.

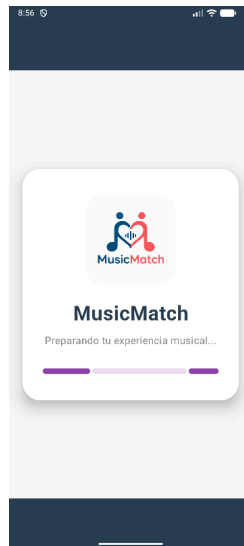


Figura 8.16 - Pantalla de carga de la aplicación

La ventana de registro se implementó mediante `RegisterScreen`. En ella se recogen los datos básicos del usuario. Para evitar la duplicación de código se crearon componentes reutilizables como `CustomLabelledTextField` y `PasswordLabelledTextField`, que permiten mantener una apariencia uniforme en los formularios de la aplicación. Además, el campo de contraseña incluye visibilidad alternable y configuración de específica de teclado.

El registro no crea el usuario inmediatamente, sino que envía una solicitud al backend y muestra una ventana que exige verificación de correo para poder continuar con la configuración del perfil musical.

```

Box(
    modifier = Modifier
        .fillMaxWidth()
        .weight( weight = 1f),
    contentAlignment = Alignment.Center
) {
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 40.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Column(verticalArrangement = Arrangement.spacedBy( space = 20.dp)) {
            CustomLabelledTextField(
                label = "Email",
                value = user.email,
                onValueChange = viewModel::onEmailChange
            )

            CustomLabelledTextField(
                label = "Usuario",
                value = user.username,
                onValueChange = viewModel::onUsernameChange
            )

            PasswordLabelledTextField(
                label = "Contraseña",
                value = user.password,
                onValueChange = viewModel::onPasswordChange
            )
        }
    }
}

```

Figura 8.17 - Definición de la ventana de registro con Jetpack Compose

La pantalla de matches se desarrolló en MatchesScreen y representa una de las funcionalidades principales del prototipo. Esta pantalla carga los candidatos desde el backend y los muestra mediante una tarjeta de perfil reutilizable (ProfileMatchCard). En cada tarjeta se enseña la información básica del músico relacionada con la afinidad, entre ellos, el nombre, el nivel de experiencia, los estilos, la ciudad y los instrumentos. Además, se incluyen acciones para aceptar o rechazar al usuario mediante botones visuales.

Cuando no existen candidatos disponibles, se muestra un estado vacío. Si el perfil de usuario todavía no está completo, la pantalla muestra un aviso indicando que debe completar su perfil para poder acceder al sistema de coincidencias.

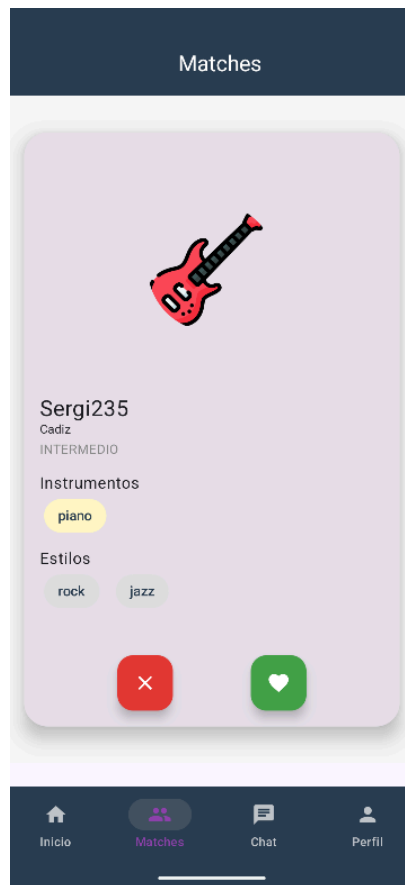


Figura 8.18 - Demostración de coincidencia generada en ventana de matching

También se implementó la parte de mensajería mediante ChatScreen y ChatDetailScreen. La primera muestra el listado de conversaciones activas, incluyendo el nombre del otro usuario, su imagen y el último mensaje. La segunda muestra los mensajes de una conversación concreta, diferenciando visualmente los mensajes recibidos de los mensajes enviados. Además, esta pantalla actualiza periódicamente la conversación para simular una conversación en tiempo real.



Figura 8.19 - Demostración de ventana de chat

Los componentes visuales reutilizables, como SmartChip, se utilizaron para representar instrumentos y estilos musicales. En el caso de los instrumentos, el color del chip cambia según el nivel de experiencia, lo que ayuda a identificar rápidamente la información musical del perfil.

8.5 Integración cliente-servidor

La integración entre la aplicación móvil y el backend se realizó mediante Retrofit, una librería que permite consumir APIs REST desde Android de forma sencilla y estructurada.

La clase ApiClient centraliza la creación del servicio Retrofit. En ella se configura la URL base del servidor, el cliente HTTP con OkHttp, los tiempos máximos de conexión, lectura y escritura, y el conversor Gson para transformar automáticamente los datos JSON en objetos Kotlin.

```

object ApiClient {

    4 Usages
    @Volatile
    private var apiService: ApiService? = null

    3 Usages
    @Volatile
    private var currentBaseUrl: String? = null

    1 Usage
    fun getService(): ApiService {
        val baseUrl = normalizeBaseUrl(url = NetworkConfig.BASE_URL)

        if (apiService == null || currentBaseUrl != baseUrl) {
            synchronized( lock = this ) {
                if (apiService == null || currentBaseUrl != baseUrl) {
                    apiService = buildRetrofit(baseUrl).create(ApiService::class.java)
                    currentBaseUrl = baseUrl
                }
            }
        }

        return apiService!!
    }

    1 Usage
    private fun buildRetrofit(baseUrl: String): Retrofit {
        val client = OkHttpClient.Builder()
            .connectTimeout( timeout = 10, unit = TimeUnit.SECONDS)
            .readTimeout( timeout = 15, unit = TimeUnit.SECONDS)
            .writeTimeout( timeout = 15, unit = TimeUnit.SECONDS)
            .retryOnConnectionFailure( retryOnConnectionFailure = true)
            .build()

        val gson = GsonBuilder().create()

        return Retrofit.Builder()
            .baseUrl(baseUrl)
            .client(client)
            .addConverterFactory( factory = GsonConverterFactory.create(gson))
            .build()
    }
}

```

Figura 8.20 - Definición de la clase ApiClient con retrofit

La interfaz ApiService define todos los endpoints consumidos por la aplicación móvil. En ella se incluyen las llamadas relacionadas con la autenticación, perfil, matches, chats, búsqueda de usuarios, subida de avatar y verificación de correo. Cada método corresponde a una ruta del backend y utiliza anotaciones como @GET, @POST, @PATCH, @Multipart, @Header, @Body o @Path.

Para las rutas protegidas se envía el JWT en la cabecera Authorization con el formato Bearer token. Esta lógica se encapsula en los repositorios, evitando que las pantallas tengan que construir manualmente las cabeceras de autenticación.

```

interface ApiService {

    @POST( value = "api/auth/register")
    suspend fun register(@Body request: RegisterRequest): ApiResponse<UserResponse>

    @POST( value = "api/auth/login")
    suspend fun login(@Body request: LoginRequest): ApiResponse<LoginResponse>

    @POST( value = "api/auth/refresh")
    suspend fun refresh(@Body request: RefreshTokenRequest): ApiResponse<RefreshTokenResponse>

    @POST( value = "api/auth/resend-verification")
    suspend fun resendVerification(@Body request: EmailRequest): ApiResponse<Any?>

    @POST( value = "api/auth/forgot-password")
    suspend fun forgotPassword(@Body request: EmailRequest): ApiResponse<Any?>

    @POST( value = "api/auth/reset-password")
    suspend fun resetPassword(@Body request: ResetPasswordRequest): ApiResponse<Any?>

    @GET( value = "api/auth/me")
    suspend fun getCurrentUser(
        @Header( value = "Authorization") token: String
    ): ApiResponse<UserProfileResponse>
}

```

Figura 8.21 - Definición de clase ApiService con peticiones REST

Los repositorios actúan como capa intermedia entre los ViewModels y la API. Por ejemplo, UserRepository gestiona login, registro, recuperación de contraseña, perfil y avatar, MatchRepository gestiona candidatos, swipes, bloqueos y solicitudes de chat, y ChatRepository se encarga de conversaciones y mensajes. Esta separación ayuda a mantener los ViewModels más limpios y facilita el mantenimiento del código.

Además, se implementó safeApiCall junto con NetworkErrorHandler, que convierte errores de red, errores HTTP o errores enviados por el backend en mensajes comprensibles para el usuario. Por ejemplo, se gestionan situaciones como servidor no disponible, sesión expirada, recurso no encontrado o error interno.

La gestión de sesión se realiza mediante TokenManager, que almacena el access token y el refresh token en SharedPreferences. También permite comprobar si existe sesión, limpiar tokens y extraer el identificador del usuario desde el payload del JWT. Esto permite mantener la sesión iniciada entre aperturas de la aplicación.

Por último, NetworkConfig y ServerConfig permiten configurar la IP del servidor. Esto resulta útil durante el desarrollo, ya que la aplicación puede conectarse tanto al emulador mediante 10.0.2.2 como a un backend ejecutado en otro equipo de la red local.

8.6 Resultado del prototipo

Como resultado del desarrollo se obtuvo un prototipo funcional de MusicMatch, compuesto por una aplicación móvil Android conectada a un backend Spring Boot y a una base de datos MySQL.

En conjunto, el prototipo demuestra la viabilidad técnica del proyecto, ya que integra correctamente el cliente móvil con el servidor y permite completar el flujo principal de la aplicación: registrarse, verificar la cuenta, iniciar sesión, configurar el perfil musical, encontrar músicos compatibles e iniciar una conversación.

Aunque se trata de una versión académica y limitada, el resultado obtenido constituye una base funcional sobre la que podrían añadirse mejoras futuras, como notificaciones push, mensajería en tiempo real mediante WebSockets, filtros avanzados o despliegue completo en un servidor externo.

9. DESPLIEGUE E INSTALACIÓN

El sistema MusicMatch ha sido preparado para su ejecución en dos contextos diferenciados. Por un lado, un entorno de producción basado en la nube (Railway). Por el otro, un entorno local orientado a pruebas y desarrollo mediante contenedores Docker.

Además, se ha implementado un sistema de integración y despliegue continuo (CI/CD) mediante GitHub Actions, que automatiza la generación de versiones instalables del proyecto.

Esta arquitectura permite garantizar tanto la disponibilidad del sistema en un entorno real como la reproducibilidad del mismo en entornos locales controlados.

9.1 Requisitos previos

Los requisitos para el despliegue del sistema dependen del entorno en el que se vaya a ejecutar.

En el caso del entorno de producción en Railway, únicamente es necesario disponer de una cuenta en la plataforma, el repositorio del proyecto conectado a GitHub y una base de datos configurada dentro del propio entorno de Railway. No se requiere instalación local de dependencias, ya que la plataforma se encarga automáticamente de la compilación y ejecución del backend.

Por otro lado, para la ejecución en local mediante Docker, es necesario disponer de Docker instalado y en funcionamiento, así como de un dispositivo Android o emulador para poder ejecutar la aplicación móvil. También es necesario que el dispositivo y el servidor se encuentren en la misma red para permitir la comunicación.

En cuanto al sistema de automatización, es necesario contar con GitHub Actions habilitado en el repositorio y trabajar sobre la rama release, que es la encargada de activar el pipeline de construcción y generación de versiones.

9.2 Proceso de despliegue

9.2.1 Despliegue en Railway

El backend del sistema se despliega en Railway con el objetivo de ofrecer un servicio accesible desde cualquier ubicación.

El proceso consiste en vincular el repositorio del proyecto con Railway, crear un servicio backend y configurar las variables de entorno necesarias para la ejecución. Entre estas variables se incluyen los datos de conexión a la base de datos, la clave secreta para la generación de tokens JWT y la configuración de CORS.

Una vez configurado, Railway se encarga automáticamente de compilar el proyecto, desplegarlo y mantenerlo en ejecución. El estado del servicio puede verificarse mediante endpoints de prueba como `/api/auth/ping`, lo que permite comprobar que el backend está operativo.

Este enfoque permite disponer de un entorno estable y persistente sin necesidad de intervención manual tras cada actualización del código.

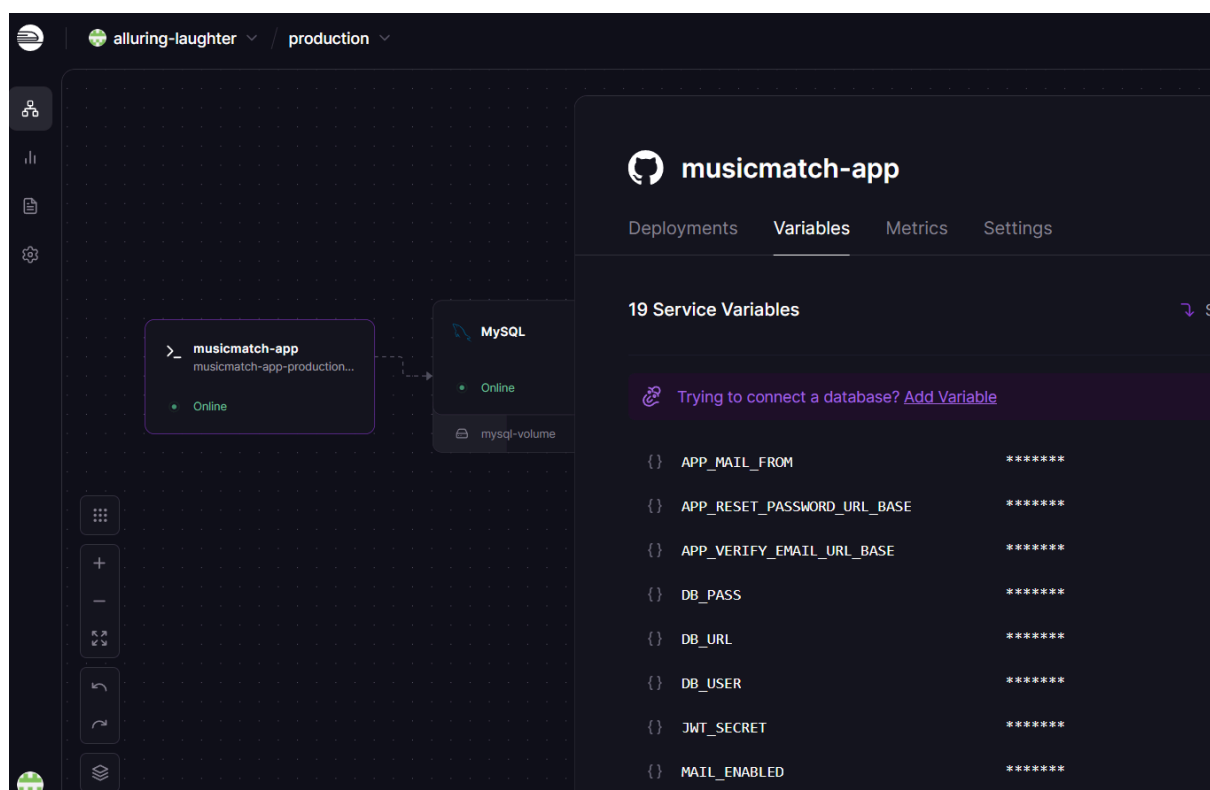


Figura 9.1 - Servicio backend y MySQL funcionando desde Railway

9.2.2 Generación de release con CI/CD

El proyecto incorpora un pipeline de integración continua implementado mediante GitHub Actions. Este pipeline se ejecuta automáticamente al realizar un push sobre la rama release.

El flujo de trabajo se divide en varias fases:

- **Ejecución de tests del backend**, garantizando que el código no presenta errores antes de continuar.
- **Construcción del backend**, generando el archivo JAR y posteriormente una imagen Docker que contiene el servidor completo.
- **Construcción de la aplicación móvil**, generando un archivo APK listo para instalar en dispositivos Android.
- **Empaquetado final**, donde se agrupan todos los elementos necesarios (imagen Docker, APK, infraestructura y documentación) en un único archivo comprimido.
- **Creación de una release**, que queda disponible en el repositorio para su descarga.

Este sistema permite automatizar completamente la generación de versiones del proyecto, facilitando tanto su distribución como su instalación en diferentes entornos.

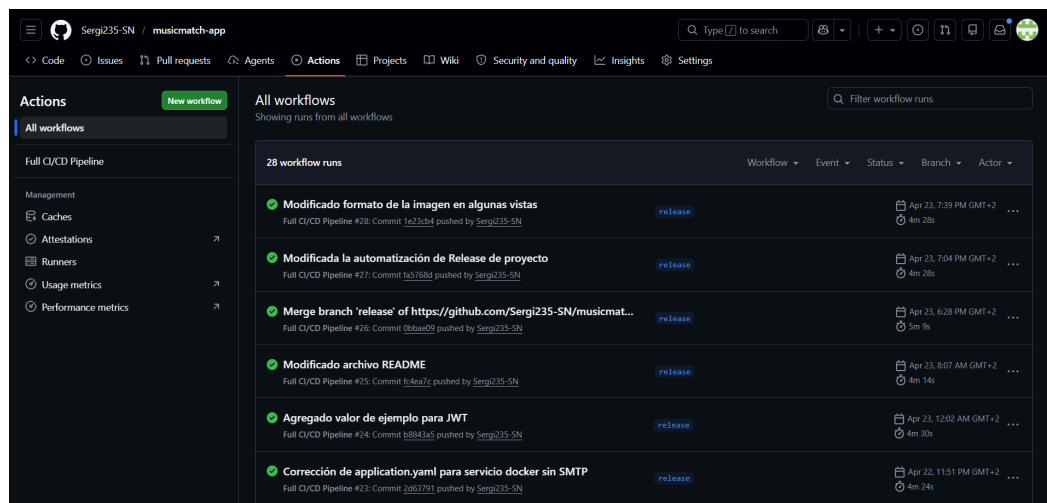
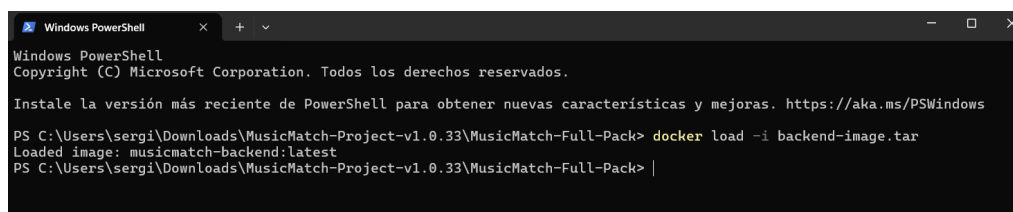


Figura 9.2 - Ejecución del flujo CI/CD en GitHub Actions

9.3 Instalación y puesta en marcha

El sistema puede ejecutarse en entorno local a partir del paquete generado mediante el pipeline de integración continua. El proceso comienza con la descarga del archivo de release desde el repositorio y su descompresión en una carpeta local, donde se incluyen todos los elementos necesarios

Una vez descomprimido el contenido, es necesario importar la imagen Docker del backend en el sistema. Este proceso se realiza mediante el comando correspondiente según el sistema operativo, lo que permite registrar la imagen dentro del entorno Docker local y prepararla para su ejecución.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

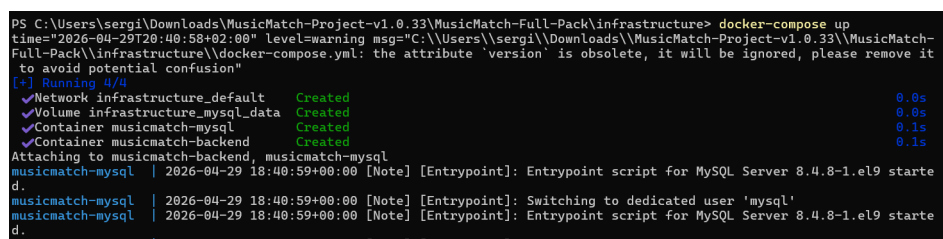
Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\sergi\Downloads\MusicMatch-Project-v1.0.33\MusicMatch-Full-Pack> docker load -i backend-image.tar
Loaded image: musicmatch-backend:latest
PS C:\Users\sergi\Downloads\MusicMatch-Project-v1.0.33\MusicMatch-Full-Pack> |
```

Figura 9.3 - Carga de imagen del servicio del backend en docker

A continuación, se deben revisar y, en su caso, ajustar las variables de entorno definidas en el archivo de configuración (docker-compose.yml o .env). Estas variables permiten adaptar el comportamiento del sistema, especialmente en lo relativo a la conexión con la base de datos, la configuración de seguridad mediante JWT y el uso opcional del servicio de correo electrónico.

Una vez completada la configuración, el sistema se pone en marcha mediante Docker Compose, que se encarga de levantar los servicios definidos y gestionar su ejecución de forma conjunta. Este proceso puede realizarse en primer plano o en segundo plano, según las necesidades del usuario.



```
PS C:\Users\sergi\Downloads\MusicMatch-Project-v1.0.33\MusicMatch-Full-Pack\infrastructure> docker-compose up
time="2026-04-29T20:49:58+02:00" level=warning msg="C:\Users\sergi\Downloads\MusicMatch-Project-v1.0.33\MusicMatch-Full-Pack\infrastructure\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[*] Running 4/4
✓Network infrastructure_default Created 0.0s
✓Volume infrastructure_mysql_data Created 0.0s
✓Container musicmatch-mysql Created 0.1s
✓Container musicmatch-backend Created 0.1s
Attaching to musicmatch-backend, musicmatch-mysql
musicmatch-mysql | 2026-04-29 18:40:59+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.8-1.el9 starte
d.
musicmatch-mysql | 2026-04-29 18:40:59+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
musicmatch-mysql | 2026-04-29 18:40:59+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.8-1.el9 starte
d.
musicmatch-mysql | 2026-04-29 18:40:59+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
musicmatch-mysql | 2026-04-29 18:40:59+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.8-1.el9 starte
d.
```

Figura 9.4 - Creación de servicios con docker compose

Tras el arranque del backend, se procede a la instalación de la aplicación móvil en un dispositivo Android o emulador, utilizando el archivo APK incluido en el paquete. Al iniciar la aplicación por primera vez, es necesario configurar la dirección IP del equipo donde se está ejecutando el backend, de modo que la aplicación pueda establecer correctamente la comunicación con el servidor.

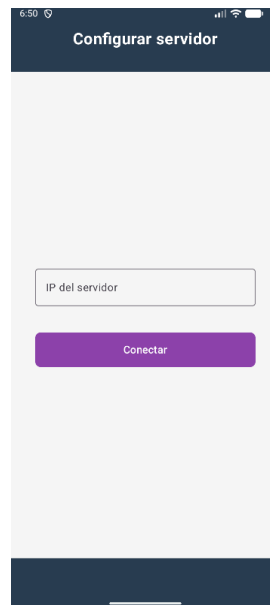


Figura 9.5 - Ventana de configuración de dirección ip en cliente móvil

Finalmente, para verificar que el sistema funciona correctamente, se puede realizar una comprobación mediante el endpoint de prueba `/api/auth/ping`. Si el servidor responde correctamente, se confirma que el backend está operativo y listo para su uso.

```
PS C:\Users\sergi> Invoke-RestMethod http://localhost:8080/api/auth/ping
ok
PS C:\Users\sergi> |
```

Figura 9.6 - Ejecución de endpoint ping en powershell

Este procedimiento permite desplegar el sistema completo en un entorno local de forma controlada y reproducible, facilitando tanto la validación del funcionamiento como la realización de pruebas sin depender del entorno de producción.

9.4 Manual de uso básico

El sistema MusicMatch ha sido diseñado con el objetivo de ofrecer una experiencia de usuario sencilla e intuitiva, centrada en la interacción entre músicos y la generación de conexiones basadas en afinidades musicales. El flujo de uso del sistema sigue una secuencia lógica que permite al usuario registrarse, configurar su perfil y comenzar a interactuar con otros usuarios.

Al iniciar la aplicación, se ejecuta automáticamente un proceso de comprobación de conectividad con el backend. En primer lugar, la aplicación intenta establecer conexión con el servidor desplegado en Railway, lo que permite el uso del sistema en producción sin necesidad de configuración adicional. En caso de no poder acceder a dicho servidor, la aplicación permite al usuario introducir manualmente la dirección del backend, lo que facilita su uso en entornos locales o de pruebas. Este comportamiento aporta flexibilidad y hace que la aplicación sea independiente del entorno de ejecución.

Una vez establecida la conexión, el usuario accede a la pantalla de autenticación. En este punto puede iniciar sesión o registrarse. El proceso de registro requiere la introducción de un nombre de usuario, un correo electrónico y una contraseña. Tras enviar el formulario, el sistema crea una cuenta asociada al usuario, pero esta no se activa inmediatamente.

El sistema implementa un mecanismo de verificación por correo electrónico, mediante el cual se envía automáticamente un enlace al usuario. Este enlace contiene un token único que permite validar la cuenta. Hasta que no se complete este proceso, el usuario no podrá acceder al sistema. Este diseño garantiza que las cuentas estén asociadas a direcciones de correo válidas y evita registros automatizados o fraudulentos.

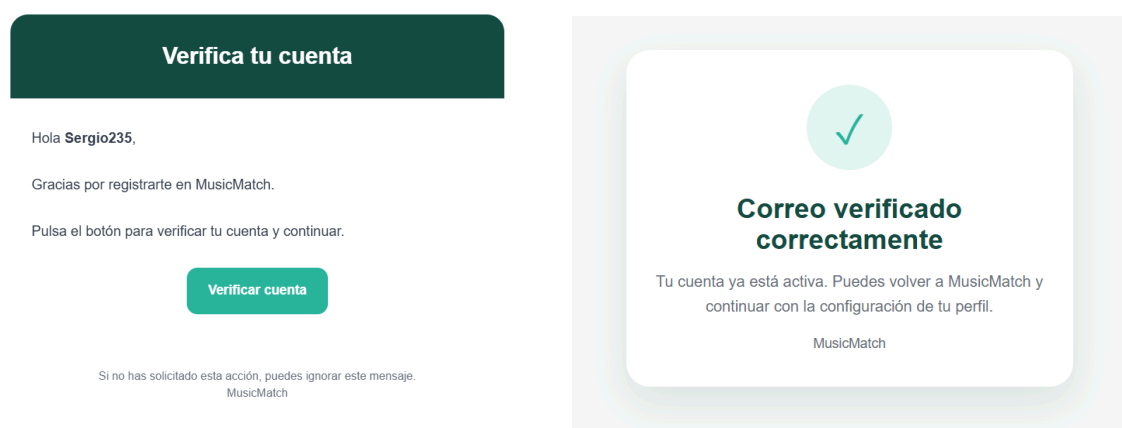


Figura 9.7 - Demostración de verificación de correo con SMTP

Una vez verificada la cuenta, el usuario puede iniciar sesión introduciendo sus credenciales. En este momento, el sistema genera un token JWT que se almacena en el dispositivo. Este token permite mantener la sesión activa y se envía en cada petición al backend, evitando la necesidad de autenticarse de forma continua.

Tras acceder al sistema, el usuario puede configurar su perfil musical. Este perfil constituye el núcleo funcional de la aplicación, ya que toda la lógica de matching se basa en la información contenida en él. El usuario puede indicar sus estilos musicales preferidos, los instrumentos que domina, su nivel de experiencia y otros datos relevantes como su ubicación o biografía.

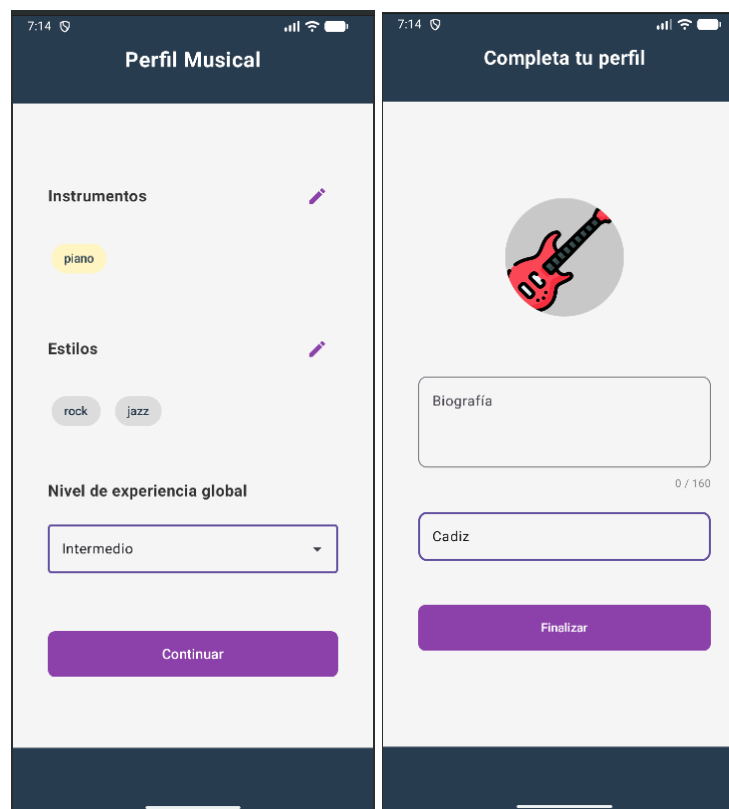


Figura 9.8 - Ventanas de configuración de perfil musical en cliente Android

A partir de esta información, el sistema ejecuta el proceso de matching, comparando los perfiles registrados y generando coincidencias en función de criterios como afinidad musical, compatibilidad de instrumentos y nivel de experiencia. Este proceso permite al usuario descubrir otros músicos con intereses similares, facilitando la interacción dentro de la aplicación.

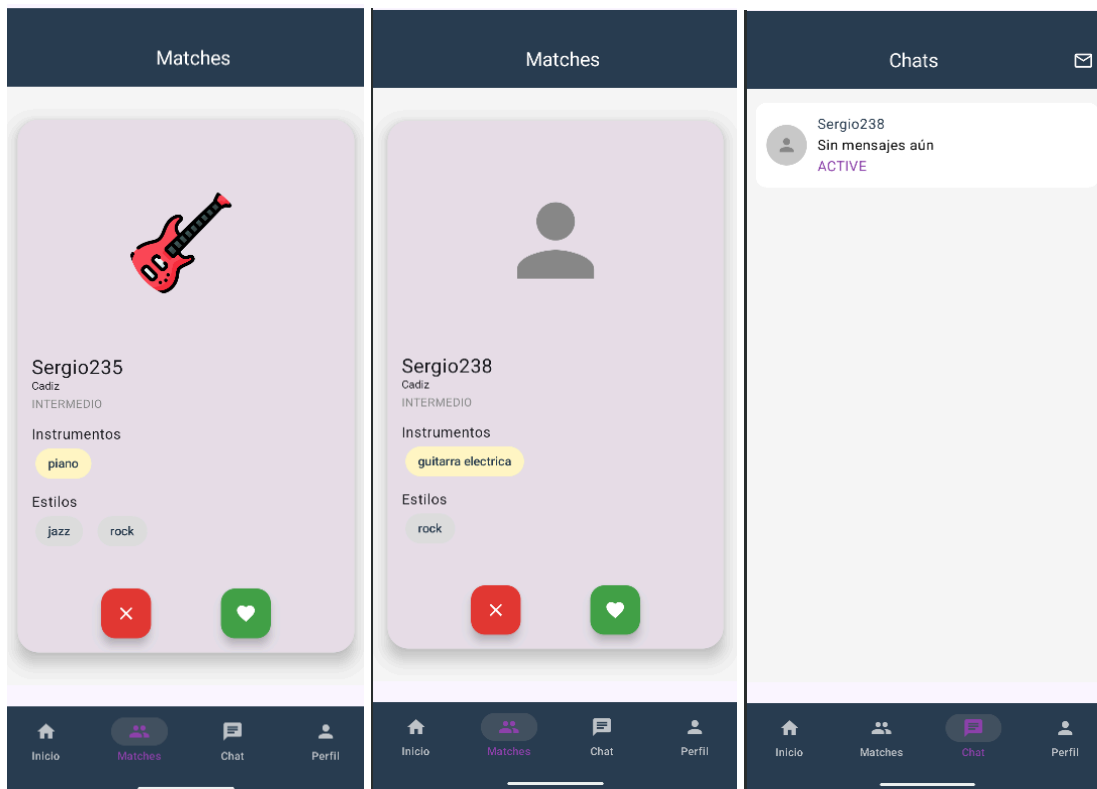


Figura 9.9 - Demostración de aceptación de swipe y generación de chat

Finalmente, el sistema incluye un mecanismo de recuperación de contraseña, que permite al usuario restablecer su acceso en caso de olvido. Este proceso se realiza mediante el envío de un correo electrónico con un enlace seguro, desde el cual el usuario puede definir una nueva contraseña.

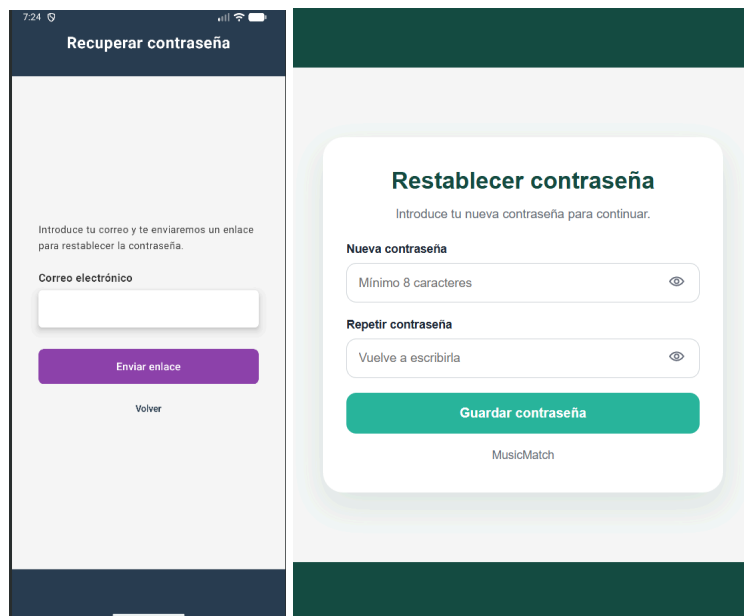


Figura 9.10 - Ventanas de restablecimiento de contraseña

En conjunto, el flujo de uso del sistema cubre todo el ciclo de vida del usuario, desde el registro inicial hasta la interacción con otros perfiles, garantizando tanto la usabilidad como la seguridad de la información.

9.5 Documentación técnica de ejecución

El sistema MusicMatch se basa en una arquitectura cliente-servidor, en la que el backend actúa como núcleo central encargado de gestionar la lógica de negocio, la seguridad y la persistencia de los datos, mientras que la aplicación móvil funciona como cliente encargado de la interacción con el usuario.

El backend ha sido desarrollado utilizando Spring Boot, lo que permite construir aplicaciones robustas y escalables. La aplicación sigue una arquitectura en capas, en la que se diferencian claramente los controladores, los servicios y los repositorios. Esta separación de responsabilidades facilita el mantenimiento del sistema y mejora su escalabilidad, ya que permite modificar cada capa de forma independiente.

Los controladores exponen los endpoints de la API REST, a través de los cuales la aplicación móvil se comunica con el backend. Estos endpoints gestionan operaciones como el registro de usuarios, la autenticación, la recuperación de contraseña o la obtención de perfiles.

La lógica de negocio se concentra en la capa de servicios, donde se implementan las operaciones fundamentales del sistema. Entre ellas destaca el servicio de matching, que constituye el núcleo funcional del proyecto, ya que es el encargado de comparar perfiles y generar coincidencias entre usuarios.

Por otro lado, la capa de repositorios utiliza Spring Data JPA para gestionar el acceso a la base de datos. Este enfoque permite abstraer gran parte de las operaciones de persistencia, reduciendo la necesidad de escribir consultas SQL manuales y facilitando el desarrollo.

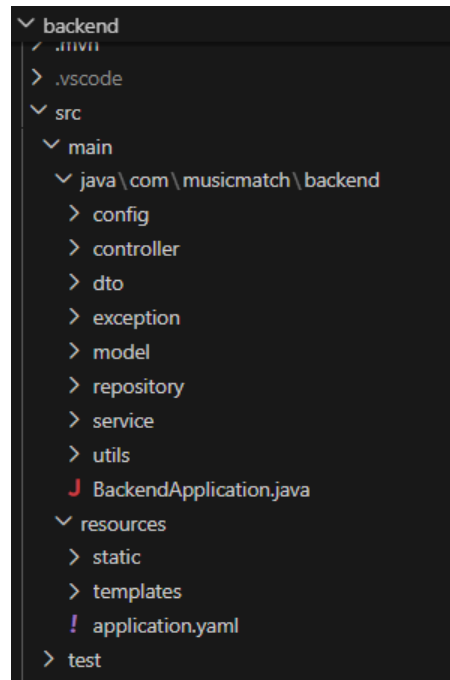


Figura 9.11 - Estructura de carpetas del backend

En cuanto a la seguridad, el sistema implementa un modelo basado en tokens JWT. Este enfoque permite trabajar con un sistema stateless, en el que el servidor no mantiene sesiones activas. En su lugar, cada petición incluye un token que identifica al usuario.

Este token se genera durante el proceso de autenticación y se envía en cada petición HTTP a través de la cabecera con el formato Authorization: Bearer <token>.

El backend valida este token mediante un filtro de seguridad, permitiendo o denegando el acceso en función de su validez.

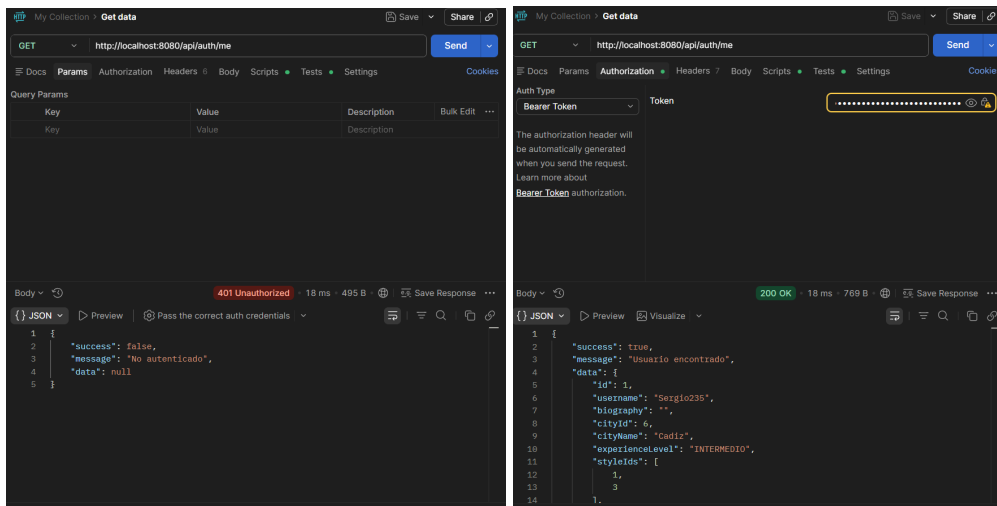


Figura 9.12 - Prueba de endpoint protegido desde postman

La base de datos utilizada es MySQL, desplegada en Railway en el entorno de producción. El uso de Hibernate permite gestionar automáticamente el esquema de la base de datos mediante la configuración `ddl-auto=update`, lo que simplifica el desarrollo y evita la necesidad de crear manualmente las tablas.

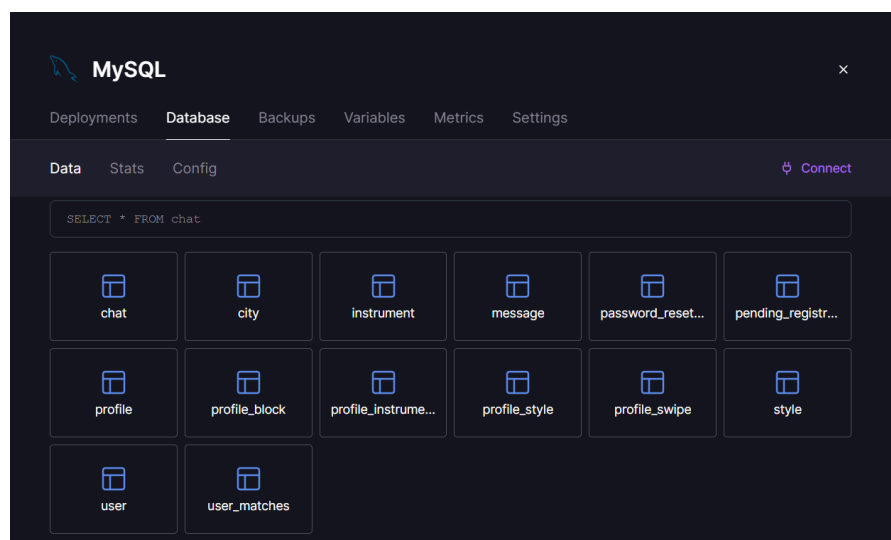


Figura 9.13 - Estructura de tablas vista desde el servicio MySQL en Railway

La aplicación móvil, desarrollada en Kotlin, utiliza Retrofit para la comunicación con el backend. Este enfoque permite realizar peticiones HTTP de forma sencilla y estructurada, manejando automáticamente la serialización y deserialización de datos en formato JSON.

Además, la aplicación implementa una gestión de sesión basada en almacenamiento local del token JWT, lo que permite mantener la autenticación incluso tras reiniciar la aplicación.

Uno de los aspectos más importantes del sistema es el uso de variables de entorno para su configuración. Estas variables permiten definir parámetros críticos como la conexión a la base de datos, la configuración SMTP o las claves de seguridad sin necesidad de modificar el código fuente.

Este enfoque resulta especialmente útil en entornos como Railway, donde las variables pueden configurarse directamente desde la plataforma.

En conjunto, la arquitectura del sistema permite garantizar la escalabilidad, la seguridad y la flexibilidad necesarias para su despliegue en distintos entornos, cumpliendo con los requisitos establecidos en el desarrollo del proyecto.

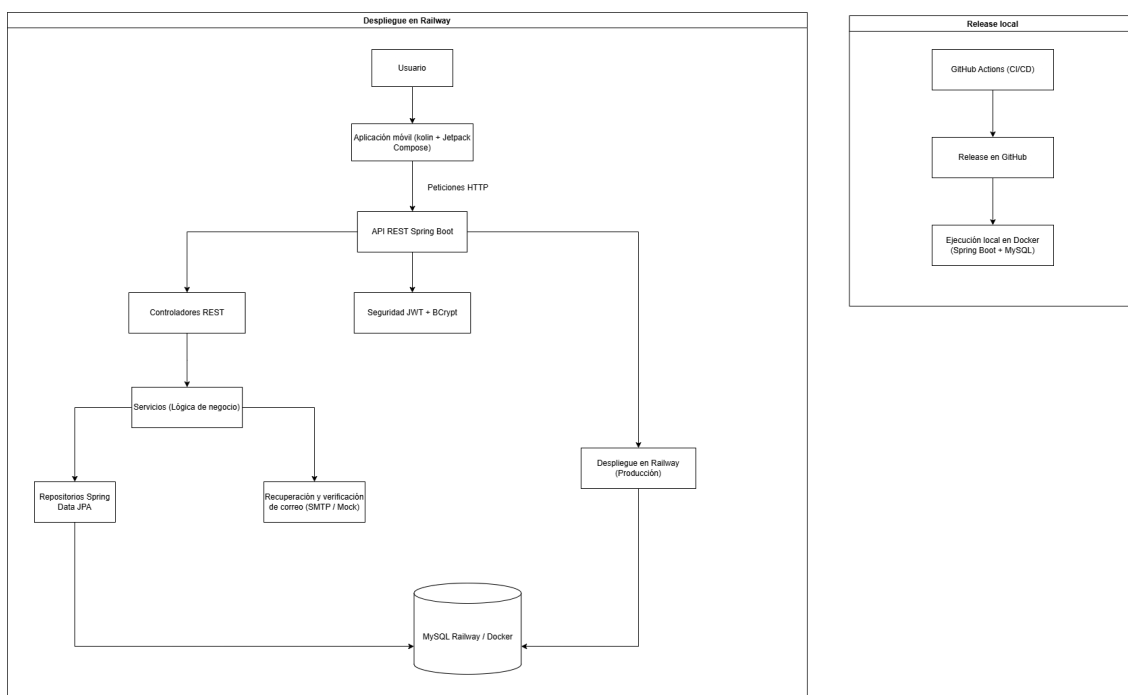


Figura 9.14 - Diagrama de arquitectura y ejecución de la aplicación

Con el diseño planteado en esta figura, el sistema queda separado de forma lógica mediante variables de entorno en una versión de pruebas ejecutada en docker de forma local con servicio de verificación mock y otra versión ejecutada en Railway para producción real con servicio de verificación mediante SMTP.

10. PRUEBAS, VALIDACIÓN Y CONTROL

Las pruebas del proyecto tienen como objetivo comprobar que el prototipo desarrollado funciona correctamente y que las funcionalidades principales cumplen con los requisitos definidos en fases anteriores. Al tratarse de una aplicación cliente-servidor, las pruebas se han organizado en distintos niveles, que incluyen pruebas automatizadas del backend, pruebas manuales de la API REST, pruebas funcionales sobre la aplicación móvil y validación general de requisitos.

Este proceso de validación se ha centrado especialmente en los flujos principales del sistema, es decir, registro de usuarios, autenticación mediante JWT, configuración del perfil musical, búsqueda de músicos, sistema de matching, bloqueo de usuarios y mensajería privada. Además, se han realizado comprobaciones sobre la seguridad de los endpoints protegidos y sobre el correcto intercambio de datos entre la aplicación Android y el backend.

10.1 Plan de pruebas

El plan de pruebas se diseñó con el objetivo de verificar tanto el comportamiento técnico del backend como el funcionamiento real de la aplicación desde el punto de vista del usuario. Para ello, se definieron diferentes tipos de pruebas en función de la parte del sistema que se quería comprobar

Tipo de prueba	Herramienta utilizada	Objetivo principal
Pruebas de arranque del backend	JUnit / Spring Boot Test	Comprobar que el contexto de Spring Boot carga correctamente el perfil de pruebas.
Pruebas de seguridad	JUnit / MockMvc	Verificar que los endpoints protegidos requieren autenticación mediante JWT.
Pruebas de tokens	JUnit	Comprobar la generación y validación de access tokens y refresh tokens.
Pruebas de servicios	JUnit / Mockito	Validar las principales reglas de negocio del backend, como chats, mensajes, perfiles y matches.
Pruebas de endpoints	Postman	Comprobar manualmente las respuestas de la API REST.
Pruebas funcionales	Emulador Android / dispositivo físico	Verificar que el usuario puede completar los flujos principales desde la aplicación.
Pruebas de integración	Android + Backend	Comprobar que el cliente móvil consume correctamente los datos enviados por el servidor.
Control de cambios	Git / GitHub	Registrar la evolución del proyecto y las correcciones realizadas.

Tabla 10.1 - Plan de pruebas y herramientas utilizadas

Para las pruebas automatizadas se utilizó el perfil test, configurado con una base de datos H2 en memoria. Esto permite ejecutar los tests sin modificar los datos reales de MySQL y sin depender de servicios externos como el correo SMTP. En cambio, las pruebas manuales con Postman y las pruebas funcionales de Android se realizaron sobre el backend local o desplegado, dependiendo del caso concreto.

```

PS C:\Users\sergi\Desktop\musicmatch-app\backend> mvn test
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::staticFieldBase has been called by com.google.inject.internal.aop.HiddenClassDefiner (file:/C:/Users/sergi/Desktop/Edito
res/apache-maven-3.9.11/lib/guice-5.1.0-classes.jar)
WARNING: Please consider reporting this to the maintainers of class com.google.inject.internal.aop.HiddenClassDefiner
WARNING: sun.misc.Unsafe::staticFieldBase will be removed in a future release
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.musicmatch:backend >-----
[INFO] Building 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ backend ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.14.1:compile (default-compile) @ backend ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ backend ---
[INFO] Copying 1 resource from src/test/resources to target/test-classes
[INFO]
[INFO] --- compiler:3.14.1:testCompile (default-testCompile) @ backend ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- surefire:3.5.5:test (default-test) @ backend ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----

```

Figura 10.1 - Prueba de ejecución del perfil de pruebas del backend

Como se observa en la figura anterior, la ejecución general de las pruebas finalizó correctamente. Esto confirma que la configuración inicial del entorno de pruebas es válida y que el backend puede arrancar con el perfil específico de test.

10.2 Resultados de las pruebas

10.2.1 Pruebas automatizadas del backend

En primer lugar, se realizaron pruebas automatizadas sobre el backend. Estas pruebas permiten comprobar que determinados comportamientos del sistema funcionan de forma repetible, sin depender únicamente de pruebas manuales. En el caso de este proyecto, se han incluido pruebas de carga del contexto, seguridad de endpoints, generación de tokens JWT y validaciones internas de servicios.

ID	Prueba realizada	Resultado esperado	Resultado obtenido
T01	Carga del contexto de Spring Boot	El backend arranca correctamente con el perfil test	Correcto
T02	Endpoint público /api/auth/ping	Devuelve ok sin necesidad de token	Correcto
T03	Endpoint protegido sin token	Devuelve error 401	Correcto
T04	Endpoint protegido con token invalido	Devuelve error 401	Correcto
T05	Generación de access token	El token es válido y de tipo access	Correcto
T06	Generación de refresh token	El token es válido y de tipo refresh	Correcto
T07	Enviar mensaje vacío	El servidor rechaza el mensaje	Correcto
T08	Iniciar chat consigo mismo	El servicio rechaza la operación	Correcto
T09	Perfil incompleto para matching	El sistema solicita completar el perfil	Correcto
T10	Subida de avatar con archivo vacío o formato no válido	El sistema rechaza el archivo	Correcto

Tabla 10.2 - Resultados de las pruebas del backend

Estas pruebas permiten validar aspectos importantes del backend. Por un lado, se comprueba que las rutas públicas y privadas se comportan correctamente. Por otro lado, se verifica que los servicios aplican reglas de negocio, como impedir mensajes vacíos, evitar chats con uno mismo o no mostrar candidatos de matching cuando el perfil musical no está completo.

```

2026-05-04T19:14:04.500+02:00 INFO 22696 --- [main] r$InitializeUserDetailsManagerConfigurer : Global AuthenticationManager configured
with UserDetailsService bean with name inMemoryUserDetailsManager
2026-05-04T19:14:04.651+02:00 INFO 22696 --- [main] c.m.backend.BackendApplicationTests : Started BackendApplicationTests in 1.04
4 seconds (process running for 10.286)
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.075 s -- in com.musicmatch.backend.BackendApplicationTests
[INFO] Running com.musicmatch.backend.ChatServiceTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.455 s -- in com.musicmatch.backend.ChatServiceTest
[INFO] Running com.musicmatch.backend.JwtUtilTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.098 s -- in com.musicmatch.backend.JwtUtilTest
[INFO] Running com.musicmatch.backend.MatchServiceTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.134 s -- in com.musicmatch.backend.MatchServiceTest
[INFO] Running com.musicmatch.backend.ProfileServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.253 s -- in com.musicmatch.backend.ProfileServiceTest
[INFO] Results:
[INFO] Tests run: 16, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 13.600 s
[INFO] Finished at: 2026-05-04T19:14:05+02:00
[INFO]
PS C:\Users\sergi\Desktop\musicmatch-app\backend>

```

Figura 10.2 - Demostración de éxito de pruebas de JUnit

Los resultados obtenidos muestran que el backend presenta una base estable para el funcionamiento del prototipo. Además, las pruebas automatizadas ayudan a detectar posibles errores si en el futuro se modifica alguna parte del sistema, ya que permiten volver a comprobar rápidamente las funcionalidades principales.

10.2.2 Pruebas de la API REST con Postman

Además de las pruebas automatizadas, se realizaron pruebas manuales de la API REST mediante Postman. Estas pruebas permiten comprobar el comportamiento real de los endpoints, enviando peticiones similares a las que realiza la aplicación Android.

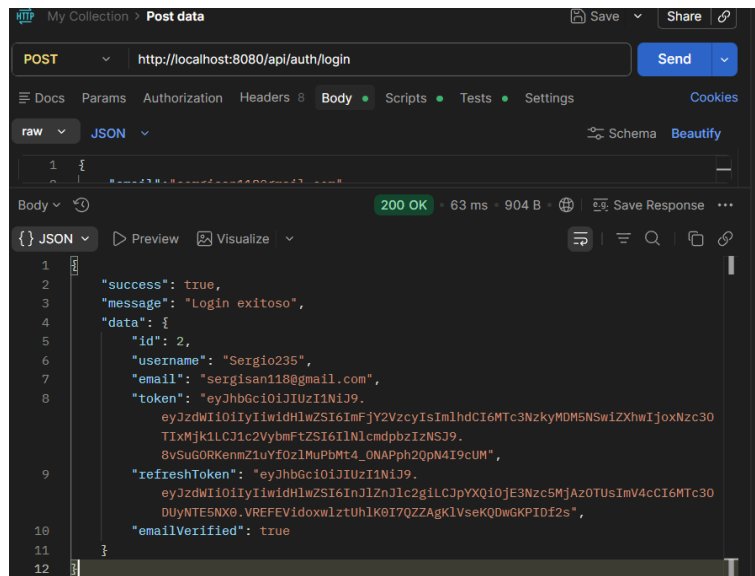
Las pruebas se organizan en módulos funcionales como pueden ser la autenticación, el perfil, la búsqueda, el matching, los bloqueos y los chats.

Módulo	Endpoint	Validación realizada
Autenticación	POST /api/auth/register	Registro de usuario con email, usuario y contraseña.
Autenticación	POST /api/auth/login	Inicio de sesión y obtención de access token y refresh token.
Autenticación	POST /api/auth/refresh	Renovación del access token.
Autenticación	GET /api/auth/me	Obtención de los datos del usuario autenticado.

Autenticación	GET /api/auth/verification-status	Consulta del estado de verificación del correo.
Perfil	GET /api/profile/musical-options	Obtención de instrumentos y estilos musicales.
Perfil	GET /api/profile/cities	Obtención de ciudades disponibles.
Perfil	PATCH /api/profile	Actualización parcial del perfil musical.
Perfil	POST /api/profile/avatar	Subida de imagen de perfil.
Búsqueda	POST /api/search/profiles	Búsqueda de músicos por nombre, estilos o instrumentos.
Matching	GET /api/matches/candidates	Obtención de candidatos compatibles.
Matching	POST /api/matches/swipe	Aceptación o rechazo de candidatos.
Bloqueos	POST /api/matches/block	Bloqueo de un usuario.
Bloqueos	POST /api/matches/unblock	Desbloqueo de un usuario.
Bloqueos	GET /api/matches/blocked	Consulta de usuarios bloqueados.
Chats	POST /api/chats/request-or-get	Creación o recuperación de un chat.
Chats	POST /api/chats/{chatId}/accept	Aceptación de solicitud de chat.
Chats	POST /api/chats/message	Envío de mensaje.
Chats	GET /api/chats/{chatId}/messages	Consulta de mensajes ordenados.

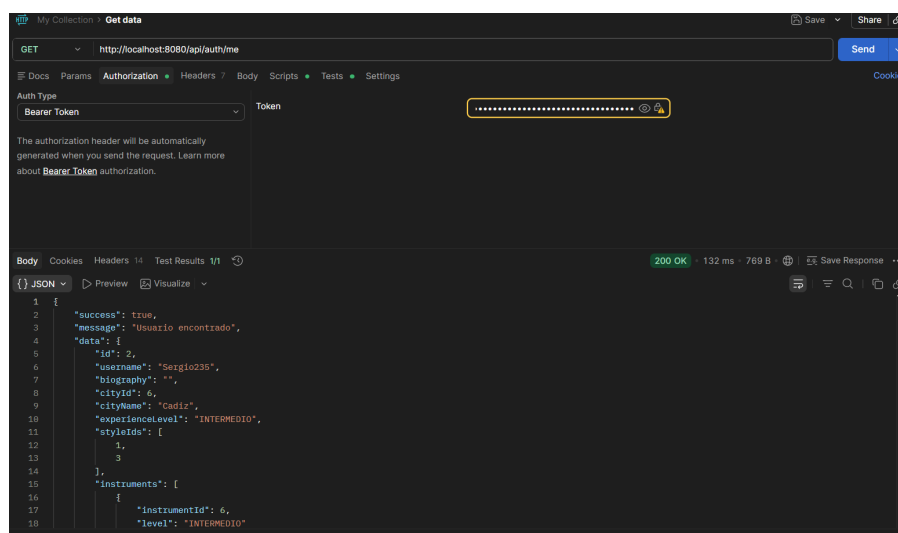
Tabla 10.3 - Estructura de pruebas de la Api REST

En primer lugar, se probó el inicio de sesión. Esta prueba es esencial porque permite comprobar que el backend valida las credenciales del usuario y devuelve los tokens necesarios para mantener la sesión activa en la aplicación móvil.



La respuesta obtenida confirma que el sistema genera correctamente los tokens tras un inicio de sesión válido. El access token se utiliza para acceder a las rutas protegidas, mientras que el refresh token permite renovar la sesión sin obligar al usuario a iniciar sesión de nuevo constantemente.

A continuación, se comprobó el acceso a un endpoint protegido mediante la cabecera Authorization. Esta prueba permite verificar que la API no permite acceder a información privada sin un token válido.



El resultado demuestra que el backend reconoce el token JWT y permite acceder a la información del usuario autenticado. También se realizaron pruebas sin token o con un token incorrecto, obteniendo una respuesta de error, lo que confirma la protección de las rutas privadas.

Después se probaron los endpoints relacionados con el sistema de matching. Esta parte es especialmente importante porque constituye una de las funcionalidades principales del proyecto, ya que permite mostrar candidatos compatibles a partir de la información musical de los perfiles.

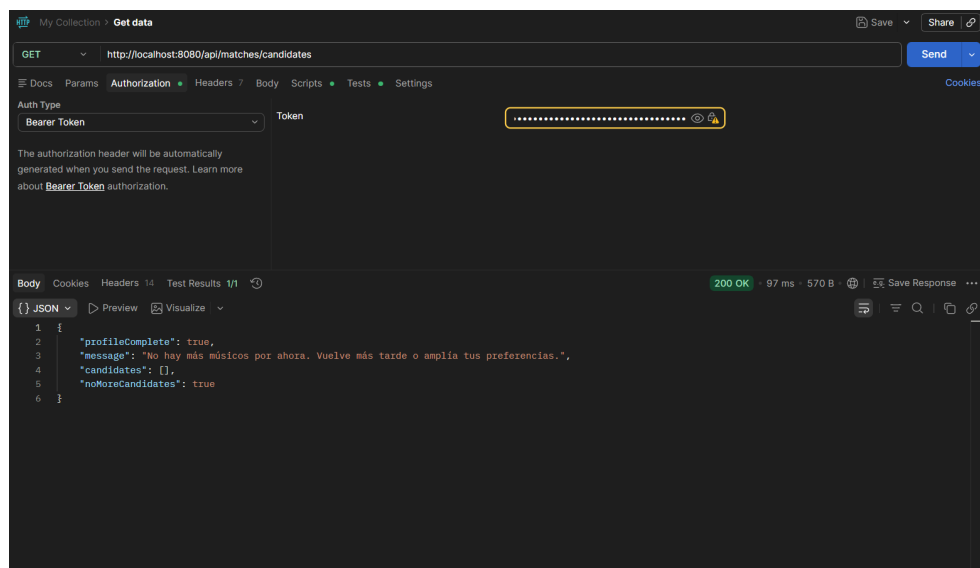


Figura 10.5 - Obtención de candidatos del usuario desde Postman

Esta prueba confirma que el backend aplica la lógica de compatibilidad entre perfiles y devuelve una respuesta adecuada al cliente. En caso de que el perfil no esté completo o no existan candidatos disponibles, el sistema también devuelve un mensaje controlado, evitando errores inesperados en la aplicación móvil.

10.2.3 Pruebas funcionales del cliente móvil

Una vez comprobado el backend y la API REST, se realizaron pruebas funcionales sobre la aplicación Android. Estas pruebas se centraron en validar que el usuario podía completar los flujos principales desde la interfaz, sin necesidad de interactuar directamente con Postman.

Flujo probado	Resultado esperado	Resultado obtenido
Registro de usuario	El usuario puede registrarse y recibir el flujo de verificación.	Correcto
Inicio de sesión	El usuario puede iniciar sesión con credenciales válidas	Correcto
Configuración de perfil	El usuario puede seleccionar la ciudad, estilos, instrumentos y nivel	Correcto
Edición de perfil	Los cambios se guardan y se muestran al volver a cargar la pantalla	Correcto
Búsqueda de músicos	Se muestran perfiles filtrados según los criterios indicados	Correcto
Matching	Se muestran candidatos compatibles se permite rechazar o aceptar	Correcto
Bloqueo de usuarios	El usuario bloqueado deja de aparecer en interacciones	Correcto
Chat	Se pueden enviar y consultar mensajes privados	Correcto
Gestión de errores	Se muestran mensajes comprensibles ante errores de red o sesión	Correcto

Tabla 10.4 - Resultado de pruebas funcionales en el cliente Android

En primer lugar, se comprobó que el usuario podía acceder a su perfil musical y visualizar o modificar los datos principales. Esta prueba permite validar la integración entre la interfaz móvil, los ViewModels, los repositorios y los endpoints del backend.

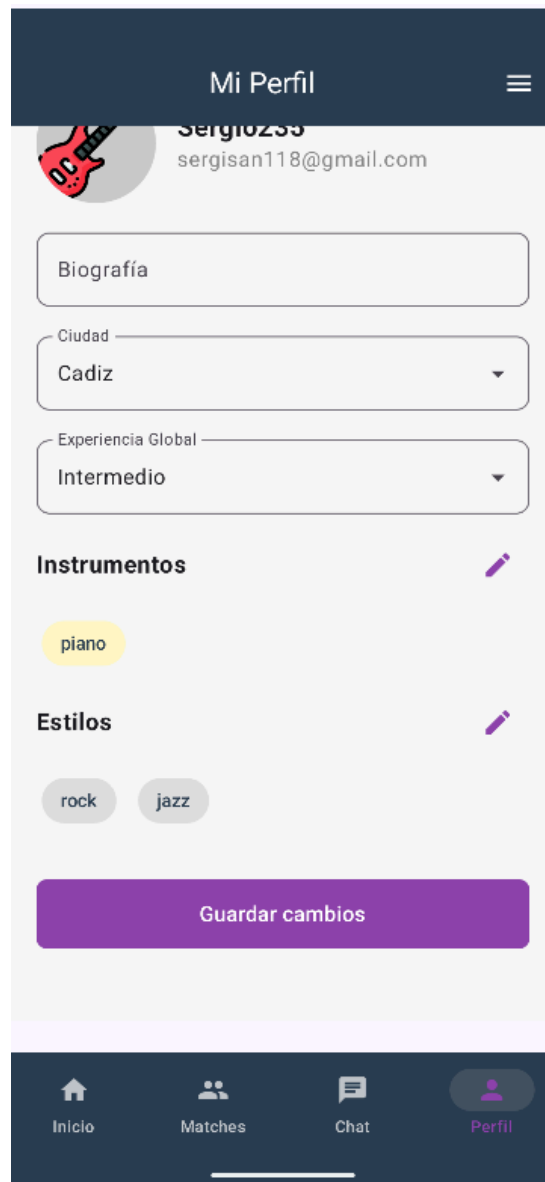


Figura 10.6 - Prueba de modificación de perfil desde la aplicación

El resultado obtenido demuestra que la aplicación móvil consume correctamente la información enviada por el backend y la representa de forma comprensible para el usuario. Además, se valida que los datos del perfil musical influyen directamente en funcionalidades como la búsqueda y el matching.

Finalmente, se probó el sistema de mensajería privada. Para ello, se verificó que un chat activo permitía enviar mensajes y recuperarlos posteriormente en orden cronológico.

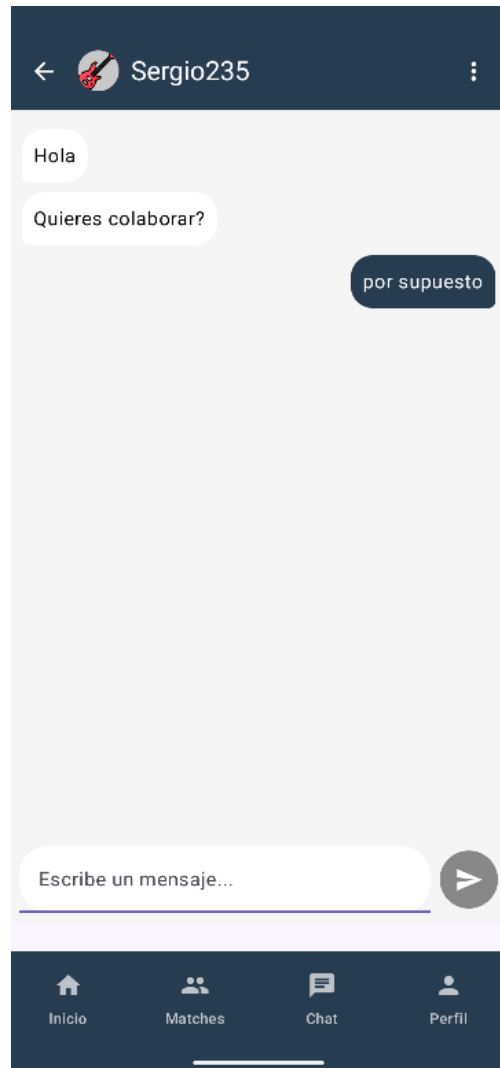


Figura 10.7 - Prueba de uso de chat en tiempo real con otro usuario

Esta prueba confirma que la comunicación entre usuarios funciona correctamente y que los mensajes se almacenan y recuperan desde el backend sin pérdida de información. También permite comprobar que el cliente actualiza la interfaz de forma adecuada tras enviar o recibir mensajes.

10.3 Gestión de incidencias y cambios

Durante el desarrollo y la validación del prototipo se detectaron distintas incidencias relacionadas principalmente con la autenticación, la conexión entre cliente y servidor, el despliegue, la gestión de estados y el control de errores. Estas incidencias se corrigieron de forma progresiva mediante revisión de código, pruebas manuales, pruebas automatizadas y control de versiones con Git.

Incidencia	Causa probable	Solución aplicada
La app Android no conectaba con el backend local	Uso incorrecto de localhost desde el emulador o ip inválida	Se configuró la URL base con la ip 10.0.2.2 correspondiente al acceso a localhost desde el emulador de Android Studio.
Acceso denegado a endpoints protegidos	No se enviaba correctamente la cabecera Authorization	Se centralizó el uso del token en los repositorios del cliente usando el prefijo Bearer.
Sesión no válida o token caducado	Expiración del token de acceso	Se implementó el uso de un refresh token para renovar la sesión.
Problemas durante la verificación de correo	Diferencias entre el entorno local y SMTP real	Se añadió un servicio mock de correo para pruebas locales y un servicio SMTP para producción, ambos implementando una interfaz común.
Error al subir avatar	Archivo vacío, formato incorrecto o tamaño superior	Se añadieron validaciones de tipo y tamaño máximo.
Chats creados en estado incorrecto	Gestión incompleta de estados PENDING y ACTIVE	Se revisó la lógica de solicitud, aceptación y rechazo de chats.
Usuarios bloqueados seguían apareciendo en algunas interacciones	Faltaba comprobar el bloqueo en algunos flujos	Se añadió comprobación de bloqueo en búsqueda, matching y chats.
Errores poco claros para el usuario	Mensajes técnicos enviados directamente desde el backend	Se gestionaron errores mediante safeApiCall y NetworkErrorHandler.

Tabla 10.5 - Incidencias y soluciones aplicadas

El control de cambios se realizó mediante Git y GitHub, utilizando commits frecuentes para registrar la evolución del proyecto. Esto permitió mantener un historial de las funcionalidades implementadas, así como de las correcciones realizadas durante la fase de pruebas.

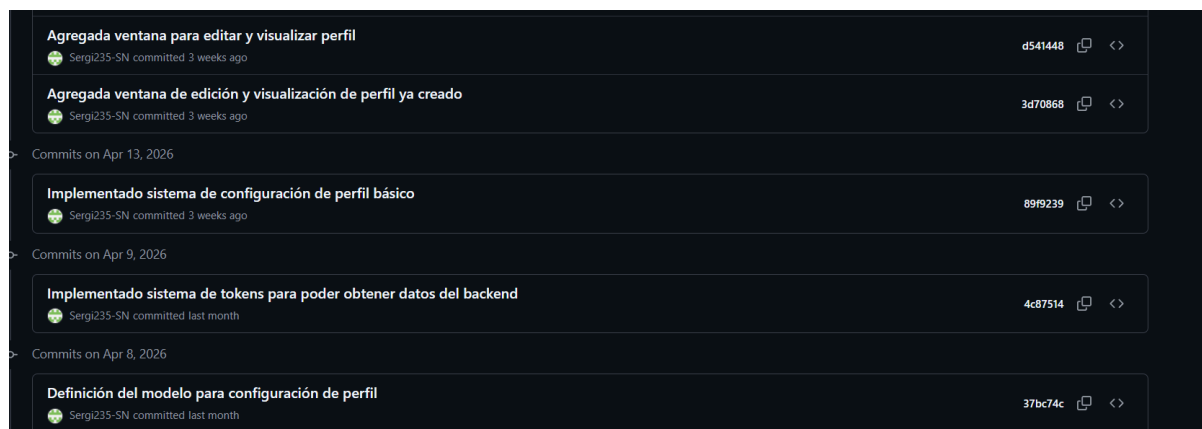


Figura 10.8 - Historial de commits y resolución de incidencias clave

Esta evidencia permite comprobar que el desarrollo no se realizó de forma aislada o desordenada, sino siguiendo un proceso de control de versiones que facilita la trazabilidad de los cambios y la recuperación de versiones anteriores en caso de error.

10.4 Validación con usuarios y verificación de requisitos

La validación final del prototipo se realizó comprobando si las funcionalidades implementadas respondían a los requisitos definidos en el apartado de análisis. Esta validación se centró especialmente en los requisitos principales del MVP:

Requisito	Funcionalidad validada	Estado
RF1	Registro de nuevos usuarios mediante email, nombre de usuario y contraseña	Cumplido
RF2	Inicio de sesión seguro mediante credenciales y JWT	Cumplido
RF3	Recuperación de contraseña mediante correo y token temporal	Cumplido
RF4	Selección de instrumentos musicales y estilos	Cumplido
RF5	Indicación del nivel de experiencia por instrumento	Cumplido
RF6	Edición de biografía y foto de perfil	Cumplido
RF7	Generación de coincidencias según afinidad musical y ciudad	Cumplido

RF8	Control de estados de match y chat	Cumplido
RF9	Aviso visual de nuevas coincidencias	Parcialmente cumplido
RF10	Visualización de músicos registrados	Cumplido
RF11	Filtrado de búsqueda por instrumentos y estilos	Cumplido
RF12	Búsqueda de músicos por nombre	Cumplido
RF13	Envío y recepción de mensajes privados	Cumplido
RF14	Visualización de mensajes en orden cronológico	Cumplido
RF15	Gestión de roles de usuarios	No incluido en el prototipo funcional
RF16	Supervisión de contenido por administrador	No incluido en el prototipo funcional
RF17	Eliminación o bloqueo de cuentas	Parcialmente cumplido

Tabla 10.6 - Verificación de cumplimiento de requisitos funcionales

En relación con los requisitos no funcionales, el prototipo cumple especialmente con los relacionados con seguridad, mantenibilidad, compatibilidad y usabilidad básica. La seguridad se ha trabajado mediante JWT, contraseñas cifradas con BCrypt y separación entre endpoints públicos y protegidos. La mantenibilidad se ha favorecido mediante una arquitectura en capas en el backend y el uso del patrón MVVM en el cliente móvil.

Por otro lado, algunos requisitos quedan parcialmente validados debido al alcance académico del proyecto. Por ejemplo, la disponibilidad real del sistema depende del entorno de despliegue y no se ha realizado una prueba de carga prolongada con muchos usuarios simultáneos. Del mismo modo, las funciones avanzadas de administración quedan planteadas como una línea futura, ya que el MVP se ha centrado principalmente en la experiencia del músico.

En conjunto, las pruebas realizadas permiten confirmar que el prototipo cumple con las funcionalidades principales previstas y que la arquitectura elegida resulta adecuada para seguir ampliando el sistema en futuras versiones.

10.5 Limitaciones detectadas durante las pruebas

Aunque los resultados obtenidos fueron satisfactorios para el alcance del MVP, durante la fase de pruebas se identificaron algunas limitaciones que podrían mejorarse en versiones futuras del proyecto.

En primer lugar, la validación con usuarios se realizó sobre un grupo reducido, por lo que sería recomendable ampliarla con más perfiles de músicos para obtener una valoración más representativa. También sería conveniente realizar pruebas de carga sobre el sistema de matching, especialmente con una base de datos más amplia y con mayor número de perfiles.

Otra limitación es que el sistema de notificaciones no se ha implementado como notificación push real, sino como aviso visual dentro de la propia aplicación. En una versión futura, se podría integrar Firebase Cloud Messaging para avisar al usuario de nuevos matches o mensajes incluso cuando no tenga la aplicación abierta.

Finalmente, las funciones de administración se han planteado a nivel de requisitos, pero no se han desarrollado completamente en el MVP. Por ello, sería recomendable añadir en futuras versiones un panel de administración para la gestión de usuarios, revisión de contenido y control de incidencias.

A pesar de estas limitaciones, el prototipo desarrollado permite demostrar la viabilidad técnica de MusicMatch y validar los flujos esenciales de la aplicación.

11. CONCLUSIONES Y TRABAJO FUTURO

Una vez finalizado el desarrollo y validación del prototipo, este apartado recoge las conclusiones generales del proyecto, la valoración personal del proceso realizado y las principales líneas futuras de desarrollo. A diferencia de la introducción, donde se plantea el contexto y la necesidad inicial, este capítulo tiene como finalidad cerrar la memoria valorando el grado de cumplimiento de los objetivos, el aprendizaje obtenido y las posibilidades de evolución de la aplicación.

11.1 Conclusiones generales

El desarrollo de MusicMatch ha permitido completar un proyecto software funcional, organizado y coherente con los objetivos establecidos al inicio de la memoria. A través del proceso realizado, se ha conseguido transformar una necesidad detectada en el ámbito musical en una solución tecnológica concreta, basada en una aplicación móvil conectada a un backend y apoyada en una base de datos relacional.

El resultado obtenido demuestra que la propuesta es viable desde el punto de vista técnico. El prototipo permite realizar los flujos principales previstos, es decir, registro de usuarios, inicio de sesión, configuración y edición del perfil musical, exploración de perfiles, generación de coincidencias y comunicación básica entre usuarios. Estas funcionalidades representan el núcleo del MVP y permiten comprobar que la idea principal de la aplicación puede llevarse a la práctica.

También se ha conseguido desarrollar una estructura técnica adecuada para futuras ampliaciones. La separación entre cliente móvil, servidor y base de datos facilita el mantenimiento del sistema y permite incorporar nuevas funcionalidades de forma progresiva. Además, el uso de una API REST, una arquitectura backend organizada por capas y una estructura móvil basada en MVVM contribuye a que el proyecto tenga una base clara y escalable.

Las pruebas realizadas han permitido validar el funcionamiento general del sistema y detectar algunas limitaciones propias de un prototipo académico. Entre ellas se encuentran la ausencia de mensajería en tiempo real, la falta de un panel de administración completo, la no implementación de notificaciones push reales y la necesidad de validar el sistema con un mayor número de usuarios. Sin embargo, estas limitaciones no impiden que el proyecto cumpla su finalidad principal, ya que el objetivo era desarrollar una versión funcional inicial y no una aplicación comercial definitiva.

En conjunto, puede concluirse que MusicMatch cumple con los objetivos planteados para el proyecto final del ciclo. Se ha desarrollado una aplicación multiplataforma en un entorno realista, aplicando conocimientos de análisis, diseño, programación, bases de datos, seguridad, despliegue y pruebas. Además, el proyecto deja una base sólida sobre la que podrían construirse nuevas versiones más completas.

11.2 Valoración personal

La realización de este proyecto ha supuesto una experiencia especialmente enriquecedora, ya que ha permitido abordar el desarrollo de una aplicación desde una perspectiva global. No se ha tratado únicamente de implementar código, sino de comprender y aplicar todas las fases que forman parte de un proyecto software.

A nivel técnico, el proyecto ha permitido consolidar conocimientos adquiridos durante el ciclo de Desarrollo de Aplicaciones Multiplataforma. El trabajo con tecnologías como Android Studio, Kotlin, Spring Boot, MySQL, Docker, Git/GitHub, JWT y Railway ha permitido acercar el desarrollo del prototipo a un entorno profesional. Además, la integración entre cliente móvil y backend ha sido una de las partes más importantes del proceso, ya que ha exigido coordinar correctamente la comunicación entre distintas capas del sistema.

Durante el desarrollo también han surgido dificultades, especialmente relacionadas con la autenticación, la configuración del entorno, la conexión entre la aplicación móvil y el servidor, el despliegue y la validación de los distintos flujos. Estas dificultades han formado parte del aprendizaje del proyecto, ya que han obligado a analizar errores, probar soluciones alternativas y mejorar progresivamente la estructura del sistema.

Desde una perspectiva personal, el proyecto ha permitido desarrollar autonomía, capacidad de organización y resolución de problemas. También ha sido motivador trabajar sobre una idea con una temática cercana y con posibilidades reales de evolución. MusicMatch no se limita a ser una práctica académica, sino que plantea una solución que podría seguir creciendo si se amplía su desarrollo en el futuro.

Por todo ello, la valoración final del proyecto es positiva. El resultado obtenido refleja el aprendizaje adquirido durante el ciclo y demuestra la capacidad de desarrollar una aplicación completa, integrando diferentes tecnologías y tomando decisiones técnicas justificadas.

11.3 Líneas futuras de desarrollo

Aunque el prototipo desarrollado cumple con los objetivos principales del MVP, MusicMatch presenta diversas posibilidades de mejora que permitirían convertir la aplicación en una plataforma más completa, escalable y cercana a un producto real.

Una de las líneas futuras más relevantes sería la incorporación de WebSockets para la mensajería en tiempo real. En la versión actual, la comunicación entre usuarios se plantea mediante peticiones al servidor, lo que permite enviar y consultar mensajes, pero no ofrece una experiencia completamente instantánea. La integración de WebSockets permitiría que los mensajes se recibieran en tiempo real, sin necesidad de actualizar manualmente la conversación. Además, esta mejora permitiría incorporar funcionalidades como indicadores de escritura, confirmaciones de lectura, estado de conexión de los usuarios o actualización automática de las conversaciones.

Otra mejora importante sería la gestión de grupos musicales. Actualmente, la aplicación se centra en la conexión entre usuarios individuales, pero una evolución natural sería permitir la creación de bandas o proyectos dentro de la plataforma. Esta funcionalidad podría incluir perfiles de grupo, descripción del proyecto, estilos musicales, instrumentos necesarios, roles de los miembros, solicitudes de entrada y mensajería grupal. De esta forma, MusicMatch no solo serviría para encontrar músicos compatibles, sino también para organizar proyectos musicales completos dentro de la propia aplicación.

También sería interesante evolucionar el sistema de matching mediante inteligencia artificial avanzada. En el prototipo actual, las coincidencias se basan en criterios definidos previamente, como instrumentos, estilos musicales, nivel de experiencia y ubicación. En futuras versiones, se podría implementar un sistema de recomendación más inteligente, capaz de aprender del comportamiento de los usuarios, de los matches aceptados o rechazados, de sus preferencias y de sus interacciones dentro de la aplicación. Esto permitiría ofrecer sugerencias más precisas y personalizadas, aumentando la probabilidad de generar conexiones útiles y colaboraciones reales.

Otra aplicación recomendable sería incorporar geolocalización real en lugar de depender únicamente de la selección manual de una ciudad entre varias opciones disponibles. En la versión actual, la ubicación del usuario se gestiona de forma simplificada mediante la elección de una ciudad, lo que resulta suficiente para el alcance del MVP, pero limita la precisión del sistema. En futuras versiones, la aplicación podría utilizar la ubicación real del dispositivo, siempre con el consentimiento del usuario, para calcular la proximidad entre músicos de forma más exacta. Esto permitiría mostrar perfiles cercanos, ordenar los resultados por distancia, establecer radios de búsqueda personalizados y mejorar la calidad del matching en función de la cercanía geográfica real. Además, esta funcionalidad resultaría especialmente útil en el ámbito musical, ya que la proximidad facilita la organización de ensayos, reuniones presenciales y colaboraciones estables.

Además, sería conveniente incorporar notificaciones push reales, por ejemplo para avisar al usuario cuando reciba un nuevo mensaje, una nueva coincidencia o una solicitud de contacto. Esta funcionalidad mejoraría la experiencia de uso, ya que permitiría mantener al usuario informado aunque la aplicación no estuviera abierta en ese momento.

Otra línea de mejora sería el desarrollo de un panel de administración completo, desde el que se pudieran gestionar usuarios, revisar contenido, bloquear cuentas, consultar incidencias y supervisar el funcionamiento general de la plataforma. Esta funcionalidad sería especialmente necesaria si la aplicación creciera y comenzara a ser utilizada por una comunidad más amplia.

Desde el punto de vista técnico, también se podrían realizar mejoras relacionadas con la escalabilidad y el rendimiento. Sería recomendable llevar a cabo pruebas de carga, optimizar consultas a la base de datos, mejorar el sistema de logs y monitorización, y reforzar el control de errores en producción. Estas mejoras permitirían preparar la aplicación para un mayor número de usuarios y facilitarían su mantenimiento a largo plazo.

Por último, MusicMatch podría ampliarse mediante integraciones con servicios externos relacionados con el ámbito musical, como calendarios de eventos, locales de ensayo, escuelas de música, salas de conciertos o plataformas de publicación musical. Estas integraciones permitirían aportar más valor a los usuarios y convertir la aplicación en un espacio más completo para la colaboración musical.

En definitiva, MusicMatch cuenta con una base funcional suficiente para seguir evolucionando. El prototipo desarrollado valida la idea principal y demuestra que la solución es técnicamente posible. Las líneas futuras planteadas permitirían avanzar hacia una aplicación más completa, con comunicación en tiempo real, gestión de proyectos musicales, recomendaciones inteligentes y herramientas de administración más avanzadas.

12. GLOSARIO

API REST

Interfaz de comunicación entre sistemas que permite intercambiar información mediante peticiones HTTP. En este proyecto se utiliza para que la aplicación móvil se comunique con el backend.

APK

Archivo instalable de una aplicación Android. Es el formato que permite distribuir e instalar la aplicación móvil en un dispositivo o emulador.

Backend

Parte del sistema encargada de gestionar la lógica de negocio, la seguridad, la autenticación, el acceso a la base de datos y la exposición de servicios mediante API.

Base de datos relacional

Sistema de almacenamiento de información organizado en tablas relacionadas entre sí mediante claves primarias y claves foráneas. En el proyecto se utiliza MySQL.

BCrypt

Algoritmo utilizado para cifrar contraseñas mediante hash. Permite almacenar las credenciales de forma segura sin guardar la contraseña original en texto plano.

Branch / Rama

Línea de desarrollo independiente dentro de un repositorio Git. Permite trabajar en nuevas funcionalidades sin modificar directamente la versión principal del proyecto.

Chat

Conversación privada entre dos usuarios de la aplicación. En MusicMatch se utiliza para que los músicos puedan comunicarse una vez establecida una coincidencia o solicitud aceptada.

CI/CD

Conjunto de prácticas de integración y despliegue continuo que permite automatizar tareas como pruebas, construcción del proyecto, generación de APK o creación de releases.

Cliente móvil

Aplicación Android que utiliza el usuario final. Se encarga de mostrar la interfaz, recoger las acciones del usuario y comunicarse con el backend.

Commit

Registro de cambios realizado en Git. Cada commit guarda una versión concreta del código y permite llevar un historial del desarrollo.

Controlador

Clase del backend encargada de recibir las peticiones HTTP de la API y delegar la lógica correspondiente a los servicios.

CORS

Mecanismo de seguridad que controla qué orígenes externos pueden realizar peticiones al backend. Se configura para permitir la comunicación entre el cliente y el servidor.

DTO

Objeto utilizado para transferir datos entre el backend y el cliente sin exponer directamente las entidades internas de la base de datos.

Docker

Herramienta que permite ejecutar servicios en contenedores. En el proyecto se utiliza para facilitar la ejecución de la base de datos MySQL y del entorno local.

Endpoint

Ruta concreta de una API a la que se puede realizar una petición. Por ejemplo, un endpoint puede servir para iniciar sesión, obtener perfiles o enviar mensajes.

Entidad

Clase Java que representa una tabla de la base de datos mediante JPA. Cada entidad contiene atributos y relaciones con otras entidades.

FK / Clave foránea

Campo de una tabla que hace referencia a la clave primaria de otra tabla. Permite establecer relaciones entre entidades.

Framework

Conjunto de herramientas y estructuras que facilitan el desarrollo de aplicaciones. En el proyecto se utiliza Spring Boot para el backend y Jetpack Compose para la interfaz móvil.

Frontend

Parte visible de una aplicación con la que interactúa el usuario. En este contexto corresponde principalmente a la aplicación móvil Android.

Git

Sistema de control de versiones utilizado para registrar los cambios del proyecto y mantener un historial del código.

GitHub

Plataforma online utilizada para alojar el repositorio del proyecto, gestionar ramas, revisar cambios y generar releases.

Hibernate

Implementación de JPA que permite trabajar con bases de datos relacionales mediante objetos Java, reduciendo la necesidad de escribir consultas SQL manuales.

HTTP

Protocolo de comunicación utilizado para enviar y recibir peticiones entre cliente y servidor.

HTTPS

Versión segura del protocolo HTTP. Utiliza cifrado para proteger la información transmitida entre cliente y servidor.

IDE

Entorno de desarrollo integrado. Es una herramienta que facilita la programación, compilación, ejecución y depuración del código. En el proyecto se utilizan Android Studio e IDEs compatibles con Java/Spring Boot.

JPA

Especificación de Java que permite mapear clases como entidades y gestionar la persistencia de datos en una base de datos relacional.

JSON

Formato ligero de intercambio de datos utilizado en la comunicación entre la aplicación móvil y el backend.

JWT

Token utilizado para autenticar usuarios en una API. En MusicMatch permite acceder a endpoints protegidos sin mantener sesiones en el servidor.

Kotlin

Lenguaje de programación utilizado para desarrollar la aplicación móvil Android.

Match

Coincidencia entre dos perfiles musicales. En MusicMatch se genera cuando existe afinidad o aceptación mutua entre usuarios.

Matching

Proceso mediante el cual el sistema compara perfiles musicales y genera posibles coincidencias en función de criterios como instrumentos, estilos, nivel de experiencia y ciudad.

MVP

Producto mínimo viable. Versión inicial del proyecto que incluye las funcionalidades esenciales para demostrar la viabilidad de la solución.

MVVM

Patrón de arquitectura utilizado en el cliente móvil. Separa la interfaz de usuario, la lógica de presentación y los datos, facilitando el mantenimiento del código.

MySQL

Sistema gestor de base de datos relacional utilizado para almacenar usuarios, perfiles, instrumentos, estilos, matches, chats y mensajes.

ORM

Técnica que permite relacionar objetos de programación con tablas de una base de datos. En el proyecto se utiliza mediante JPA/Hibernate.

PendingRegistration

Entidad utilizada para almacenar temporalmente una solicitud de registro antes de crear el usuario definitivo tras la verificación del correo electrónico.

PK / Clave primaria

Campo que identifica de forma única cada registro dentro de una tabla.

Postman

Herramienta utilizada para probar manualmente los endpoints de la API REST y comprobar las respuestas del backend.

Profile

Entidad que representa el perfil musical de un usuario. Contiene información como biografía, ciudad, estilos, instrumentos, nivel de experiencia y foto de perfil.

Railway

Plataforma en la nube utilizada para desplegar el backend y la base de datos del proyecto en un entorno accesible.

Repositorio

Espacio donde se almacena el código fuente del proyecto junto con su historial de cambios. En el contexto de este proyecto se aloja en GitHub.

Retrofit

Librería utilizada en Android para consumir APIs REST de forma sencilla y transformar respuestas JSON en objetos Kotlin.

Spring Boot

Framework de Java utilizado para desarrollar el backend del proyecto, crear la API REST, gestionar la seguridad y conectar con la base de datos.

Spring Data JPA

Módulo de Spring que simplifica el acceso a datos mediante repositorios y entidades JPA.

Spring Security

Módulo de Spring utilizado para configurar la autenticación, autorización y protección de endpoints del backend.

Swipe

Acción mediante la cual un usuario acepta o rechaza un perfil dentro del sistema de matching.

Token

Cadena de texto utilizada para identificar o validar una acción. En el proyecto se utilizan tokens JWT, tokens de verificación de correo y tokens de recuperación de contraseña.

Trello

Herramienta de organización utilizada para planificar tareas, fases y seguimiento del desarrollo del proyecto.

User

Entidad que representa la cuenta del usuario dentro de la aplicación. Almacena datos de autenticación como email, contraseña cifrada, nombre de usuario y estado de verificación.

WebSockets

Tecnología de comunicación bidireccional en tiempo real entre cliente y servidor. No forma parte del MVP, pero se plantea como mejora futura para la mensajería instantánea.

Wireframe

Boceto inicial de una pantalla o interfaz. Se utiliza para representar la estructura y disposición de elementos antes del diseño visual final.

13. REFERENCIAS

Caules, C. Á. (2025, 3 marzo). *Spring Boot ¿Qué es y cómo funciona?* Arquitectura Java.

<https://www.arquitecturajava.com/spring-boot-que-es/>

Spring boot. (s. f.). Spring Boot. <https://spring.io/projects/spring-boot>

Introduction. (s. f.). Retrofit. <https://square.github.io/retrofit/>

GeeksforGeeks. (2025, 18 febrero). *Introduction to Retrofit in Android.* GeeksforGeeks.

<https://www.geeksforgeeks.org/android/introduction-retrofit-2-android-set-1/>

Aprende jetpack compose. (s. f.). OpenCompose.

<https://www.jetpackcompose.pro/home/guide>

Railway. (2026, 12 mayo). *Deploy a Spring Boot App.* Deploy A Spring Boot App | Railway

Guides. <https://docs.railway.com/guides/spring-boot>

Descripción de GitHub Actions - Documentación de GitHub. (s. f.). GitHub Docs.

<https://docs.github.com/es/actions/get-started/understand-github-actions>

Gomez, C., & Gomez, C. (2024, 3 diciembre). *Qué es la arquitectura cliente servidor y cómo funciona.* Daemon4.

<https://www.daemon4.com/empresa/noticias/arquitectura-cliente-servidor/>

Jakubiak, N. (2026, 27 febrero). Tutorial de JUnit: Configuración, escritura y ejecución de pruebas unitarias de Java. *Parasoft*.

<https://es.parasoft.com/blog/junit-tutorial-setting-up-writing-and-running-java-unit-tests/>